

# LeRobot SO-101 을 활용한 지능형 로봇 팔 제어



202155520 김도환 dowhan1225@pusan.ac.kr

202155616 천성민 russoll@pusan.ac.kr

202255540 김혜은 ww3700@pusan.ac.kr

지도교수 백윤주

---

## 목 차

|                                                     |    |
|-----------------------------------------------------|----|
| 1. 서론.....                                          | 1  |
| 1.1. 연구 목적과 문제 정의 .....                             | 1  |
| 1.2. 과제 요건과 평가 기준 .....                             | 1  |
| 1.3. 연구의 주요 내용 .....                                | 4  |
| 1.4. 구성원별 역할과 개발 일정 .....                           | 5  |
| 2. 연구 배경.....                                       | 7  |
| 2.1. 로봇 조작과 모방학습.....                               | 7  |
| 2.2. Action Chunking 기반 모방학습과 ACT.....              | 8  |
| 2.3. Vision-Language-Action 모델 .....                | 9  |
| 2.4. SmoVLA 모델.....                                 | 10 |
| 2.5. Flow Matching 기반 행동 생성.....                    | 12 |
| 2.6. 사전학습 SmoVLA Base 모델과 파인튜닝 전략 .....             | 13 |
| 2.7. 시각 기반 좌표 변환과 기하 보정.....                        | 14 |
| 2.8. 사람 개입 기반 모방학습과 복구 데이터 수집 .....                 | 16 |
| 2.9. Reward-Weighted Flow Matching 기반 행동 품질 학습..... | 17 |
| 3. 연구 내용 및 시스템 설계.....                              | 19 |
| 3.1. 개발 단계별 제어 방식 및 최종 구조로의 전환 .....                | 19 |
| 3.1.1. 초기 개발 환경 구축 및 단일 블록 기반 ACT 실험.....           | 19 |
| 3.1.2. ACT 기반 5블록 전체 작업 시도.....                     | 21 |
| 3.1.3. OpenCV-FSM-ACT 하이브리드 방식.....                 | 22 |
| 3.1.4. YOLO-1K 기반 전체 자동 제어 시도와 수집용 자동 접근으로의 전환 ..   | 30 |

---

|        |                               |    |
|--------|-------------------------------|----|
| 3.2.   | 최종 시스템 구조 .....               | 39 |
| 3.2.1. | 최종 시스템 개요 .....               | 40 |
| 3.2.2. | 시연 데이터 수집 구조 .....            | 41 |
| 3.2.3. | SmoIVLA 정책 학습 구조 .....        | 41 |
| 3.2.4. | 최종 정책 실행 구조 .....             | 42 |
| 3.2.5. | 수집 단계와 실행 단계의 모듈 차이 .....     | 43 |
| 3.3.   | 하드웨어와 소프트웨어 환경 .....          | 43 |
| 3.3.1. | 로봇 및 카메라 구성 .....             | 43 |
| 3.3.2. | Jetson 로봇 클라이언트와 GPU 서버 ..... | 44 |
| 3.3.3. | 소프트웨어 및 학습 환경 .....           | 44 |
| 3.3.4. | 카메라·관절·통신 설정 .....            | 45 |
| 3.4.   | 모델 특성을 고려한 데이터셋 설계 .....      | 46 |
| 3.4.1. | 모델 특성과 데이터 설계 원칙 .....        | 46 |
| 3.4.2. | 공간·색상 분포와 조작 순서 설계 .....      | 47 |
| 3.4.3. | 밀집도·방향·미세 위치 조건 설계 .....      | 48 |
| 3.4.4. | 파지 방향과 시연 궤적의 일관성 .....       | 48 |
| 3.4.5. | 제약 조건 기반 배치 시나리오 생성 .....     | 49 |
| 3.4.6. | 실제 배치 절차와 분포 점검 .....         | 50 |
| 3.5.   | 최종 시연 데이터 수집 시스템 .....        | 51 |
| 3.5.1. | YOLO 검출과 RBF 기반 자동 접근 .....   | 51 |
| 3.5.2. | 자동 접근 후 텔레오퍼레이션 전환 .....      | 52 |
| 3.5.3. | 파지·이송·배치 시연 .....             | 52 |
| 3.5.4. | Enter 입력과 Observe 자세 복귀 ..... | 53 |
| 3.5.5. | 다음 블록 검출과 반복 처리 .....         | 53 |

---

|         |                                   |    |
|---------|-----------------------------------|----|
| 3.5.6.  | 전체 과정의 단일 에피소드 기록 .....           | 54 |
| 3.6.    | 최종 데이터셋 구성과 품질 관리 .....           | 54 |
| 3.6.1.  | 정리·적층 데이터 구성 .....                | 54 |
| 3.6.2.  | 에피소드 및 프레임 수 .....                | 55 |
| 3.6.3.  | 데이터 정제와 제외 기준 .....               | 55 |
| 3.6.4.  | 위치·색상·회전 조건 분포 .....              | 56 |
| 3.7.    | 멀티태스크 학습 설계와 반복 실험 .....          | 56 |
| 3.7.1.  | 정리·적층 데이터 통합 .....                | 56 |
| 3.7.2.  | 학습 설정 변화 .....                    | 57 |
| 3.7.3.  | 중앙 배치 편향 분석 .....                 | 58 |
| 3.7.4.  | 체크포인트별 관찰 .....                   | 58 |
| 3.8.    | 작업별 자연어 지시문 설계 .....              | 59 |
| 3.8.1.  | 정리 작업 지시문 .....                   | 59 |
| 3.8.2.  | 적층 작업 지시문 .....                   | 59 |
| 3.8.3.  | 지시문 차별화와 작업 조건화 .....             | 59 |
| 3.9.    | 영상 전처리와 증강 .....                  | 60 |
| 3.9.1.  | 학습·실행 입력 전처리 일치 .....             | 60 |
| 3.9.2.  | 영상 크기 조정과 패딩 .....                | 60 |
| 3.9.3.  | 색상·밝기·선명도 증강 .....                | 60 |
| 3.9.4.  | 기하 증강 제외와 행동 라벨 정합성 .....         | 61 |
| 3.10.   | 최종 SmolVLA 모델 학습 설정 .....         | 61 |
| 3.10.1. | 기반 모델과 VLM 구성 .....               | 61 |
| 3.10.2. | 미세조정 방식의 비교와 전체 미세조정의 선택 근거 ..... | 61 |
| 3.10.3. | Action Chunk 설정 .....             | 64 |

---

|         |                                  |    |
|---------|----------------------------------|----|
| 3.10.4. | 학습 단계·손실·체크포인트 .....             | 64 |
| 3.11.   | 비동기 실행과 진단 도구 .....              | 65 |
| 3.11.1. | GPU 서버·로봇 클라이언트 구조 .....         | 65 |
| 3.11.2. | 관측 전송과 행동 청크 반환 .....            | 66 |
| 3.11.3. | 행동 대기열과 제어 주기 .....              | 66 |
| 3.11.4. | 관절 이동량 제한과 추종 감시 .....           | 66 |
| 3.11.5. | 입력 영상·행동 진단 .....                | 67 |
| 4.      | 연구 결과 분석 및 평가 .....              | 68 |
| 4.1.    | 평가 범위와 결과 해석 .....               | 68 |
| 4.2.    | 최종 모델의 작업 수행 결과 .....            | 68 |
| 4.2.1.  | 정리 작업 결과 .....                   | 68 |
| 4.2.2.  | 적층 작업 결과 .....                   | 69 |
| 4.2.3.  | HIL 추가 학습 이후의 변화 .....           | 70 |
| 4.3.    | 미세조정 방식·학습 손실·체크포인트별 결과 분석 ..... | 71 |
| 4.3.1.  | 미세조정 방식별 실물 관찰 .....             | 71 |
| 4.3.2.  | 성능 차이의 가능한 원인과 해석 범위 .....       | 72 |
| 4.3.3.  | 학습 손실과 실물 성능의 관계 .....           | 73 |
| 4.3.4.  | 체크포인트별 행동 관찰 .....               | 74 |
| 4.4.    | 실행 설정과 자원 사용 .....               | 74 |
| 4.5.    | 실패 유형과 개선 방향 .....               | 76 |
| 4.5.1.  | 정밀 파지 실패 .....                   | 76 |
| 4.5.2.  | 중앙 배치 편향 .....                   | 76 |
| 4.5.3.  | 배치 과정의 겹침과 위치 오차 .....           | 76 |
| 4.5.4.  | 적층 높이 누적 오차 .....                | 77 |

---

|        |                                                                            |    |
|--------|----------------------------------------------------------------------------|----|
| 4.5.5. | HIL 데이터의 불완전한 구간 .....                                                     | 77 |
| 4.6.   | 결과물 구성과 구현 범위 .....                                                        | 77 |
| 4.7.   | 과제 평가 항목에 따른 수행 결과 .....                                                   | 78 |
| 5.     | 결론 및 향후 연구 방향 .....                                                        | 89 |
| 5.1.   | 결론 .....                                                                   | 89 |
| 5.2.   | Correction-only HIL 기반 복구 데이터 학습오류! <b>책갈피가 정의되어 있지 않습니다.</b>              |    |
| 5.3.   | Reward-Weighted Flow Matching 기반 자율 행동 품질 최적화오류! <b>책갈피가 정의되어 있지 않습니다.</b> |    |
| 6.     | 참고 문헌 .....                                                                | 95 |

---

# 1. 서론

## 1.1. 연구 목적과 문제 정의

본 과제의 목적은 LeRobot SO-101 로봇 팔을 이용하여 무작위로 배치된 5개 블록을 지정 영역에 정리하거나 수직으로 적층하는 시스템을 구현하는 것이다. 로봇은 영상으로 작업 상태를 파악하고, 대상 선택부터 접근·파지·이송·배치까지의 과정을 연속적으로 수행해야 한다. 블록의 위치와 방향에 따라 접근 자세와 파지 동작을 조절해야 하며, 앞선 블록의 파지나 배치에서 발생한 오차는 이후 작업에도 영향을 미칠 수 있다. 따라서 개별 블록을 정확하게 조작하는 능력과 함께 5개 블록의 처리를 끝까지 이어가는 수행 안정성이 요구된다.

정리와 적층은 블록을 선택하고 집어 이동시키는 과정을 공유하지만, 최종 배치 방식이 다르다. 정리에서는 블록을 지정 영역 안에 배치해야 하고, 적층에서는 앞서 놓인 블록의 위치와 높이에 맞추어 다음 블록을 내려놓아야 한다. 또한 본 과제에서는 두 작업에 하나의 신경망 모델을 사용해야 하므로, 동일한 초기 장면에서도 작업 지시에 따라 서로 다른 배치 행동을 생성할 수 있어야 한다. 이를 위해 시각적 관측뿐 아니라 수행할 작업의 조건을 정책 입력에 반영하는 구조가 필요하다.

본 연구에서는 시각·언어 정보와 로봇 상태를 바탕으로 행동을 생성하는 SmolVLA를 조작 정책으로 채택하였다[1]. 상단 카메라와 손목 카메라의 영상, 관절 상태, 자연어 지시문을 입력하여 연속된 관절 목표값을 출력하도록 구성하였다. 두 작업의 시연을 함께 학습하고, 실행 시 동일한 체크포인트에 서로 다른 지시문을 전달하여 정리와 적층을 수행하도록 하였다.

연구 범위는 로봇과 카메라의 구동 환경 구축, 초기 정책 실험과 데이터 수집 방식의 개선, 시연 데이터의 수집·정제 및 멀티태스킹 학습을 포함한다. 또한 GPU 서버와 로봇 클라이언트를 연결하는 비동기 실행 시스템을 구현하고, 실물 환경에서 정리와 적층의 수행 결과와 실패 양상을 평가하였다.

## 1.2. 과제 요건과 평가 기준

본 과제는 1차 목표인 블록 정리와 2차 도전 과제인 수직 적층을 독립적으로 평가한다. 정리 작업을 평가한 후 블록을 다시 배치하여 적층 작업을 진행하며, 두 작업의 제한 시간은 별도로 부여된다. 각 작업의 초기 상태에서는 서로 다른 색의 블록 5개가 지정 영역 밖의 로봇 도달 가능 범위에 놓인다. 블록은 넓은 면이 바닥을 향하고 서로 겹치지

않도록 배치하되, 나란히 붙어 있는 배치는 허용된다. 초기 배치는 평가자가 무작위로 결정하며 모든 팀에 동일하게 적용한다.

학습과 평가에는 배부된 4×4×2 cm 규격의 블록을 사용한다. 지정 영역은 경계선 안쪽을 기준으로 가로 200 mm, 세로 100 mm이며, 경계선의 두께는 최대 20 mm까지 허용된다. 지정 영역은 카메라를 제외한 로봇 끝을 기준으로 25 cm 떨어진 지점에 배치한다. 경기장은 배부된 체크보드를 사용하거나 그 위에 별도로 제작한 보드를 설치할 수 있으며, 보드 크기를 A3로 제한하지 않는다. 작업별 주요 평가 기준은 표 1과 같다.

표 1 과제 조건과 성공 판정 기준

| 항목       | 정리 작업                                      | 적층 작업                       |
|----------|--------------------------------------------|-----------------------------|
| 과제 구분    | 1 차 목표                                     | 2 차 도전 과제                   |
| 조작 목표    | 5 개 블록을 지정 영역 안에 배치                        | 5 개 블록을 지정 영역에서 수직으로 적층     |
| 제한 시간    | 180 초                                      | 300 초                       |
| 점수 산정 기준 | 제한 시간 내 지정 영역에 배치한 블록 수                    | 제한 시간 내 적층한 블록 수와 적층 높이     |
| 성공 판정    | 지정 영역 안에 배치하며, 경계선에 걸친 경우 최대 20 mm 범위까지 허용 | 적층한 상태가 5 초 이상 안정적으로 유지     |
| 시간 보너스   | 5 개 전체 배치 완료 시 소요 시간에 따라 부여                | 5 개 전체 적층 완료 시 소요 시간에 따라 부여 |

경기장과 목표 영역의 구성은 그림 1-1과 같다.

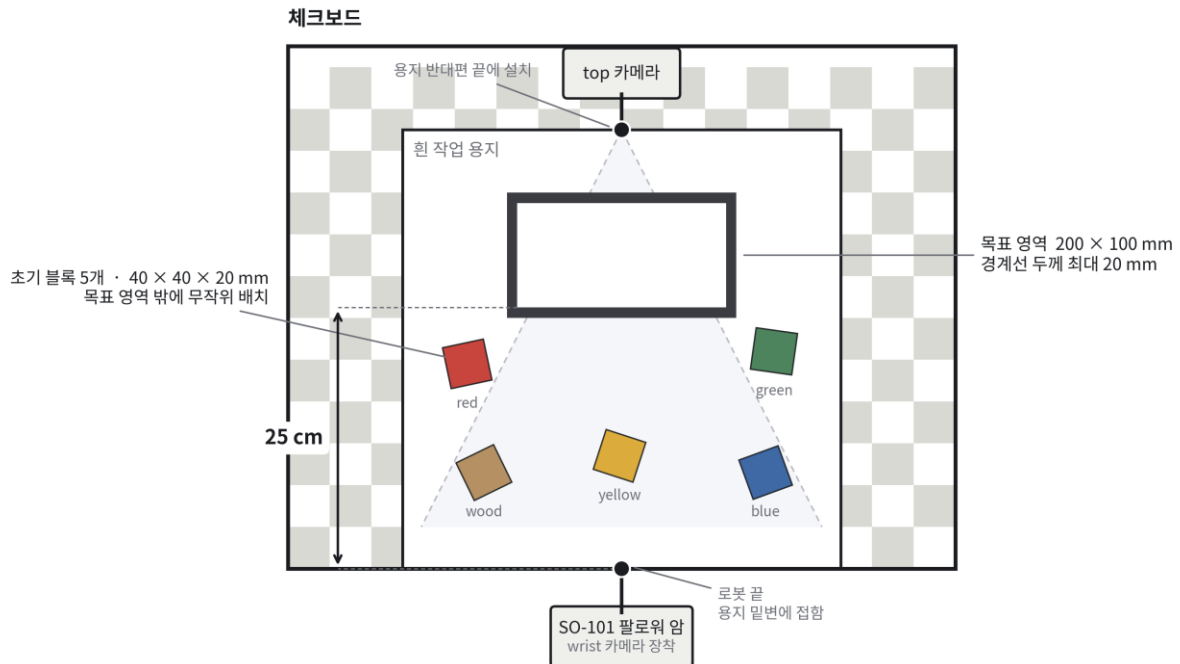


그림 1-1 경기장 및 지정 영역 도면

정리 작업에서는 블록을 세워 놓거나 일부 겹치게 배치하더라도 지정 영역의 포함 조건을 만족하면 성공으로 인정한다. 다만 블록을 위로 쌓는 것은 정리 작업의 성공으로 인정하지 않는다. 자연어 지시에 따른 색상 순서와 고정 슬롯 배치는 본 연구에서 설정한 내부 목표이며, 공식 성공 판정의 필수 조건은 아니다.

추가 안내에서는 제한 시간 내 성공한 블록 수를 우선하고, 소요 시간 보너스는 동일한 개수를 성공한 경우 우열을 구분하는 용도로 적용한다고 설명하였다. 따라서 본 연구에서도 전체 블록의 수행 완료와 반복 실행의 안정성을 우선적인 개발 목표로 설정하였다. 작업별 약 3회 수행 후 평균을 반영하는 방식과 정리·적층의 성공 블록당 배점을 약 1.5:1로 설정하는 방안도 제시되었으나, 구체적인 반복 횟수와 배점은 최종 확정 전의 검토 사항으로 구분한다.

신경망 기반 AI 모델을 사용하는 경우에는 두 작업 전체에서 하나의 모델만 사용해야 한다. 작업마다 서로 다른 신경망 모델을 사용하는 방식은 감점 대상이며, 단일 모델의 입력 형식과 출력 구조에는 제한이 없다. 규칙 기반 제어는 작업별로 별도 구성할 수 있고,

---

신경망 모델과 함께 사용하거나 규칙 기반 방식만으로 구현하는 것도 허용된다. 이러한 조건은 작업별 모델을 따로 개발하기보다 입력·출력, 데이터 구성 및 후처리 설계를 통해 하나의 모델이 여러 작업을 수행하도록 하는 데 목적이 있다.

평가 환경은 팀별로 로봇과 카메라를 직접 구성하며, 카메라 수에는 제한이 없다. 세부사항에서는 Jetson Orin Nano를 통해 평가 서버와 통신하고, 서버 실행 환경을 Docker 이미지로 준비하도록 요구한다. 실물 데모에서는 작업 달성 결과와 소요 시간을 평가하며, 발표에서는 사용한 제어 기법과 알고리즘의 유효성·독창성·우수성을 함께 평가한다.

### 1.3. 연구의 주요 내용

본 연구는 SO-101 로봇 팔을 이용하여 데이터 수집, 정책 학습, 비동기 추론 및 실물 제어를 연결하는 로봇 조작 시스템을 구현하였다. 초기 환경 구축과 단일 블록 조작 실험에서 출발하여, 5개 블록의 정리와 적층을 하나의 SmolVLA 정책으로 수행하도록 시스템을 확장하였다. 개발 과정에서는 인식 오차, 접근 자세의 불일치, 배치 위치 편향 등 실제 수행 중 관찰된 문제를 바탕으로 수집 방식과 학습·실행 구성을 개선하였다.

데이터 수집에서는 자동 접근과 리더 암을 이용한 수동 시연을 결합하여 5개 블록의 전체 작업을 하나의 에피소드로 연속 기록하였다. 자동 접근에는 영상 좌표를 로봇 좌표로 변환하는 보정과 실측 관절값 기반 보간을 적용하고, 정밀한 파지와 배치는 작업자가 시연하도록 구성하였다. 이를 통해 개별 블록의 조작뿐 아니라 배치 후 다음 블록으로 이동하는 과정까지 학습 데이터에 포함하였다.

정책 학습에서는 정리 353개와 적층 222개로 구성된 575개 에피소드를 통합하여 SmolVLA의 전체 미세조정을 수행하였다. 작업별 데이터 수와 배치 위치별 행동 분포를 검토하고, 정리와 적층의 배치 방식이 구분되도록 자연어 지시문을 구체화하였다. 또한 영상의 공간적 위치와 관절 행동 간 대응이 어긋날 수 있는 기하 증강을 제외하고, 영상 정규화와 크기 변환, 관절 목표값 제한을 점검하여 학습과 실행 과정의 불일치를 수정하였다.

실행 시스템은 로봇 클라이언트의 관측 전달과 GPU 서버의 정책 추론을 비동기 방식으로 연결하였다. 동일한 정책에 작업별 지시문을 입력하여 정리와 적층을 실행하도록 구성하였으며, 데이터 수집에 사용하는 검출기와 좌표 변환 모듈은 정책의 자율 실행 경로와 구분하여 운용하였다.

---

또한 정책 실행 중 실패가 확인되면 사람이 개입하여 복구와 교정을 수행하는 Correction-only HIL 데이터 수집 방식을 구현하였다. 이 방식에서는 실패 이전의 자율 실행 궤적, 실패를 유발한 자율 행동, 리더 암과 팔로워 암의 동기화 구간 및 개입 이후의 불필요한 대기 구간을 학습 데이터에서 제외하였다. 대신 실패가 확인된 시점 이후부터 사람이 수행한 복구·교정 행동만을 별도의 에피소드로 저장하였다. 이를 통해 실패 상태에서 정상적인 파지와 배치 상태로 복귀하는 행동을 명확한 모방학습 대상으로 구성하였다.

Correction-only HIL 로 수집한 복구·교정 데이터는 기존 성공 시연 데이터와 결합하여 SmolVLA 의 실패 대응 능력을 보강하는 데 사용하였다. 이때 기존 작업 지시문을 유지하여 별도의 복구 명령에 의존하지 않고, 카메라 영상과 로봇 상태를 바탕으로 현재 상황에 적절한 복구 행동을 생성하도록 학습하였다.

HIL 을 통해 복구 행동을 학습한 뒤에는 자율 실행 과정에서 생성되는 행동 사이의 품질 차이를 반영하기 위해 Reward-Weighted Flow Matching 을 적용하였다. 정상적인 작업 수행과 성공적인 복구 행동은 정책 학습에 더 크게 반영하고, 실패를 유발하거나 충돌·정체로 이어지는 행동은 학습 기여도를 낮추는 방식으로 Action Expert 의 행동 생성을 조정하였다. 이에 따라 Correction-only HIL 은 실패 상태에서 수행해야 할 복구·교정 행동을 학습하는 역할을 담당하고, RWFM 은 실제 자율 실행에서 나타나는 행동의 상대적인 품질을 반영하여 행동 선택을 정제하는 역할을 수행하였다.

성능 평가는 기본 SmolVLA 정책과 Correction-only HIL 및 RWFM 을 적용한 정책의 작업 수행 결과를 구분하여 분석하였다. HIL 적용 전 기본 정책의 내부 시험에서는 정리 작업의 5 블록 완료율 80%와 평균 소요 시간 70 초가 기록되었으며, 적층에서는 지시문에 따른 중앙 접근과 수직 배치 동작을 관찰하였다. 이후 Correction-only HIL 및 RWFM 이 적용된 정책에 대해서는 기존 정리·적층 능력의 유지 여부, 실패 상태에서의 복구 수행 여부, 실패를 유발하는 행동과 충돌·정체 발생의 변화를 기준으로 성능을 평가하였다.

#### 1.4. 구성원별 역할과 개발 일정

구성원은 정책 학습, 시각 인식 및 제어, 후속 정책 검증을 분담하고, 데이터 수집과 통합 시험에서 결과를 공유하였다. 중간 단계에는 독립 모듈의 동작 가능성을 확인하는 데 집중하였으며, 최종 단계에는 연속 시연 수집과 단일 정책의 멀티태스크 실행으로 개발 범위를 통합하였다. 역할은 중간보고서의 담당 영역과 이후 구현 내역을 연결하여 정리하였다.

구성원별 역할과 주요 산출물은 표 2와 같다.

표 2 구성원별 역할과 주요 산출물

| 구성원 | 중간 단계 수행 내용                                        | 최종 단계 수행 내용                                           |
|-----|----------------------------------------------------|-------------------------------------------------------|
| 김도환 | ACT 전체 작업 데이터와 모델군 분석, 데이터 정제 및 체크 설정에 따른 실패 양상 정리 | SmolVLA 미세조정, 멀티태스크 학습 및 비동기 추론 구성                    |
| 김혜은 | OpenCV 검출, 구역별 접근과 FSM 구성, ACT 파지 구간 실험            | YOLO 검출, 좌표 보정과 규칙 기반 제어, 수집기의 자동 접근에 필요한 인식·좌표 모듈 개발 |
| 천성민 | Diffusion Policy 예비 실험, 후속 정책의 데이터 형식과 서버 연결 조건 검토 | SmolVLA 미세조정, 멀티태스크 학습 및 비동기 추론 구성                    |
| 공동  | 실패 원인 검토, 통합 실험, 중간보고서와 발표 자료 작성                   | 시연 데이터 구축 및 정제, 작업별 실물 관찰, 결과 분석과 최종 보고서 작성           |

개발은 환경 구축, 정책 검증, 인식·좌표 제어, 연속 수집 및 통합 학습의 단계로 진행하였다. 인식과 좌표 제어 개발은 정책 학습과 병행되었으며, 해당 결과를 최종 데이터 수집기에 통합하였다. 중간보고서에서는 9월 둘째 주에 통합 평가를 수행할 계획이었으나, 최종 단계에도 복구 데이터 학습과 정량 평가 보완이 이어졌다.

단계별 개발 내용과 산출물은 표 3과 같다.

표 3 개발 일정과 단계별 산출물

| 기간            | 주요 개발 내용                                    | 산출물 및 도달 상태                               |
|---------------|---------------------------------------------|-------------------------------------------|
| 6월 27일~7월 8일  | 로봇·카메라·GPU 환경 구축, ACT 단독 데이터 수집과 학습         | 원격 학습·실행 환경, 초기 및 정제 데이터, 전체 작업의 실패 요인 확인 |
| 7월 9일~7월 26일  | OpenCV와 FSM 개발, ACT 근접 파지 및 Diffusion 예비 실험 | 중간보고서, C1 빨간 블록 파지 시험, 모듈별 실패 분리          |
| 7월 27일~8월 18일 | SmolVLA 도입, 단일 블록 시연 보강, 전처리·명령 처리 수정       | 단일 블록 실험군, 영상 입력 경로 수정, 공통 비율 명령 제한       |
| 8월 19일~9월 2일  | YOLO 인식과 좌표 보정, 규칙 기반 정리 및 회전각 검출 개발        | 370장 검출 데이터, 201장 OBB 데이터, 5블록 정리 단회 시험   |
| 8월 22일~9월 9일  | 연속 시연 수집기, 정리·적층 데이터, 멀티태스크 학습              | 575개 에피소드, 300,000단계 학습, 단일 정책 실행 및 내부 평가 |
| 9월 9일 이후      | HIL 추가 데이터와 후속 학습, 평가 보완                    | 704개 에피소드 구성과 후속 학습, 복구 행동 개선 추진          |

---

## 2. 연구 배경

### 2.1. 로봇 조작과 모방학습

로봇 조작은 로봇이 주변 환경을 관측하고 작업 목적에 맞는 행동을 생성하여 물체를 이동하거나 배치하는 과정이다. 조작 작업에서는 물체의 위치와 방향, 로봇과 물체 사이의 상대 자세, 그리퍼의 파지 상태 및 주변 환경을 함께 고려해야 한다. 특히 파지 이후의 이송과 배치 과정에서는 초기 오차가 후속 동작에 누적될 수 있으므로, 개별 동작의 정확도뿐 아니라 여러 동작을 연속적으로 수행하는 능력이 중요하다.

모방학습(Imitation Learning)은 사람이 수행한 시연 데이터를 이용하여 로봇의 행동 정책을 학습하는 방법이다. 시연 데이터는 카메라 영상과 로봇의 관절 상태 등의 관측값, 그리고 해당 상황에서 수행한 로봇 행동의 쌍으로 구성된다. 정책은 관측  $o_t$ 가 주어졌을 때 시연자가 선택한 행동  $a_t$ 를 예측하도록 학습되며, 이를 다음과 같이 표현할 수 있다.

$$a_t = \pi_{\theta}(o_t) \quad (2-1)$$

여기서  $o_t$ 는 시점  $t$ 의 관측값,  $a_t$ 는 로봇의 관절 목표값 또는 제어 명령,  $\pi_{\theta}$ 는 학습 가능한 정책을 의미한다. 대표적인 행동 복제(Behavior Cloning)는 시연 데이터의 관측과 행동 사이의 오차를 줄이는 지도학습 방식으로, 별도의 보상 함수를 설계하지 않고도 사람이 수행한 조작 전략을 학습할 수 있다는 장점이 있다.

시연 데이터를 수집하는 방법 중 하나인 텔레오퍼레이션(Teleoperation)은 사람이 직접 조작하는 리더 장치의 움직임을 로봇의 팔로워 장치에 전달하는 방식이다. 작업자는 로봇의 관절을 직접 제어하면서 물체에 접근하고, 파지하고, 이동시키고, 배치하는 전체 과정을 시연할 수 있다. 따라서 미리 모든 경로를 수식이나 규칙으로 정의하기 어려운 조작 작업에서도 사람의 경험적 판단과 미세한 자세 조정을 데이터에 포함할 수 있다. 반면 작업자의 조작 속도와 자세가 일정하지 않거나, 시연 중 불필요한 동작이 포함되면 데이터의 품질이 저하될 수 있으므로 수집 과정의 일관성과 검증이 필요하다.

텔레오퍼레이션으로 수집한 관측과 행동을 학습에 사용하기 위해서는 로봇, 카메라 및 제어 장치 사이의 좌표와 상태 표현을 일치시켜야 한다. 캘리브레이션(Calibration)은 센서와 로봇의 기준 좌표계, 관절의 영점과 동작 범위, 카메라와 작업 공간 사이의 상대적인 위치를 확인하고 보정하는 과정이다. 좌표계가 올바르게 정렬되지 않으면 영상에서 확인한 물체 위치와 로봇이 실제로 접근하는 위치 사이에 차이가 발생하며, 동일한 관절 명령도 수집 시점과 실행 시점에 서로 다른 자세를 만들 수 있다. 그러므로 캘리브레이션

---

션은 시연 데이터를 정확하게 기록하고, 학습된 정책을 실제 로봇에서 재현하기 위한 기본 조건이다.

그러나 행동 복제만으로 학습된 정책은 실제 실행 중 발생하는 오차에 취약할 수 있다. 로봇이 물체를 약간 벗어나 파지하거나 목표 위치에 정확히 도달하지 못하면, 이후에는 학습 데이터에 충분히 포함되지 않은 상태가 나타날 수 있기 때문이다. 이와 같은 오차 누적 문제를 완화하려면 다양한 위치와 자세를 포함한 시연 데이터를 수집하고, 필요에 따라 실패 상황이나 복구 동작도 학습에 반영해야 한다. 정책이 방문한 상태에서 전문가의 교정을 수집하는 접근은 이러한 분포 차이를 줄이기 위한 방법이다[4]. 또한 동일한 장면에서도 작업 지시에 따라 서로 다른 행동이 필요할 수 있으므로, 영상과 로봇 상태 뿐 아니라 언어 지시를 함께 사용하는 정책이 요구된다.

## 2.2. Action Chunking 기반 모방학습과 ACT

로봇 조작에서 정책이 매 시점마다 하나의 행동만 예측하면, 짧은 시간 간격으로 생성된 행동 사이에 불연속적인 변화가 발생하거나 작은 예측 오차가 반복적으로 누적될 수 있다. 이러한 문제를 완화하기 위한 방법으로 액션 청킹(Action Chunking)이 사용된다. 액션 청킹은 현재 관측에 대응하는 단일 행동만 출력하는 대신, 일정 구간 동안 수행할 여러 개의 미래 행동을 하나의 묶음으로 예측하는 방식이다.

시점  $t$ 의 관측을  $o_t$ , 행동 청크의 길이를  $K$ 라고 하면 정책의 출력은 다음과 같이 표현할 수 있다.

$$A_{t:t+K-1} = [a_t, a_{t+1}, \dots, a_{t+K-1}] = \pi_\theta(o_t) \quad (2-2)$$

각 행동  $a_t$ 는 로봇의 관절 목표값과 그리퍼 제어값으로 구성될 수 있다. 실행 단계에서는 예측된 행동 청크를 일정 구간 동안 순차적으로 적용한 뒤 새로운 관측을 이용해 다음 청크를 생성한다. 이와 같은 방식은 접근, 파지, 이송 및 배치처럼 시간적으로 연결된 동작을 하나의 궤적으로 다루는 데 적합하다.

ACT(Action Chunking with Transformers)는 액션 청킹을 Transformer 기반 모방학습 정책에 적용한 방법이다[3]. ACT는 카메라 영상과 로봇 관절 상태를 입력으로 받아 미래의 관절 행동 시퀀스를 생성한다. Transformer의 어텐션 구조를 이용하여 관측 정보와 행동의 시간적 관계를 학습하며, 단일 시점의 반응보다 시연자가 수행한 연속적인 조작 패턴을 직접 모델링할 수 있다. 이를 통해 파지 전 접근, 물체를 들어 올리는 동작, 목표 위치로 이동하는 동작과 같이 서로 연결된 행동을 하나의 정책 안에서 학습한다.

ACT의 또 다른 특징은 동일한 시점에서 생성된 행동 청크를 시간적으로 결합할 수 있다는 점이다. 새로운 관측마다 미래 행동 청크가 다시 예측되면 일부 행동 구간이 서로 겹치게 되는데, 이때 겹치는 예측을 평균하거나 가중 결합하여 최종 행동을 결정할 수 있다. 이러한 temporal ensemble 방식은 각 예측 청크 사이의 급격한 변화를 줄이고, 로봇의 관절 궤적을 보다 연속적으로 만드는 데 도움을 준다.

액션 청킹은 장시간 조작에서 행동의 시간적 연속성을 반영할 수 있다는 장점이 있지만, 예측된 청크의 초반 행동이 잘못되면 해당 청크에 포함된 여러 행동이 연속적으로 실행될 수 있다는 한계도 가진다. 또한 기본 ACT는 주로 영상과 관절 상태를 이용해 행동을 생성하므로, 동일한 장면에서 작업 지시에 따라 서로 다른 조작 전략을 선택해야 하는 멀티태스크 상황에서는 별도의 조건 입력이나 작업 구분 방법이 필요하다.

본 연구에서는 ACT를 초기 모방학습 정책으로 활용하여 시연 데이터 기반의 연속적인 로봇 조작 가능성을 확인하였다. 이후에는 정리와 적층처럼 작업 목적이 다른 상황을 자연어 지시로 구분할 수 있도록 정책 입력을 확장하였다. 이때 SmolVLA 역시 여러 시점의 행동을 묶음으로 출력한다는 점에서 액션 청킹의 기본 개념을 공유하지만, 영상과 관절 상태뿐 아니라 언어 지시를 함께 처리하고 Flow Matching 기반 행동 생성부를 사용하는 점에서 ACT와 구분된다.

### 2.3. Vision-Language-Action 모델

Vision-Language-Action(VLA) 모델은 시각 정보와 자연어 지시를 바탕으로 로봇의 행동을 생성하는 멀티모달 정책이다. 기존의 로봇 제어 정책은 주로 카메라 영상이나 관절 상태와 같은 수치 정보만을 이용하여 현재 상황에 대한 행동을 예측하였다. 그러나 실제 조작 작업에서는 동일한 장면에서도 사용자의 지시나 작업 목적에 따라 수행해야 할 행동이 달라질 수 있다. VLA 모델은 이러한 작업 조건을 자연어 형태로 함께 입력받아 환경 인식과 행동 결정을 하나의 정책 안에서 처리한다.

VLA 모델의 입력은 일반적으로 카메라 영상, 자연어 작업 지시 및 로봇의 현재 상태로 구성된다. 영상은 물체의 위치, 색상, 방향 및 주변 환경을 나타내고, 자연어 지시는 수행해야 할 작업의 목적과 조건을 제공한다. 로봇 상태는 관절 위치, 그리퍼 상태 및 기타 제어 정보를 나타낸다. 정책은 이러한 정보를 결합하여 다음 행동 또는 일정 구간의 행동 시퀀스를 출력하며, 이를 조건부 정책으로 표현하면 다음과 같다.

$$A_{t:t+K-1} = \pi_{\theta}(o_t, l_t, s_t) \quad (2-3)$$

---

여기서  $o_t$ 는 시각 관측,  $l_t$ 는 자연어 지시,  $s_t$ 는 로봇 상태이며,  $A_{t:t+K-1}$ 은 시점  $t$ 이후  $K$ 개 시점에 대한 행동 묶음이다. 출력 행동은 관절 목표값, 그리퍼 제어값 또는 이들의 연속적인 조합으로 구성될 수 있다. 따라서 VLA 모델은 언어 조건을 반영하면서도 액션 청킹 기반 정책과 결합하여 여러 단계로 이루어진 조작 동작을 연속적으로 생성할 수 있다.

VLA 모델은 입력 정보를 처리하는 방식에 따라 여러 구성 요소로 나뉜다. 시각 인코더는 영상에서 물체와 작업 공간의 특징을 추출하고, 언어 인코더는 자연어 지시를 의미적 특징으로 변환한다. 이후 멀티모달 결합부는 시각 정보와 언어 정보를 연결하여 현재 작업에 필요한 조건을 형성하며, 행동 생성부는 결합된 특징과 로봇 상태를 이용해 실제 제어 명령을 생성한다. 이 구조를 통해 모델은 단순히 물체를 인식하는 데 그치지 않고, 지시에 맞는 대상과 행동 방식을 선택할 수 있다.

VLA 모델의 중요한 특징은 동일한 시각 환경에서도 언어 지시에 따라 서로 다른 행동을 생성할 수 있다는 점이다. 예를 들어 동일한 블록과 목표 영역이 관측되더라도 “지정 영역에 배치하라”는 지시와 “블록을 수직으로 쌓으라”는 지시에 따라 목표 위치와 접근 궤적이 달라져야 한다. 이때 언어 지시는 정책이 작업 목적을 구분하기 위한 조건으로 사용되며, 영상과 로봇 상태만으로는 구분하기 어려운 행동 차이를 표현할 수 있다.

그러나 VLA 모델의 성능은 언어와 행동 데이터의 대응 관계에 크게 영향을 받는다. 동일한 작업을 서로 다른 표현으로 기록하거나, 지시문과 실제 시연 행동이 일치하지 않으면 모델이 작업 조건을 정확하게 구분하기 어려울 수 있다. 또한 영상의 위치 정보와 행동 라벨 사이에 불일치가 존재하면 언어 정보를 추가하더라도 정확한 파지와 배치가 보장되지 않는다. 따라서 VLA 정책을 학습할 때에는 시각 관측, 자연어 지시, 로봇 행동이 동일한 작업 의도를 나타내도록 데이터를 구성해야 한다.

본 연구에서는 VLA 모델을 정리와 적층이라는 서로 다른 조작 작업을 하나의 정책으로 처리하기 위한 기반으로 활용하였다. ACT와 마찬가지로 여러 시점의 행동을 하나의 묶음으로 출력할 수 있지만, VLA 모델은 여기에 자연어 작업 지시를 추가하여 작업 목적과 배치 방식을 구분한다. 다음 절에서는 본 연구에서 사용한 SmolVLA의 구성과 행동 생성 방식을 구체적으로 설명한다.

## 2.4. SmolVLA 모델

SmolVLA는 영상, 자연어 및 로봇 상태를 함께 처리하여 연속적인 로봇 행동을 생성하는 경량 Vision-Language-Action 모델이다[1]. 대규모 VLA 모델은 다양한 작업을 처리할 수 있지만, 모델 규모가 커질수록 학습과 추론에 필요한 계산 자원과 메모리도 증가한다.

SmolVLA는 비교적 작은 규모의 사전학습 비전·언어 모델과 행동 생성부를 결합하여, 제한된 계산 자원에서도 로봇 조작 정책을 학습하고 실행할 수 있도록 설계되었다.

SmolVLA의 구조는 비전·언어 처리부와 행동 전문가(Action Expert)로 구성된다. 비전·언어 처리부는 입력 영상과 자연어 지시를 인코딩하여 현재 장면과 작업 목적을 나타내는 특징을 생성한다. 로봇의 관절 상태와 그리퍼 상태는 별도의 상태 표현으로 처리된 뒤 행동 생성부에 함께 입력된다. 행동 전문가는 이 정보를 이용하여 현재 작업에 필요한 연속적인 관절 행동을 생성한다. 이를 개념적으로 표현하면 다음과 같다.

$$z_t = f_{\text{VLM}}(o_t, l_t)$$

$$A_{t:t+K-1} = g_{\text{action}}(z_t, s_t) \quad (2-4)$$

여기서  $o_t$ 는 카메라 영상,  $l_t$ 는 자연어 지시,  $s_t$ 는 로봇 상태,  $z_t$ 는 비전·언어 처리부가 생성한 멀티모달 특징이다.  $A_{t:t+K-1}$ 은 시점  $t$ 이후  $K$ 개 시점에 대한 행동 묶음이며, 각 행동은 관절 목표값과 그리퍼 제어값으로 구성될 수 있다.

SmolVLA의 행동 전문가는 Flow Matching 기반으로 학습된다. 행동을 이산적인 토큰으로 분류하는 대신, 현재 상태에서 시연 행동으로 이어지는 연속적인 행동 분포를 학습하여 로봇의 관절 위치와 그리퍼 제어값을 생성한다. 이 방식은 접근, 파지, 이송 및 배치처럼 시간적으로 연결된 동작을 연속적인 관절 궤적으로 표현하는 데 적합하다. 또한 ACT와 마찬가지로 여러 시점의 행동을 하나의 action chunk로 출력하므로, 개별 관절 명령을 독립적으로 생성하는 방식보다 시연 동작의 시간적 구조를 반영할 수 있다.

SmolVLA를 선택한 이유는 자연어 조건을 이용한 멀티태스크 처리와 모델의 계산 효율을 함께 고려할 수 있기 때문이다. 본 과제에서는 동일한 장면에서 정리와 적층이라는 서로 다른 작업을 수행해야 하므로, 영상 정보만으로는 구분하기 어려운 작업 조건을 자연어 지시로 전달할 필요가 있다. SmolVLA는 하나의 정책에 영상과 언어 정보를 함께 입력할 수 있으므로, 작업별로 별도의 신경망을 사용하지 않고 하나의 모델이 두 작업을 수행하도록 구성할 수 있다. 이는 두 작업에 하나의 신경망만 사용해야 한다는 과제 조건에도 부합한다.

SmolVLA는 약 4.5억 개의 파라미터로 구성된 경량 모델이며, 단일 소비자용 GPU에서도 파인튜닝할 수 있도록 설계되었다.

SmolVLA의 성능은 모델 구조뿐 아니라 입력 데이터의 정합성에도 영향을 받는다. 카메라 영상의 위치 정보와 관절 행동 사이에 오차가 있거나, 자연어 지시와 실제 시연 행동

이 일치하지 않으면 모델이 작업 의도와 행동을 정확하게 연결하기 어렵다. 따라서 실제로 로봇에 적용하기 위해서는 시각 관측, 자연어 지시, 로봇 상태 및 행동 데이터의 대응 관계를 유지하고, 학습과 실행 과정에서 동일한 입력-출력 표현을 사용해야 한다.

본 연구에서는 이러한 특성을 고려하여 SmolVLA를 정리와 적층 작업에 공통으로 사용하는 단일 신경망 정책으로 채택하였다. 작업별 자연어 지시와 시연 데이터를 함께 구성하고, 하나의 모델이 서로 다른 배치 행동을 생성하도록 설계하였다. 이후 장에서는 SmolVLA의 구체적인 학습 설정과 데이터 구성, 비동기 추론 및 실물 실행 결과를 설명한다.

## 2.5. Flow Matching 기반 행동 생성

Flow Matching은 무작위 잡음에서 목표 데이터까지의 연속적인 변환 과정을 학습하여 데이터를 생성하는 방법이다[5]. 확산 모델과 유사하게 생성 과정에서 초기 상태와 목표 상태 사이의 변환을 학습하지만, 각 단계에서 노이즈를 제거하는 과정을 직접 반복하는 대신 상태를 이동시키는 벡터장(vector field)을 학습한다. 이를 통해 연속적인 값으로 구성된 로봇의 관절 행동을 생성할 수 있다.

로봇 행동 생성에서는 먼저 시연 행동  $A$ 와 무작위 초기 상태  $x_0$ 를 설정하고, 두 상태 사이의 중간 상태를 다음과 같이 구성할 수 있다.

$$x_\tau = (1 - \tau)x_0 + \tau A, 0 \leq \tau \leq 1 \quad (2-5)$$

여기서  $\tau$ 는 생성 과정의 진행 정도를 나타내며,  $\tau = 0$ 에서는 초기 잡음에 가깝고  $\tau = 1$ 에서는 목표 행동에 가까운 상태가 된다. 모델은 현재 중간 상태  $x_\tau$ , 진행 시점  $\tau$ , 관측 조건  $c$ 를 입력받아 목표 행동 방향을 나타내는 벡터장  $v_\theta(x_\tau, \tau, c)$ 를 예측한다. 학습 과정에서는 예측 벡터장과 목표 변환 방향 사이의 차이를 줄이도록 모델을 최적화한다.

$$\mathcal{L}_{\text{FM}} = \mathbb{E}[\|v_\theta(x_\tau, \tau, c) - v^*(x_\tau, \tau, c)\|^2] \quad (2-6)$$

추론 단계에서는 초기 잡음에서 시작하여 학습된 벡터장을 따라 상태를 연속적으로 이동시키고, 최종적으로 로봇이 실행할 행동을 얻는다. 이때 조건  $c$ 에는 카메라 영상, 자연어 지시 및 로봇 상태가 포함될 수 있다. 따라서 모델은 동일한 작업 공간에서도 관측된 물체의 위치와 작업 지시에 따라 서로 다른 연속 행동을 생성할 수 있다.

SmolVLA에서는 이러한 Flow Matching 과정을 Action Expert에 적용한다. 비전-언어 처리부가 영상과 자연어 지시로부터 작업 조건을 나타내는 특징을 생성하면, Action Expert는 해당 특징과 로봇 상태를 이용하여 관절 행동과 그리퍼 제어값으로 구성된 행동 묶음을

---

생성한다. 출력은 여러 시점의 행동을 포함하는 action chunk 형태로 구성될 수 있으므로, 접근·파지·이송·배치로 이어지는 연속적인 조작 궤적을 한 번에 모델링할 수 있다.

액션 청킹과 Flow Matching은 서로 다른 역할을 수행한다. 액션 청킹은 정책이 여러 시점의 행동을 묶어서 출력하는 시간적 표현 방식이고, Flow Matching은 그 행동 묶음의 연속적인 값을 생성하는 학습 및 생성 방법이다. 따라서 SmolVLA는 Flow Matching으로 관절 행동을 생성하면서, 해당 행동을 action chunk 단위로 출력할 수 있다.

Flow Matching 기반 정책은 로봇 관절값처럼 연속적인 제어량을 직접 생성할 수 있고, 여러 가능한 행동 궤적을 모델링할 수 있다는 장점이 있다. 반면 생성 과정의 계산량과 추론 단계 수에 따라 실행 지연이 발생할 수 있으며, 학습 데이터의 행동 분포가 편향되어 있으면 특정 위치나 자세로 행동이 집중될 수 있다. 그러므로 실제 로봇에 적용할 때에는 행동 데이터의 다양성, 입력 영상과 관절 행동의 정합성, 추론 속도를 함께 고려해야 한다.

본 연구에서는 SmolVLA의 Flow Matching 기반 Action Expert를 이용하여 정리와 적층 작업에 필요한 연속적인 관절 행동을 생성하였다. 구체적인 모델 초기화 방식과 파인튜닝 전략은 다음 절에서 설명한다.

## 2.6. 사전학습 SmolVLA Base 모델과 파인튜닝 전략

SmolVLA는 사전학습된 Base 모델을 초기 가중치로 사용한 뒤, 새로운 로봇과 작업에 수집된 시연 데이터로 정책을 조정하는 방식으로 활용된다. 본 연구에서는 LeRobot[2]에서 제공하는 lerobot/smolvla\_base 체크포인트를 공통 초기 모델로 사용하였다. 따라서 Full Fine-Tuning, Expert-only Fine-Tuning 및 LoRA는 서로 다른 모델을 사용하는 방식이 아니라, 동일한 Base 모델에서 어떤 파라미터를 업데이트할 것인지에 따른 학습 전략의 차이를 의미한다.

파인튜닝은 사전학습 모델이 보유한 일반적인 시각·언어 표현과 행동 생성 능력을 새로운 로봇의 관절 구조, 카메라 환경 및 작업 조건에 맞게 조정하는 과정이다. 사전학습 모델을 그대로 사용하면 새로운 로봇의 관절 표현이나 작업 공간의 시각적 특성이 기존 학습 분포와 다를 수 있다. 반대로 모든 가중치를 무제한으로 변경하면 과제 데이터에 과도하게 적응하여 사전학습된 일반화 능력을 잃을 수 있으므로, 데이터 규모와 작업의 특성에 따라 파라미터 업데이트 범위를 결정해야 한다.

Full Fine-Tuning은 Base 모델의 학습 가능한 파라미터 전체를 과제 데이터에 맞게 업데이트하는 방식이다. 비전·언어 처리부와 행동 생성부를 함께 조정할 수 있으므로, 새로운

---

카메라 환경이나 로봇의 관절 표현처럼 사전학습 분포와 차이가 큰 조건에 폭넓게 적응할 수 있다. 반면 업데이트되는 파라미터 수가 많아 GPU 메모리와 학습 시간이 많이 필요하며, 데이터가 충분하지 않으면 과적합이 발생할 가능성이 있다.

Expert-only Fine-Tuning은 비전·언어 처리부의 가중치를 고정하고, 로봇 행동을 생성하는 Action Expert와 상태 입력을 변환하는 계층 등 일부 모듈만 학습하는 방식이다. 이 방법은 기존의 시각·언어 표현을 유지하면서 행동 생성부를 새로운 로봇과 작업에 맞게 조정할 수 있으므로 Full Fine-Tuning보다 계산 비용이 작다. 그러나 카메라 환경이나 작업 조건이 사전학습 데이터와 크게 다를 경우, 고정된 비전·언어 처리부가 새로운 시각적 특징을 충분히 반영하지 못할 수 있다.

LoRA(Low-Rank Adaptation)는 기존 Base 모델의 가중치를 고정하고, 일부 선형 계층에 저랭크 어댑터를 추가하여 어댑터 파라미터만 학습하는 방법이다[6]. 학습해야 하는 파라미터 수와 저장 공간을 줄일 수 있으며, 여러 과제에 대한 어댑터를 별도로 관리하기 쉽다는 장점이 있다. 반면 모델의 전체 표현을 변경할 수 없기 때문에, 정밀한 실물 조작이나 새로운 환경에 대한 큰 폭의 적응이 필요한 경우에는 업데이트 범위가 제한될 수 있다.

본 연구에서는 동일한 lerobot/smolvlm\_base 모델을 출발점으로 하여 위의 파인튜닝 전략을 적용하고, 데이터셋 규모와 작업 조건에 따른 정책의 변화를 비교하였다. 각 방식의 구체적인 학습률, 배치 크기, 학습 스텝 및 실물 실행 결과는 연구 내용과 실험 결과 장에서 설명한다.

## 2.7. 시각 기반 좌표 변환과 기하 보정

로봇이 카메라 영상에서 인식한 물체를 실제로 파지하기 위해서는 영상의 픽셀 좌표를 로봇이 이해할 수 있는 작업 공간 좌표로 변환해야 한다. 카메라 영상에서 검출된 물체의 중심은  $(u, v)$ 로 표현되지만, 로봇 제어에는 작업판 또는 로봇 기준 좌표계에서의 위치  $(x, y)$ 가 필요하다. 이때 카메라와 작업 공간 사이의 위치 및 방향 관계를 추정하고 보정하는 과정을 카메라·로봇 캘리브레이션이라고 한다.

작업판을 평면으로 가정할 수 있는 경우에는 호모그래피(Homography)를 이용하여 영상 좌표와 평면 좌표 사이의 원근 변환을 모델링할 수 있다. 동차 좌표를 사용하면 변환 관계는 다음과 같이 표현된다.

$$s \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad (2-7)$$

여기서  $H$ 는 카메라 영상 평면과 작업 공간 평면 사이의  $3 \times 3$  변환 행렬이며,  $s$ 는 동차 좌표의 크기 조정을 위한 척도값이다.  $H$ 를 계산하기 위해서는 영상에서 확인한 기준점과 실제 작업판 좌표 사이의 대응 관계가 필요하다. 일반적으로 서로 다른 위치의 기준점을 여러 개 수집한 뒤, 이 대응점을 이용하여 변환 행렬을 추정한다.

호모그래피는 평면 위 물체의 위치를 변환하는 데 효과적이지만, 카메라 렌즈 왜곡, 기준점 측정 오차, 작업판의 높이 차이 및 로봇 구조의 기계적 오차까지 직접 보정하지는 못한다. 특히 카메라가 관측하는 영역이 넓거나 기준점의 범위를 벗어난 위치를 외삽할 경우, 변환된 좌표와 실제 로봇의 접근 위치 사이에 오차가 커질 수 있다. 따라서 변환 결과는 별도의 검증점이나 실제 로봇의 접근 결과를 통해 확인해야 한다.

영상 좌표를 로봇의 관절값으로 직접 변환해야 하는 경우에는 실측 데이터 기반 보간 모델을 사용할 수 있다. 방사 기저 함수(Radial Basis Function, RBF) 보간은 여러 기준 위치에서 측정한 입력과 출력의 관계를 이용하여 기준점 사이의 값을 추정하는 방법이다. 작업 공간의 위치  $x = (X, Y)$ 에 대한 관절  $j$ 의 목표값은 다음과 같이 표현할 수 있다.

$$q_j(x) = \sum_{i=1}^N w_{ij} \phi(\|x - c_i\|) + p_j(x) \quad (2-8)$$

여기서  $c_i$ 는 실측 기준점,  $w_{ij}$ 는 각 기준점의 가중치,  $\phi$ 는 거리 기반 방사 함수,  $p_j(x)$ 는 전체적인 위치 변화 경향을 나타내는 저차 다항식이다. RBF 보간은 기구학적 수식만으로 표현하기 어려운 프레임 휨, 백래시 및 관절별 비선형 오차를 실측 데이터에 포함하여 모델링할 수 있다는 장점이 있다.

호모그래피와 RBF 보간은 서로 다른 목적에 사용될 수 있다. 호모그래피는 영상에서 검출한 물체의 위치를 작업 공간의 평면 좌표로 변환하는 데 적합하고, RBF 보간은 해당 작업 공간 위치에서 로봇이 취해야 할 호버 자세나 관절 목표값을 추정하는 데 적합하다. 따라서 두 방법을 연속적으로 사용하면 물체의 영상 위치를 로봇의 접근 위치와 관절 행동으로 변환하는 좌표 기반 제어 경로를 구성할 수 있다.

본 연구에서는 카메라 영상에서 얻은 블록 위치를 로봇의 접근 행동으로 연결하기 위해 평면 좌표 변환과 실측 관절값 기반 보간을 사용하였다. 초기에는 기하학적 변환과 규칙 기반 자세를 이용했으며, 이후 실제 로봇의 기계적 오차를 반영하기 위해 실측 기준점

---

기반의 RBF 보간을 적용하였다. 구체적인 기준점 구성, 좌표 변환 절차 및 보정 오차 검증 결과는 3장과 4장에서 설명한다.

## 2.8. 사람 개입 기반 모방학습과 복구 데이터 수집

모방학습으로 학습한 로봇 정책은 실행 과정에서 발생하는 작은 오차로 인해 시연 데이터에 충분히 포함되지 않은 상태에 진입할 수 있다. 예를 들어 블록 파지에 실패한 뒤에도 이송 동작을 계속하면, 정책은 물체를 잡고 이동하는 정상 시연과 다른 관측을 입력 받게 된다. 이러한 학습 및 실행 상태 분포의 차이는 후속 행동의 오류를 증가시킬 수 있다. 이를 보완하기 위한 상호작용 모방학습은 현재 정책을 실행하면서 추가적인 전문가 지도를 수집하고, 이를 정책 학습에 반영한다.

DAGger(Dataset Aggregation)는 학습 정책이 방문한 상태에서 전문가 행동을 수집하고, 이를 기존 데이터에 누적하여 정책을 반복적으로 학습하는 방법이다. 전문가 시연의 상태 분포에만 의존하지 않고 정책이 실제로 마주하는 상태에 대한 행동 지도를 확보하는 것이 핵심이다. 따라서 실패 상황을 학습에 활용한다는 것은 실패를 유발한 행동을 그대로 모방한다는 의미가 아니라, 해당 상태에서 전문가가 선택할 적절한 행동을 학습한다는 의미이다.

실물 로봇에서는 사람이 정책의 모든 방문 상태에 대해 행동 정답을 제공하기 어렵다. HG-DAGger(Human-Gated DAGger)는 사람이 로봇의 실행을 관찰하다가 개입이 필요한 시점을 판단하고, 제어권을 가져와 전문가 행동을 제공하는 방식으로 이러한 부담을 완화한다. 사람은 개입 구간에서 로봇을 직접 제어하므로, 자신이 제시한 명령에 따른 상태 변화를 확인하면서 연속적인 교정 시연을 수행할 수 있다. 이와 같은 사람 주도 제어권 전환은 HIL(Human-in-the-Loop) 모방학습을 구성하는 한 방법이다. HIL은 사람이 학습 또는 실행 과정에 참여하는 구성을 뜻하며, 그 자체가 특정 학습 목적함수나 강화학습 알고리즘을 의미하지는 않는다.

개입 과정에서 제공하는 행동은 복구와 교정으로 구분할 수 있다. 복구는 실패하거나 불안정해진 상태에서 다시 작업을 시도할 수 있는 상태로 이동하는 과정이고, 교정은 그 상태에서 올바른 행동으로 작업을 진행하는 과정이다. 블록 파지에서는 그리퍼를 열고 블록 상공으로 물러나는 동작이 복구에, 블록 중심으로 재정렬하여 파지하는 동작이 교정에 해당할 수 있다. RaC(Recovery and Correction)는 이러한 두 종류의 행동을 함께 수집하여 재시도 능력을 학습하는 접근이다. 이때 복구 상태는 반드시 전체 작업의 초기 자세일 필요는 없으며, 현재 하위 작업을 다시 수행할 수 있는 익숙한 상태를 의미한다.

---

HIL 데이터의 학습 효과는 기록 범위와 학습 대상의 선택에도 영향을 받는다. 자율 실행과 사람 개입을 포함한 전체 궤적을 저장하면 실패에 이르는 맥락을 보존할 수 있지만, 모든 행동을 동일한 모방 목표로 사용하면 실패를 유발한 자율 행동까지 학습에 포함될 수 있다. 이에 대한 대응으로 개입 이후의 복구·교정 구간을 선택하거나, 행동의 출처와 품질에 따라 학습 비중을 조절할 수 있다. SIRIUS는 기존 전문가 시연, 자율 행동, 사람 개입 및 개입 직전 구간을 구분하고, 개입 데이터의 비중을 높이는 한편 개입 직전의 부적절한 행동이 학습에 미치는 영향을 줄인다. 따라서 전체 궤적의 저장 여부와 각 구간을 모방 학습에 사용하는 여부는 별도로 결정해야 한다.

본 연구에서는 사람이 개입 시점을 판단하고, 개입 이후의 복구·교정 구간을 별도 에피소드로 수집하는 방식을 사용한다. 본문에서 사용하는 Correction-only HIL은 개입 이전의 자율 실행을 학습 시연에서 제외한다는 의미이며, 기록되는 개입 구간에는 복구와 교정이 모두 포함된다. 복구에는 자동 제어를 활용하고 정밀한 파지와 배치는 사람이 시연하므로, 수집 데이터는 사람 감독 아래 수행한 자동 복구와 수동 교정의 결합으로 구성된다. 이 방식은 HG-Dagger의 사람 주도 개입 원리와 RaC의 복구·교정 개념을 과제 환경에 맞게 적용한 것으로 볼 수 있다. 수집한 데이터는 기존 작업 시연과 결합하고, 별도의 복구 지시문 대신 기존 작업 지시문을 유지하여 관측과 로봇 상태에 따른 행동을 학습하도록 구성한다.

개입 구간 중심의 학습은 실패 상태에서 적절한 복구 행동을 제공하지만, 기록에서 제외된 이전 자율 행동의 품질을 직접 평가하지는 않는다. 또한 채택된 시연을 동일하게 학습하면 성공한 행동들 사이의 안정성이나 효율 차이를 명시적으로 반영하기 어렵다. 이러한 한계를 보완하기 위해 본 연구에서는 후속 단계에서 전체 실행 데이터를 수집하고, 구간별 행동 품질을 Flow Matching 손실의 가중치에 반영하는 확장을 고려한다. 이 경우 낮은 비음수 가중치는 해당 행동의 모방 기여도를 줄이는 역할을 하며, 실패 행동에 직접적인 벌점을 부여하는 것과는 구분된다. 구체적인 구간 판정 기준과 가중치 설정, 실제 수행 능력의 변화는 구현 및 평가 단계에서 확인해야 한다.

## 2.9. Reward-Weighted Flow Matching 기반 행동 품질 학습

Flow Matching 기반 로봇 정책은 관측 조건에서 시연 행동 궤적을 생성하도록 학습한다. 일반적인 모방학습에서는 학습 데이터에 포함된 행동을 동일한 정답으로 취급하므로, 정상적인 행동과 상대적으로 품질이 낮은 행동 사이의 차이를 직접 반영하기 어렵다. 특히

---

장시간 조작 작업에서는 접근, 파지, 이송, 배치 과정이 연속적으로 연결되기 때문에 일부 구간의 작은 오류가 후속 행동의 실패로 이어질 수 있다.

사람 개입 기반 모방학습은 정책이 실제로 방문한 실패 상태에서 전문가가 수행하는 복구와 교정 행동을 수집함으로써, 기존 시연에 포함되지 않은 상태에 대한 대응 능력을 높인다. 그러나 이러한 방식은 주로 실패가 발생한 이후에 어떤 행동으로 복구해야 하는지를 제공한다. 실패를 유발한 이전 자율 행동이 얼마나 부적절했는지, 성공한 행동 중 어떤 궤적이 더 안정적인지, 또는 동일한 작업을 수행한 여러 행동 중 어떤 행동을 우선해야 하는지에 대한 상대적인 품질 정보는 충분히 제공하지 못한다.

Reward-Weighted Flow Matching(RWFM)은 이러한 한계를 보완하기 위한 행동 품질 기반 정책 학습 방법이다. RWFM은 시연 또는 자율 실행에서 얻은 행동에 품질 정보를 부여하고, 그 품질에 따라 Flow Matching 손실에 대한 각 행동의 학습 기여도를 조절한다. 품질이 높은 행동은 정책 학습에 더 큰 영향을 주고, 일반적인 행동은 기존 모방학습과 유사한 비중으로 반영하며, 품질이 낮은 행동은 학습에 미치는 영향을 줄이는 방식이다.

이를 통해 모델은 모든 행동을 동일하게 재현하는 것이 아니라, 주어진 관측 조건에서 어떤 행동을 더 선호해야 하는지를 학습할 수 있다. 예를 들어 정상적으로 파지와 배치에 성공한 궤적이나 실패 이후 안정적으로 복구한 궤적은 정책의 행동 분포를 형성하는데 더 크게 반영할 수 있다. 반대로 충돌, 정체 또는 실패를 유발한 행동은 모델이 그대로 모방할 가능성을 줄일 수 있다.

RWFM은 초기 모방학습이나 사람 개입 기반 학습을 대체하는 방법이라기보다, 이미 학습된 정책의 행동 선택을 추가로 조정하는 후속 학습 방법으로 이해할 수 있다. 기존 시연 데이터는 정상적인 작업 수행 능력을 유지하는 기준으로 사용하고, 사람 개입 데이터는 실패 상태에서의 복구와 교정 능력을 보완한다. RWFM은 여기에 실제 자율 실행에서 나타난 행동의 상대적인 품질을 반영하여, 정책이 정상 행동과 복구 행동을 선택하는 경향을 강화하고 실패를 유발하는 행동의 선택 가능성을 낮춘다.

Dagger 및 Human-in-the-Loop 방식과 비교하면 역할의 차이를 구분할 수 있다. Dagger 계열 방법은 정책이 방문한 상태에서 전문가의 행동을 수집하여 해당 상태에 대한 정답 행동을 추가하는 데 초점을 둔다. 반면 RWFM은 수집된 행동을 동일한 정답으로 취급하지 않고, 행동의 결과와 품질 차이를 학습 과정에 반영하는 데 초점을 둔다. 따라서 사람 개입 기반 학습이 “실패 상황에서 어떻게 복구할 것인가”를 학습하는 과정이라면, RWFM은 “여러 가능한 행동 중 어떤 행동을 더 선호할 것인가”를 조정하는 과정으로 볼 수 있다.

---

RWFM을 적용하면 연속적인 로봇 행동 궤적의 품질 차이를 학습에 반영할 수 있고, 긴 작업 과정에서 성공과 실패의 원인을 보다 세밀하게 구분할 수 있다. 또한 성공적인 정상 행동을 유지하면서 복구 행동의 활용도를 높이고, 실패를 유발하는 행동이 정책에 반복적으로 반영되는 문제를 완화할 수 있다. 이러한 특성은 파지와 배치가 연속적으로 연결되고, 초기 오차가 후속 동작에 누적되는 본 연구의 블록 조작 과제에 적합하다.

다만 RWFM은 보상이나 품질 정보가 정확하게 정의되어 있다는 전제에 의존한다. 낮은 품질의 행동에 대한 학습 비중을 줄이는 것은 해당 행동을 직접적으로 벌점화하는 것과 다르며, 최종 정책의 성능은 행동 품질을 얼마나 일관되게 평가하는지에 따라 달라질 수 있다. 따라서 RWFM은 완전한 강화학습 알고리즘이라기보다, Flow Matching 기반 모방학습에 행동 품질 정보를 결합한 보상 가중 정책 개선 방법으로 이해하는 것이 적절하다.

본 연구에서는 사람 개입 기반 복구 학습을 통해 확보한 정책의 행동 선택을 추가로 조정하기 위한 후속 학습 개념으로 RWFM을 도입한다. 이를 통해 정상 작업 수행과 실패 복구 능력을 유지하면서, 실제 자율 실행 과정에서 나타나는 행동의 상대적인 품질 차이를 정책 학습에 반영하고자 한다.

### 3. 연구 내용 및 시스템 설계

#### 3.1. 개발 단계별 제어 방식 및 최종 구조로의 전환

초기 개발에서는 정리 작업의 안정적인 수행을 우선 확보한 뒤 적층 작업으로 확장하는 순서로 개발을 진행하였다. 이에 따라 초기 ACT 실험과 OpenCV·FSM·ACT 하이브리드 단계의 데이터 수집 및 실물 시험은 정리 작업을 대상으로 수행하였다.

##### 3.1.1. 초기 개발 환경 구축 및 단일 블록 기반 ACT 실험

초기 개발 단계에서는 SO-101 로봇 팔을 안정적으로 제어하고, 이후 모방학습에 사용할 영상 및 관절 상태 데이터를 수집할 수 있는 개발 환경을 구축하였다. 로봇 제어용 컴퓨터에는 Ubuntu 기반의 LeRobot 실행 환경을 구성하고, SO-101의 리더 암과 팔로워 암을 연결하였다. 리더 암의 움직임을 팔로워 암이 추종하도록 설정하여, 사람이 직접 로봇을 조작하면서 시연 데이터를 기록할 수 있는 텔레오퍼레이션 환경을 마련하였다.

데이터 수집에 앞서 리더 암과 팔로워 암의 관절 순서, 회전 방향 및 초기 위치를 일치시키는 캘리브레이션을 수행하였다. 관절 표현이 일치하지 않으면 리더 암의 시연 동작

---

과 팔로워 암의 실제 동작 사이에 오차가 발생하므로, 각 관절의 초기 자세와 그리퍼 상태를 확인한 뒤 시연을 진행하였다. 이 과정에서 수집되는 영상, 관절 상태 및 행동 명령이 동일한 시간 순서로 기록되도록 데이터 수집 경로도 함께 점검하였다.

초기에는 상단 카메라와 손목 카메라를 기본으로 사용하고, 작업 장면을 더 넓게 관측하기 위해 측면 카메라를 추가하는 구성도 시험하였다. 카메라는 OpenCV 기반으로 수집하였으며, 영상 형식은 MJPG, 해상도는 640×480으로 설정하고 약 25~30 FPS로 운용하였다. 그러나 세 대의 카메라를 동시에 사용할 경우 USB 대역폭과 영상 처리 부하가 증가하여 프레임 드롭과 추론 지연이 발생하였다. 카메라 두 대를 USB 허브에 연결하고 한 대를 컴퓨터에 직접 연결하는 방식이 상대적으로 안정적이었으며, 이후 카메라 구성과 입력 경로를 단순화하는 방향으로 설계를 조정하였다.

첫 번째 학습 정책으로는 영상과 로봇 관절 상태를 입력으로 받아 연속적인 관절 행동을 생성하는 ACT를 적용하였다. 이 단계에서는 자연어 지시를 사용하지 않고, 카메라 영상과 관절 상태만으로 단일 블록의 접근, 파지, 이송 및 배치 동작을 재현하는 것을 목표로 하였다. 초기 실행 설정에서는 30 Hz 제어 주기에 맞춰 정책이 한 번에 여러 시점의 행동을 예측하도록 구성하였다. 예측된 행동 청크 중 일부를 순차적으로 실행하고, 청크의 잔여 행동이 줄어들면 새로운 관측을 요청하여 다음 행동 청크를 생성하는 방식으로 동작시켰다.

초기에는 로봇 제어용 컴퓨터에서 정책 추론까지 함께 수행하려 했으나, 실제 제어 주기가 약 8.8~9.2 Hz 수준으로 떨어져 목표 제어 주기인 30 Hz를 만족하지 못하였다. 이로 인해 관측과 행동 사이에 지연이 발생하고, 파지 순간의 자세가 불안정해지는 문제가 나타났다. 이에 따라 이후에는 로봇 제어와 영상 수집은 로봇 컴퓨터에서 수행하고, 신경망 추론은 RTX 3090 GPU 서버에서 수행하는 클라이언트-서버 구조를 도입하는 방향으로 전환하였다.

단일 블록 실험에서는 블록의 위치가 제한적이고 작업 환경이 일정할 때 접근 및 파지 동작이 일부 재현되었다. 그러나 블록의 위치나 회전이 달라지면 파지점과 접근 자세가 흔들렸으며, 목표 영역으로의 배치와 작업 종료 후 초기 관측 자세로 복귀하는 동작은 안정적으로 수행되지 않았다. 또한 손목 카메라에 블록이 보이는 순간 파지 동작을 우선하는 경향이 나타나 상단 카메라를 이용한 목표 선택과 전체 작업 순서 판단이 충분히 이루어지지 않았다.

이 실험을 통해 초기 ACT 정책만으로는 블록 검출, 다음 대상 선택, 작업 단계 전환 및 장시간 연속 동작을 동시에 안정적으로 처리하기 어렵다는 점을 확인하였다. 또한 추론

---

자연, 카메라 입력 구성, 캘리브레이션 오차 및 시연 데이터의 범위가 실제 실행 성능에 직접적인 영향을 미치는 것으로 분석하였다. 이러한 결과를 바탕으로 이후에는 추론을 GPU 서버로 분리하고, 목표 검출과 작업 단계 전환을 OpenCV 및 FSM으로 보완하며, 단일 블록 중심의 데이터를 다양한 위치와 자세로 확장하는 방향으로 시스템을 변경하였다.

### 3.1.2. ACT 기반 5블록 전체 작업 시도

초기 단일 블록 실험 이후에는 ACT 정책이 5개의 블록을 하나의 에피소드에서 연속적으로 처리할 수 있는지 확인하였다. 이를 위해 블록의 접근, 파지, 이송 및 배치 과정을 중간에 중단하지 않고 하나의 시연 데이터로 기록하였다. 당시 대부분의 ACT 실험은 약 50~125개 에피소드 규모로 수행되었으며, 초기 5블록 전체 작업 데이터셋은 약 100개 에피소드와 123,700개 프레임으로 구성하였다.

이와 같은 데이터 규모를 설정한 이유는 Hugging Face에 공개된 LeRobot 기반 pick-and-place 데이터셋을 참고했기 때문이다. 당시에는 해당 데이터셋에서 약 100개 내외의 에피소드만으로도 집기와 놓기 동작이 수행되는 점에 주목하여, 본 과제에서도 유사한 규모의 데이터로 학습이 가능할 것으로 판단하였다. 그러나 당시 참고한 공개 데이터셋은 하나의 물체를 고정된 위치에서 집어 제한된 범위의 바구니나 목표 지점에 놓는 단순한 구조였으며, 물체의 회전 방향이나 여러 물체 중 다음 대상을 선택하는 과정은 크게 고려하지 않았다. 본 과제는 무작위로 배치된 5개 블록을 서로 다른 위치와 방향에서 순차적으로 파지하고, 서로 다른 위치와 방향의 블록을 순차적으로 파지하여 목표 영역에 배치해야 하므로 요구되는 데이터의 범위와 다양성이 훨씬 컸다. 당시에는 이러한 작업 조건의 차이를 충분히 고려하지 않은 채 에피소드 개수만을 기준으로 데이터 규모를 설정하였다.

또한 초기 데이터 수집에서는 블록에 접근하는 동작을 자동화하지 않고, 작업자가 텔레오퍼레이션으로 각 블록을 하나씩 직접 파지하였다. 작업자는 블록을 집은 뒤 목표 영역까지 이동하여 배치하고, 다시 다른 블록을 집기 위해 작업판 위를 이동하는 전 과정을 수동으로 조작하였다. 이로 인해 에피소드마다 블록에 접근하는 방향, 이동 경로, 속도, 호버 높이 및 복귀 자세가 달라졌으며, 동일한 작업에 대해서도 서로 다른 관절 궤적이 기록되었다. 결과적으로 학습 데이터 내 행동 분포가 넓어지고, 동일한 시각적 상태에 대응하는 행동 라벨의 일관성이 낮아졌다.

블록을 목표 영역에 배치한 뒤 다음 블록으로 이동하는 과정에서는 로봇 팔과 그리퍼가 상단 카메라의 시야를 가리는 현상도 빈번하게 발생하였다. 이때 상단 카메라는 다음 블

---

록의 위치와 전체 작업판의 상태를 충분히 관측하지 못했으며, 손목 카메라에는 현재 파지 대상만 크게 나타났다. 따라서 모델 입력 영상이 작업의 진행 상태에 따라 크게 달라졌고, 다음 블록을 선택하는 데 필요한 전역적인 위치 정보가 불안정하게 제공되었다.

초기 수집에서는 5개 블록을 집는 순서도 시계방향으로 고정하였다. 이 방식은 모델이 블록의 색상이나 실제 작업 지시를 바탕으로 대상을 선택하기보다, 블록의 공간적 배치와 시계방향 이동 패턴을 작업 순서의 단서로 학습하게 만들 가능성이 있었다. 학습 데이터와 유사한 배치에서는 해당 순서가 재현되었지만, 블록의 위치가 달라지거나 일부 블록의 관측이 가려지는 경우에는 다음 대상 선택이 불안정하게 나타났다.

ACT는 자연어 지시나 별도의 목표 검출 결과를 사용하지 않고, 상단·손목 카메라 영상과 로봇 관절 상태에 대응하는 시연 행동을 직접 학습하였다. 여러 시점의 행동을 action chunk 형태로 출력하여 전체 동작의 연속성을 유지하고자 하였으나, 데이터에 포함된 특정 이동 순서와 접근 궤적이 실제 작업 목표보다 강한 학습 단서로 작용하였다. 그 결과 첫 번째 블록의 접근과 파지는 일부 재현되었지만, 이후 블록을 선택하거나 배치 후 다음 동작으로 전환하는 과정에서는 오류가 누적되었다.

사후 분석 결과, 당시 ACT 실험의 주된 원인은 본 과제의 복잡도에 비해 데이터 수와 데이터 다양성이 모두 부족했다는 점이었다. 공개 데이터셋의 약 100개 에피소드를 참고했지만, 하나의 물체를 제한된 목표 위치에 놓는 작업과 무작위로 배치된 5개 블록을 순차적으로 처리하는 작업은 필요한 데이터 커버리지에서 큰 차이가 있었다. 여기에 수동 텔레오퍼레이션으로 인한 불규칙한 궤적, 상단 카메라 가림, 시계방향 순서 편향이 더해지면서 모델이 다양한 위치와 작업 상태를 충분히 학습하지 못하였다.

이에 따라 이후에는 단순히 에피소드 수를 늘리는 것뿐 아니라, 블록 접근 및 복귀 구간을 자동화하여 궤적의 변동을 줄이고 텔레오퍼레이션은 파지와 배치처럼 정밀한 조작이 필요한 구간에 집중하도록 수집 방식을 변경하였다. 또한 블록 검출과 작업 단계 관리를 보완하기 위해 OpenCV 및 FSM 기반 모듈을 결합하는 방향으로 제어 구조를 변경하였다.

### 3.1.3. OpenCV·FSM·ACT 하이브리드 방식

초기 ACT 실험에서는 소규모 시연 데이터로 5개 블록의 선택과 접근, 파지, 배치를 연속적으로 학습하였다. 이 과정에서 사람이 직접 다음 블록으로 이동하는 궤적이 일정하지 않았고, 이동 중 로봇팔이 상단 카메라의 시야를 가리는 문제가 있었다. 이러한 조건에서는 다음 대상을 선택하는 전역 작업과 블록 근처에서 수행하는 정밀 파지를 하나의 정책

---

으로 학습하기 어려웠다.

이에 OpenCV 기반 검출기, 유한 상태 기계(Finite State Machine, FSM), ACT를 결합한 하이브리드 구조를 시도하였다. OpenCV는 상단 영상에서 블록의 위치와 색상을 검출하고, FSM은 Observe 자세에서 블록 검출, 대상 선택, 대표 상공 자세로의 이동, ACT 제어로의 전환, 배치 완료 후 Observe 자세 복귀까지의 작업 순서를 관리하였다. ACT는 블록 상공에 도달한 뒤 미세 정렬, 파지, 들어 올리기, 목표 영역으로의 이송 및 배치를 담당하였다. 다만 이 단계의 학습 데이터에는 OpenCV와 FSM이 수행한 검출·상공 접근·배치 후 복귀 구간을 포함하지 않고, 상공 자세에 도달한 시점부터 ACT가 수행하는 파지·이송·배치 구간만 기록하였다. 개발 과정에서 초기 HSV 검출기를 배경 차분과 색상 캘리브레이션 사용하는 방식으로 개선하였으며, 이 절에서는 초기 접근 구조와 후속 검출기 개선을 구분하여 설명한다.

#### 3.1.3.1. 경기장 대표 지점과 접근 자세 보정

초기 자동 접근에서는 경기장에 8개의 대표 지점을 설정하고, 각 지점의 영상 좌표와 블록 상공에 대응하는 로봇의 관절 자세를 저장하였다. 대표 지점은 A1, A2, A3, B1, B2, C1, C2, C3으로 구성하였으며, 검출된 블록의 중심에서 가장 가까운 대표 지점의 자세를 접근 목표로 선택하였다.

각 보정점에는 상단 영상의 픽셀 좌표와 shoulder\_pan, shoulder\_lift, elbow\_flex, wrist\_flex, wrist\_roll 및 gripper의 위치 제어값을 함께 기록하였다. 이를 통해 블록을 검출한 뒤 사전에 확인한 상공 자세로 접근할 수 있도록 구성하였다. 각 제어값은 로봇 인터페이스의 위치 표현을 따르므로, 그리퍼를 포함한 모든 값을 동일한 각도 단위로 해석하지는 않는다.

코드 3-1 대표 접근 자세의 저장 형식

```
{
  "label": "A1",
  "u": 76.0,
  "v": 120.6,
  "pose": {
    "shoulder_pan.pos": -43.21,
    "shoulder_lift.pos": -32.84,
    "elbow_flex.pos": 16.40,
    "wrist_flex.pos": 36.40,
    "wrist_roll.pos": -14.77,
    "gripper.pos": 35.87
  }
}
```

검출된 블록 중심을  $(u, v)$ ,  $i$ 번째 보정점의 영상 좌표를  $(u_i, v_i)$ 라 하면, 가장 가까운 보정점은 영상 평면의 유클리드 거리를 이용하여 선택한다.

$$d_i = \sqrt{(u - u_i)^2 + (v - v_i)^2} \quad (3-1)$$

$$i^* = \arg \min_i d_i \quad (3-2)$$

코드 3-2 최근접 보정점과 목표 자세 선택

```
points = self.runtime["pregrasp_points"]
best = None
best_dist = float("inf")

for point in points:
    du = float(point["u"]) - float(u)
    dv = float(point["v"]) - float(v)
    dist = math.sqrt(du * du + dv * dv)
    if dist < best_dist:
        best, best_dist = point, dist

label = str(best["label"])
pose = {key: float(value) for key, value in best["pose"].items() }
```

이 방식은 영상 좌표마다 역기구학을 계산하지 않고도 접근 자세를 재현할 수 있다는 장점이 있었다. 다만 실제 블록 위치와 대표 지점 사이의 오차가 남고, 블록의 회전 방향도 접근 자세 선택에 직접 반영되지 않았다. 상공 도달 이후의 미세 정렬과 파지뿐 아니라, 블록을 들어 올린 뒤 목표 영역으로 이송하고 내려놓는 동작까지 ACT가 담당하도록 구성하였다.

### 3.1.3.2. 블록 검출과 작업 영역 설정

초기 검출기는 상단 카메라 영상을 HSV 색공간으로 변환한 뒤, 빨강·노랑·나무색·초록·파랑에 해당하는 색상별 임계값으로 후보 영역을 분리하였다. cv2.inRange를 이용하여 이진 마스크를 생성하고, 형태학적 열기 연산으로 작은 잡음을 제거한 뒤 외곽선을 추출하였다.

코드 3-3 HSV 마스크 생성과 후보 중심 계산 설명용 재구성

```

hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
lower = np.array([h_min, s_min, v_min], dtype=np.uint8)
upper = np.array([h_max, s_max, v_max], dtype=np.uint8)
mask = cv2.inRange(hsv, lower, upper)

kernel = np.ones((3, 3), dtype=np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
contours, _ = cv2.findContours(
    mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

for contour in contours:
    moments = cv2.moments(contour)
    if moments["m00"] == 0:
        continue
    cx = moments["m10"] / moments["m00"]
    cy = moments["m01"] / moments["m00"]

```

블록 후보의 중심은 외곽선의 영상 모멘트로 계산하였다. 면적에 해당하는 영차 모멘트가 0인 후보는 제외하고, 나머지 후보의 중심 좌표를 자동 접근을 위한 위치 정보로 사용하였다.

$$c_x = \frac{M_{10}}{M_{00}}, \quad c_y = \frac{M_{01}}{M_{00}} \quad (3-3)$$

검출기 설정 단계에서는 상단 카메라 화면에서 전체 작업 영역, 블록을 배치할 목표 영역, 팔로워 암 주변의 검출 제외 영역을 직접 지정하였다. 전체 작업 영역은 블록 검출 범위를 제한하는 데 사용하고, 팔로워 암 주변 영역은 로봇 구조물이 블록 후보로 인식되지 않도록 제외하였다. 목표 영역은 이미 배치된 블록을 다음 파지 대상에서 제외하기 위한 기준으로 사용하였다. 지정한 영역은 마스크와 영역 내부 판정에 활용하였다.

$$M_{\text{valid}} = M_{\text{workspace}} \wedge \neg M_{\text{ignore}} \quad (3-4)$$

여기서  $\wedge$ 는 논리곱,  $\neg$ 는 논리 부정을 의미한다. 마스크는 경기장 밖의 물체나 고정 구조물이 후보로 선택되는 것을 줄이는 수단이다. 다만 로봇팔에 가려진 블록의 영상 정보를 복원하는 것은 아니므로, 검출 시점에는 로봇을 Observe 자세로 이동시켜 상단 시야를 확보하도록 하였다.

### 3.1.3.3. 배경 차분과 색상 캘리브레이션을 통한 검출 개선

후속 검출기에서는 고정 HSV 임계값에 대한 의존도를 줄이기 위해, 블록이 없는 배경 영상과 현재 영상의 차이를 이용하였다. opencv\_block\_detector.py의 후속 구현에는 배경 영상 정렬, 가우시안 블러, LAB 색차 및 명도차를 결합하는 전경 분리 경로가 포함되어

있다. 이는 앞서 설명한 초기 HSV 검출에 이후 개선이 누적된 구현이다.

현재 영상과 배경 영상을 LAB 색공간으로 변환한 뒤, a와 b 채널의 차이를 이용해 색차를 계산하였다. 현재 영상의 채널값을  $a_t$ ,  $b_t$ , 배경의 채널값을  $a_{bg}$ ,  $b_{bg}$ 로 나타내면 색차는 식 (3-5)와 같다.

$$D_C(u, v) = \sqrt{(a_t - a_{bg})^2 + (b_t - b_{bg})^2} \quad (3-5)$$

코드 3-4 LAB 색차를 이용한 전경 후보 생성

```
current_lab = cv2.cvtColor(
    current_blurred, cv2.COLOR_BGR2LAB
).astype(np.float32)
background_lab = cv2.cvtColor(
    background_blurred, cv2.COLOR_BGR2LAB
).astype(np.float32)

delta_a = current_lab[:, :, 1] - background_lab[:, :, 1]
delta_b = current_lab[:, :, 2] - background_lab[:, :, 2]
chroma_difference = cv2.magnitude(delta_a, delta_b)

chroma_boolean = (
    (chroma_difference >= chroma_threshold)
    & (saturation >= chroma_min_saturation)
)
```

색차만으로 분리하기 어려운 후보는 LAB의 L 채널을 이용한 명도차로 보완하였다. 밝은 배경 위에서 어두워진 부분과 어두운 배경 위에서 밝아진 부분을 각각 계산하되, 그림자 등이 그대로 후보가 되지 않도록 채도와 최소 색차 조건을 함께 적용하였다.

$$D_{\text{dark}} = L_{\text{bg}} - L_t, \quad D_{\text{bright}} = L_t - L_{\text{bg}} \quad (3-6)$$

색차, 어두워짐, 밝아짐으로 얻은 후보 마스크를 각각  $M_C$ ,  $M_D$ ,  $M_B$ 라 할 때, 결합 후보 마스크는 식 (3-7)과 같이 정리할 수 있다. 이후 형태학 연산과 후보 형상 필터를 적용한다.

$$M_{\text{fg}} = (M_C \vee M_D \vee M_B) \wedge M_{\text{valid}} \quad (3-7)$$

외곽선 후보는 면적뿐 아니라 볼록 껍질 대비 면적 비율(Solidity), 축 정렬 경계 사각형 대비 면적 비율(Extent), 회전 경계 사각형 대비 면적 비율(Rectangularity)을 이용해 필터링하였다. 후보 면적을  $A_C$ , 볼록 껍질 면적을  $A_H$ , 두 경계 사각형의 폭과 높이를 각각  $w$ ,

h 및  $w_r$ ,  $h_r$ 라 하면 다음과 같다.

$$\text{Solidity} = \frac{A_C}{A_H}, \text{ Extent} = \frac{A_C}{wh} \quad (3-8)$$

$$\text{Rectangularity} = \frac{A_C}{w_r h_r} \quad (3-9)$$

색상 분류에는 실제 경기장 조명에서 수집한 표본을 사용하였다. 색상마다 중앙과 네 모서리에서 표본을 수집하고, 여러 프레임에 걸쳐 관측한 LAB의 a, b 및 HSV의 S 값으로 색상 특징을 구성하였다. 중앙값과 중앙절대편차(MAD)를 이용하여 각 색상의 대표값과 변동 폭을 계산하였다.

$$x = [a_{\text{LAB}}, b_{\text{LAB}}, S_{\text{HSV}}]^T \quad (3-10)$$

검출 후보와 각 색상 대표값 사이의 정규화 거리를 비교하여 색상을 선택하고, 거리가 지나치게 크거나 색상 간 구분이 모호한 후보는 분류에서 제외하였다. 검출기는 회전 사각형의 방향도 산출할 수 있었으나, 8개 대표 접근 자세를 선택하는 단계에서는 중심 좌표만 사용하였다.

검출 결과의 예는 그림 3-1과 같다.

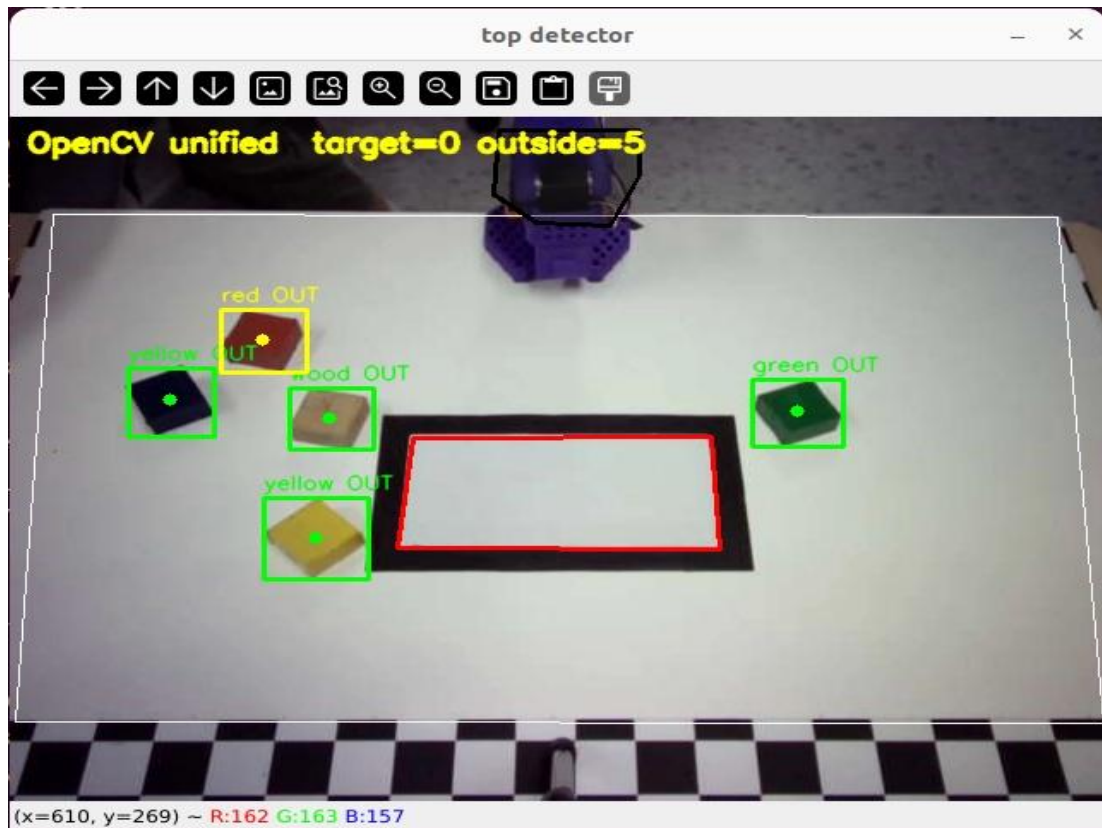


그림 3-1 OpenCV 통합 검출기의 블록 검출 결과

#### 3.1.3.4. 목표 선택과 FSM 기반 작업 순서

검출된 후보 중 다음에 처리할 블록을 선택하기 위해 목표 영역의 내부 여부를 먼저 판단하였다. 후보 중심과 목표 다각형의 위치 관계를 cv2.pointPolygonTest로 계산하고, 중심이 내부 또는 경계에 있으면 목표 영역 안에 있는 것으로 처리하였다.

코드 3-5 후보 중심을 이용한 목표 영역 내부 판정

```
in target = cv2.pointPolygonTest(  
    target_polygon.astype(np.float32),  
    (float(cx), float(cy)),  
    False,  
) >= 0
```

이 판정은 이미 배치된 블록을 다음 파지 대상에서 제외하기 위한 내부 기준이었다. 블록 중심이 목표 영역에 있다는 사실만으로 블록 전체의 포함 여부를 확인할 수는 없으므로, 공식 평가의 성공 판정과는 구분하였다.

초기 수동 시연에서는 블록을 시계방향으로 선택하였으나, 이후에는 빨강·노랑·나무색·초록·파랑의 색상 순서로 변경하였다. 색상별 목표 슬롯을 함께 지정하여 대상 선택과 배치 순서를 일정하게 유지하였다. 이 순서는 데이터 수집과 제어를 위한 내부 규칙이었다.

색상 순서 수집기에서는 저장된 에피소드 수를 기준으로 다음에 시연할 색상과 목표 슬롯을 순환 선택하였다.

코드 3-6 색상과 목표 슬롯의 순환 선택 핵심 흐름 재구성

```
color_order = ["red", "yellow", "wood", "green", "blue"]  
slot_order = ["S1", "S2", "S3", "S4", "S5"]  
  
sequence_index = dataset.num_episodes % len(color_order)  
target_color = color_order[sequence_index]  
target_slot = slot_order[sequence_index]
```

대상 위치는 여러 프레임의 검출 결과를 모아 결정하였다. 반복 관측에서 충분히 확인된 후보의 중심 좌표를 중앙값으로 집계하고, 같은 색상의 후보가 여러 개이거나 유효 검출이 부족하면 재확인하도록 구성하였다. FSM은 이러한 검출 결과와 현재 작업 단계를 바탕으로 관측, 접근, ACT 제어 전환, 배치 완료 후 Observe 자세 복귀의 상태 전이를 관리하였다.

#### 3.1.3.5. 관절 보간 기반 자동 접근과 파지·배치 실행

접근 목표 자세를 선택한 뒤에는 현재 관절 상태와 목표 관절 상태 사이를 선형 보간하

여 이동하였다. 현재 제어값 벡터를  $q_0$ , 목표값을  $q_g$ , 전체 보간 단계 수를  $N$ 이라 하면  $k$  번째 단계의 명령은 식 (3-11)과 같다.

$$q_k = (1 - \alpha_k)q_0 + \alpha_k q_g, \quad \alpha_k = \frac{k}{N} \quad (3-11)$$

$$N = \max(2, \lceil Tf \rceil) \quad (3-12)$$

여기서  $T$ 는 설정된 이동 시간,  $f$ 는 제어 주파수이며, 최소 2단계 이상의 명령을 생성하였다. 목표 자세에 포함되지 않은 제어 항목은 현재값을 유지한다. 코드 3-7은 move\_to\_pose의 핵심 보간 연산을 정리한 것이다.

코드 3-7 현재 자세와 목표 자세 사이의 관절 보간 핵심 로직 발췌

```
steps = max(2, int(duration s * self.config.fps))
for i in range(1, steps + 1):
    alpha = i / steps
    action = {
        key: (1.0 - alpha) * current[key]
            + alpha * float(target_pose[key])
        for key in keys
    }
    for key in self.robot.action_features:
        if key not in action and key in current:
            action[key] = current[key]
    self.send_script_action(action)
    time.sleep(self.config.environment_dt)
```

이 방식은 관절 공간에서 목표값을 연결하므로, 그리퍼가 작업 공간에서 반드시 직선으로 이동하는 것은 아니다. 따라서 대표 자세와 접근 높이를 사전에 점검하고, 블록 상공으로 이동하는 동안 그리퍼를 열린 상태로 유지하였다.

대표 상공 자세는 손목 카메라에 현재 처리할 블록이 들어오도록 설정하였다. 이는 상공 도달 후 ACT가 손목 영상에서 목표 블록을 관측한 상태로 미세 정렬과 파지를 시작하도록 하기 위한 것이었다.

데이터 수집에서는 OpenCV의 블록 검출, FSM의 대표 상공 자세 접근 및 배치 후 Observe 자세 복귀 구간을 기록에서 제외하였다. 자동 접근이 완료되면 작업자가 리더 암을 이용하여 블록에 접근하고, 미세 정렬과 파지를 수행한 뒤 들어 올려 목표 영역으로 이송하고 내려놓는 전 과정을 기록하였다. 이 데이터에서 ACT는 상공 자세 이후의 파지·이송·배치 동작을 학습하도록 하였다. 자동 접근이 완료되면 작업자가 리더 암을 이용하여 블록에 접근하고, 미세 정렬과 파지를 수행한 뒤 들어 올려 목표 영역으로 이송하고 내려놓는 전 과정을 기록하였다. ACT는 이 시연 데이터를 이용하여 상공 접근 이후

---

의 파지·이송·배치 동작을 학습하도록 하였다.

자율 실행에서는 OpenCV와 FSM으로 대상 블록을 검출하고 대표 상공 자세까지 이동한 뒤 ACT 정책으로 제어를 전환하였다. ACT는 해당 위치에서 블록의 파지·이송·배치를 수행하고, 배치가 끝나면 FSM이 Observe 자세로 복귀시켜 다음 블록을 다시 검출하도록 설계하였다.

**코드 3-8 자동 접근과 ACT 파지 및 배치의 실행 순서 핵심 흐름 재구성**

```
self.clear_action_queue()
self.move_to_pose(observe_pose, observe_time, "observe")
result = self.detect_top()
if result.chosen is None:
    return False

, pregrasp = self.nearest pregrasp pose(
    result.chosen.cx, result.chosen.cy
)
pregrasp["gripper.pos"] = open value
self.move_to_pose(pregrasp, approach_time, "pregrasp")
self.run_act_grasp_window(grasp_time)

drop_name, drop_pose = self.drop_pose_for_count(self.placed_count)
drop_pose = {
    key: value for key, value in drop_pose.items()
    if "gripper" not in key
}
self.move_to_pose(drop_pose, drop_time, drop_name)
self.open_gripper()
self.placed_count += 1
self.move_to_pose(observe_pose, observe_time, "observe")
```

배치 후 Observe 자세로 복귀하는 절차는 다음 블록을 동일한 기준 자세에서 관측하기 위한 것이었다. 목표 영역에서 곧바로 다음 블록으로 이동할 때 발생하던 상단 카메라 가림을 줄이고, 매번 작업판을 다시 관측한 뒤 다음 접근을 시작하도록 하였다.

이 구조에서는 검출과 접근, 파지 정책의 문제를 나누어 확인할 수 있었다. 다만 ACT 구간은 시간 기준으로 종료되었으며, 배치 명령 후 증가하는 내부 진행값은 실제 파지 성공을 확인한 결과가 아니었다. 이러한 실행상의 한계와 검출 불안정성은 후속 제어 구조를 검토하는 근거가 되었다.

#### **3.1.4. YOLO-IK 기반 전체 자동 제어 시도와 수집용 자동 접근으로의 전환**

OpenCV와 FSM을 이용한 8개 대표 구역 접근 구조를 구현한 뒤, 검출된 블록의 실제 위치와 방향을 반영하기 위해 YOLO와 역기구학(Inverse Kinematics, IK)을 결합한 제어 경로

를 추가로 시도하였다. OpenCV·FSM·ACT 기반 하이브리드 구조를 구현한 뒤, 검출된 블록의 실제 위치와 회전 방향을 반영하여 전체 작업을 자동화하기 위해 YOLO와 역기구학(Inverse Kinematics, IK)을 결합한 제어 경로를 설계하고 구현을 시도하였다. 이 경로에서는 상단 카메라에서 YOLO로 블록을 검출하고, 검출된 위치와 방향을 작업판 좌표 및 로봇 좌표로 변환한 뒤 IK를 이용해 관절 목표값을 계산하도록 구성하였다. 계산된 관절 목표값을 바탕으로 블록 접근, 파지, 들어 올리기, 목표 영역으로의 이송, 배치 및 배치 후 복귀까지 전 과정을 로봇이 자동으로 수행하는 것이 목표였다. 따라서 이 단계에서는 파지 이후 동작을 ACT나 텔레오퍼레이션에 맡기는 것이 아니라, 검출·좌표 변환·IK·FSM을 연결한 완전 자동 제어를 지향하였다.

실험 결과 YOLO·IK 경로는 블록의 위치를 로봇 좌표와 관절 목표값으로 연결하여 전체 작업을 자동화할 수 있는 가능성을 확인하였다. 특히 YOLO 기반 색상 검출과 대상 선택은 안정적으로 수행되었으며, OpenCV·FSM·ACT 방식에서 나타났던 빨강·노랑·나무색·초록·파랑 순서에 따른 다음 블록 선택 및 위치 접근 오류를 해결하였다. 검출된 블록의 위치를 기준으로 상공까지 접근하는 동작도 비교적 정확하게 수행되었다. 그러나 상공에 도달한 이후 블록을 실제로 파지하는 과정에서 문제가 발생하였다. 블록의 회전 방향에 따라 그리퍼의 접근 방향, 손목 회전값, 하강 자세 및 파지 위치를 매번 다르게 계산해야 했지만, 이를 모든 회전 조건에서 안정적으로 구현하기는 어려웠다. 이러한 회전별 파지 자세 계산 문제는 OpenCV 기반 경로와 YOLO·IK 기반 경로 모두에서 공통적으로 나타난 한계였다. 한편 OpenCV 기반 검출은 실험실과 실제 시연장의 조명, 그림자, 배경 무늬 및 카메라 노출 차이에 민감하여 동일한 임계값을 안정적으로 적용하기 어려웠으므로 최종 제어 경로에서 제외하였다. 결과적으로 YOLO·IK 경로는 블록 검출과 상공 접근에는 효과적이었지만, 회전 방향을 반영한 안정적인 파지까지 포함한 전체 자동화를 달성하지 못하였다.

#### 3.1.4.1. YOLO·IK 기반 전체 자동 제어 목표와 처리 흐름

전체 자동 제어 경로의 블록 검출에는 YOLO11n을 사용하였다. 기존 125개 에피소드에서 에피소드당 6장의 후보 이미지를 추출하여 총 750장을 확보하였고, 중복 라벨을 제거한 뒤 영상과 라벨을 직접 검토하여 최종 370장의 데이터를 구성하였다. 학습 데이터와 검증 데이터는 각각 321장과 49장으로 분할하고 150 epoch 동안 학습하였다. 검증 결과 mAP@50은 0.971, mAP@50-95는 0.583, 정밀도는 0.966, 재현율은 0.976이었다. 나무색 블록의 mAP@50은 0.897로 다른 색상보다 낮았다.

학습된 YOLO11n은 상단 카메라 영상에서 블록의 위치와 색상 클래스를 검출하였으며,

검출 결과의 예시는 그림 3-x와 같다.

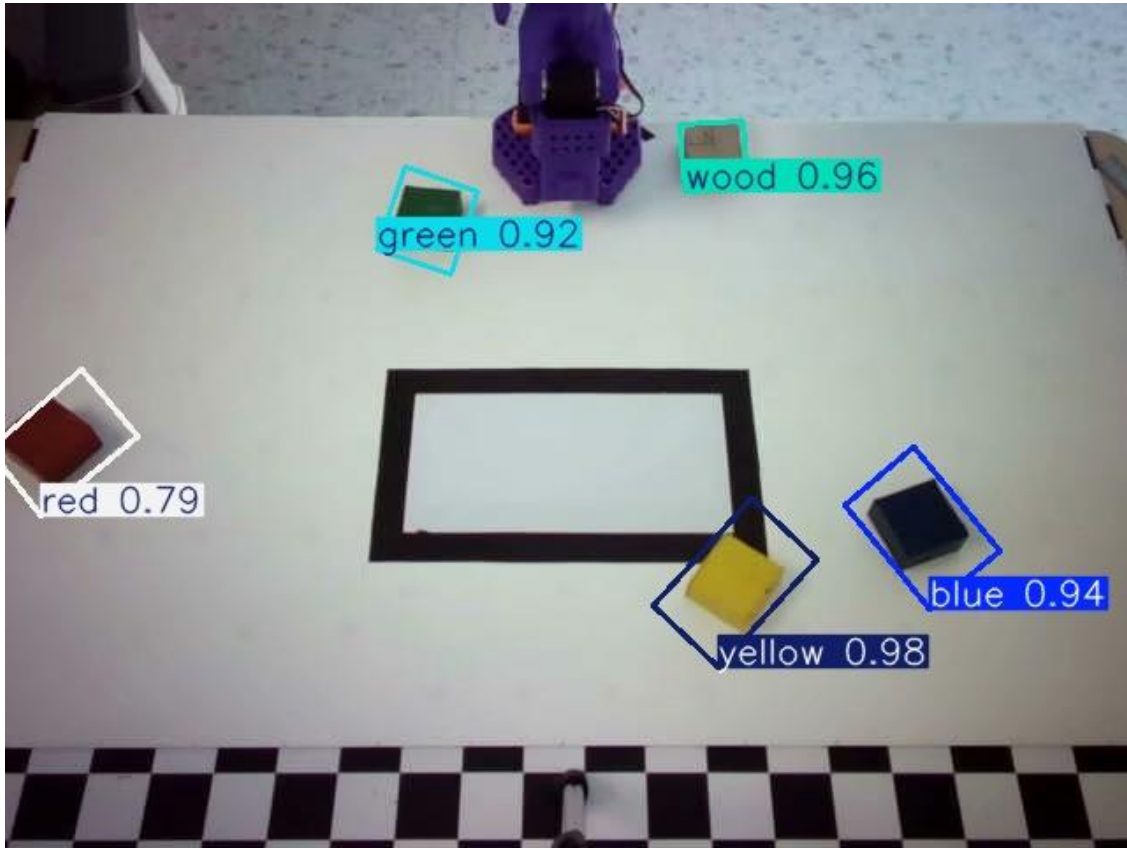


그림 3-2 YOLO 검출 결과 예시

YOLO 검출기는 상단 영상에서 블록 후보의 경계 상자와 중심 좌표를 제공하고, 색상 또는 클래스 정보를 이용해 다음에 처리할 대상을 선택하였다. 좌표 변환 모듈은 영상 픽셀 좌표를 작업판 좌표로 바꾸고, 보정된 작업판 좌표를 로봇 기준 좌표 또는 관절 목표값으로 연결하였다. IK 모듈은 목표 말단 자세를 만족하는 관절 해를 계산하고, 로봇 클라이언트는 계산된 해를 보간하여 실행하였다.

입력 영상에서 관절 명령까지의 처리 흐름은 다음과 같이 나타낼 수 있다.

$$q_{goal} = IK(p_{goal}, R_{goal}; q_{seed}) \quad (3-13)$$

여기서  $p_{goal}$ 은 픽셀 좌표를 작업판 좌표와 로봇 기준 좌표로 변환한 접근 목표 위치이며,  $R_{goal}$ 은 그리퍼의 목표 방향,  $q_{seed}$ 는 IK 계산의 초기 관절값이다.

코드 3-9 YOLO 검출부터 IK 접근까지의 전체 처리 흐름 설명용 재구성

```

frame = top_camera.read()
detections = yolo.detect(frame, conf=0.5, iou=0.5)
chosen = choose_next_block(detections, color_order)

if chosen is not None:
    u, v = chosen.cx, chosen.cy
    table_xy = pixel_to_table(u, v, homography)
    robot_xyz = table_to_robot(table_xy, affine_map)
    target_q = solve_ik(robot_xyz, pregrasp_orientation)
    move_to_joint_target(target_q, duration_s=1.2)

```

코드 3-9는 YOLO 검출부터 블록 상공 접근까지의 핵심 흐름을 나타낸 것이다. 실제 전체 자동 제어 경로에서는 이후 회전 방향에 따른 파지 자세 계산, 하강, 그리퍼 폐쇄, 들어 올리기, 목표 영역 이송, 그리퍼 개방 및 Observe 자세 복귀까지 순차적으로 수행하도록 설계하였다.

#### 3.1.4.2. 영상 영역 설정과 YOLO 검출 전처리

YOLO 검출 전에는 상단 영상에서 경기장 영역, 목표 영역 및 로봇팔 제외 영역을 다각형으로 지정하였다. 경기장 영역은 검출 범위를 제한하고, 목표 영역은 이미 배치된 블록을 다음 대상에서 제외하기 위해 사용하였다. 로봇팔 영역은 팔이 블록으로 오검출되는 것을 줄이기 위해 마스크에서 제외하였다. 이 전처리는 검출 범위를 제한하는 기능이며 가려진 블록을 복원하지는 않으므로, 검출 시에는 Observe 자세에서 상단 시야를 확보하였다.

코드 3-10 경기장·팔·제외 영역 마스크 적용 핵심 로직 발췌

```

arena_mask = polygon_mask(frame.shape[:2], arena_polygon)
arm_mask = polygon_mask(frame.shape[:2], arm_polygon)
ignore_mask = polygon_mask(frame.shape[:2], ignore_polygon)
valid_mask = arena_mask & ~arm_mask & ~ignore_mask

roi = frame.copy()
roi[~valid_mask] = 0
detections = detector.predict(roi)

```

목표 영역 내부 여부는 검출된 블록 중심을 목표 다각형에 대입하여 판정하였다. 중심이 목표 영역 안에 있으면 다음 파지 대상 후보에서 제외하거나 배치 상태 확인에 사용하였다. 이 판정은 과제 성공 여부를 대신하는 값이 아니라, 수집기와 FSM의 내부 상태를 갱신하기 위한 값이다.

코드 3-11 블록 중심의 목표 영역 포함 여부 판정

```
inside_target = cv2.pointPolygonTest(
    target_polygon.astype(np.float32),
    (float(cx), float(cy)),
    False,
) >= 0
```

#### 3.1.4.3. YOLO 검출 결과와 초기 접근 자세의 연결

YOLO로 검출한 블록 중심은 초기 접근 단계에서 사용한 8개 대표 pregrasp 지점과 연결하였다. 검출 중심과 대표 지점 사이의 영상 좌표 거리를 계산하고, 가장 가까운 지점의 관절 자세를 접근 목표로 선택하였다. 이 방식은 별도의 IK 계산 없이 빠르게 접근할 수 있다는 장점이 있었지만, 대표 지점 사이의 위치 오차가 남고 블록의 회전 방향을 접근 자세에 반영하지 못했다. 이러한 한계를 보완하기 위해 이후에는 픽셀 좌표를 작업판 좌표로 변환하고 IK를 계산하는 경로를 시도하였다.

#### 3.1.4.4. 픽셀 좌표의 작업판·로봇 좌표 변환 및 실물 보정

작업판의 네 모서리와 영상에서 대응하는 네 픽셀을 측정하여 호모그래피를 계산하였다. 작업판 좌표는 실험 설정에 맞춰 20 cm × 10 cm 평면으로 정의하였으며, 네 점의 대응 관계로 H를 구한 뒤 블록 중심에 적용하였다. 동차 좌표를 사용하면 픽셀 점 p와 작업판 점 X의 관계는 다음과 같다.

$$\lambda(XxY1) = H(uv1) \quad (3-14)$$

호모그래피만으로 로봇의 실제 파지 좌표가 결정되는 것은 아니므로, 작업판 좌표와 로봇 기준 좌표 사이에는 별도의 평면 보정 변환을 적용하였다. 대응점 N개로부터 3차원 로봇 좌표를 적합할 때, 행렬 A와 관측 벡터 b의 최소제곱 해는 다음과 같이 구하였다.

$$\theta = A + B \quad (3-15)$$

실제 계산에서는 역행렬을 직접 만들기보다 수치적으로 안정적인 최소제곱 연산을 사용한다. 적합 후에는 대응점별 잔차와 별도 검증점의 오차를 분리해서 확인하였다. 호모그래피 왕복 오차가 작아도 카메라와 로봇 좌표계의 원점, TCP 위치, 작업 높이가 잘못되면 실제 파지 위치가 어긋날 수 있기 때문이다.

코드 3-13 작업판 좌표에서 로봇 좌표로의 평면 최소제곱 보정

```

src = np.asarray(table_points, dtype=np.float32)
dst = np.asarray(robot_points, dtype=np.float32)
A = np.c_[src, np.ones(len(src))]
coef, _, _, _ = np.linalg.lstsq(A, dst, rcond=None)

# [x, y, 1] @ coef -> [X, Y, Z]
robot_xyz = np.r_[table_xy, 1.0] @ coef

```

좌표 변환의 수학적 적합 결과와 실제 로봇 접근 오차는 별도로 확인하였다. 초기 4점 호모그래피는 영상 좌표의 왕복 오차가 작았지만 실제 접근 위치가 약 2~3 cm 짧게 도달하였다. 이후 8개 티칭 지점을 이용한 보정에서는 기록된 최대 위치 오차가 10.4 mm로 나타났다. 기존 시연에서 추출한 629쌍의 영상·관절 대응점 중 RANSAC으로 555쌍을 선별한 경우 교차 검증 오차는 약 32 mm에서 11 mm로 감소하였다. 그러나 TCP 보정에서는 학습 지점의 적합 잔차가 3~7 mm였음에도 별도 검증 오차가 평균 104.8 mm로 나타나, 적합 지점의 오차만으로 실제 일반화 성능을 판단하기 어렵다는 점을 확인하였다.

이후 IK 모듈은 로봇 위치와 접근 방향을 목표로 받아 현재 관절 상태를 초기값으로 사용하였다. 해가 계산되면 순방향 기구학으로 얻은 말단 위치와 목표 위치의 차이를 다시 계산하여 허용 오차를 넘는 해를 제외하였다.

$$e = \|p_{FK}(q) - p_{goal}\|_2 \quad (3-16)$$

코드 3-14 현재 관절 상태를 seed로 사용하는 IK 수렴 및 위치 오차 확인

```

q_seed = robot.read_joint_positions()
q_solution = ik.solve(target_pose, seed=q_seed)
if q_solution is None:
    raise RuntimeError("IK did not converge")

fk_position = robot.forward_kinematics(q_solution).position
if np.linalg.norm(fk_position - target_position) > 0.010:
    raise RuntimeError("TCP position error is too large")

```

#### 3.1.4.5. IK 기반 관절 목표 연결과 블록 접근

계산된 관절 목표값은 한 번에 적용하지 않고 여러 제어 주기에 걸쳐 보간하였다. 또한 일부 경로에서는 현재 말단 위치와 목표 위치 사이를 Cartesian waypoint로 나누고, 각 waypoint마다 이전 IK 해를 초기값으로 사용하여 목표 자세를 반복 계산하였다. 이를 통해 블록 상공까지의 접근 궤적을 작업 공간에서 연속적으로 연결하고자 하였다.

코드 3-15 Cartesian waypoint와 반복 IK를 이용한 상공 접근 핵심 로직

```

start = robot.forward_kinematics(q_current).position
steps = max(2, int(np.linalg.norm(target - start) / 0.01))
q_prev = q_current

for i in range(1, steps + 1):
    alpha = i / steps
    waypoint = start + alpha * (target - start)
    q_next = ik.solve(pose at(waypoint), seed=q_prev)
    if q_next is None:
        raise RuntimeError("waypoint IK failed")
    robot.send joint action(q_next)
    q_prev = q_next

```

YOLO-ik 전체 자동 제어 경로에서는 상공에 도달한 뒤 블록의 회전 방향과 접근 방향을 반영하여 파지 자세를 계산하고, 하강·파지·들어 올리기·목표 영역 이송·배치까지 자동으 로 수행하도록 설계하였다. 다만 회전 방향별 파지 자세 계산과 실제 로봇의 도달 오차 가 안정적으로 보정되지 않아, 이후 데이터 수집기에서는 상공 또는 블록 근처까지 자동 접근한 뒤 정밀 파지와 이송·배치를 텔레오퍼레이션으로 수행하는 방식으로 전환하였다.

#### 3.1.4.6. 블록 회전각 검출과 파지 자세 계산

블록이 놓인 방향이 달라지면 같은 중심 좌표라도 손목 회전값과 파지 접근 방향이 달라 져야 한다. 이를 반영하기 위해 별도 실험에서는 YOLO의 축 정렬 경계 상자 안쪽을 잘 라 LAB 색차와 형태학 연산으로 블록 영역을 분리하고, 가장 큰 외곽선에 cv2.minAreaRect를 적용하여 회전각을 추정하였다. 반환 각도는 직사각형 표현의 긴 변 기준으로 정규화한 뒤 허용 범위 안에서 손목 회전 보정값으로 변환하였다.

코드 3-16 minAreaRect 회전각의 정규화와 손목 회전 보정 핵심 로직

```

rect = cv2.minAreaRect(block contour)
(_, _), (_, _), raw_angle = rect
angle = ((raw_angle + 45.0) % 90.0) - 45.0
wrist_delta = np.clip(
    angle * angle_scale,
    -max_align_deg,
    max_align_deg,
)
target_wrist_roll = wrist_roll_offset + wrist_delta

```

그러나 이 방법은 블록의 외곽선이 정확히 분리된다는 전제에 의존한다. 체크무늬 배경, 그림자, 모서리의 색상 손실이 외곽선에 섞이면 회전각이 불안정해지고, 노란색과 나무색 처럼 색상 거리가 작은 블록은 잘못 분류될 수 있다. 회전각을 얻더라도 해당 각도에서 로봇이 도달 가능한 파지 자세인지, 그리퍼가 블록을 안정적으로 물 수 있는지는 별도의 검증이 필요하였다.

실제 검증에서는 기록된 33개 대상 중 31개가 0°에 가까운 값으로 추정되어 minAreaRect 기반 회전각을 최종 방법으로 채택하기 어려웠다. 이에 블록의 중심과 방향을 함께 표현하기 위해 회전 박스 라벨링 도구를 구현하고 YOLOv8n-OBB를 학습하였다. OBB 데이터셋은 201장 이미지와 1,001개 블록 라벨로 구성하였으며, 학습 171장과 검증 30장으로 분할하여 100 epoch 동안 학습하였다. 검증 결과 mAP@50은 0.995, mAP@50-95는 0.954였다. 그러나 회전각 검출 성능이 높더라도 해당 각도를 실제 그리퍼의 손목 회전과 접근 자세로 변환하고 안정적인 파지를 보장하는 문제는 별도로 남아 있었다.

결과적으로 OpenCV와 YOLO 모두에서 블록의 위치와 회전 방향을 동시에 반영한 파지 자세 자동 보정은 구현 난도가 높았다. 회전각을 검출하더라도 해당 각도에서 로봇이 도달 가능한지와 그리퍼가 안정적으로 블록을 파지하는지는 별도의 검증이 필요하였다.

#### 3.1.4.7. FSM 기반 하이브리드 실행과 데이터 수집 범위

OpenCV·FSM·ACT 하이브리드 단계에서는 FSM이 Observe 자세 이동, 블록 검출, 대상 선택, 대표 상공 자세 접근, ACT 제어 전환 및 배치 후 Observe 자세 복귀를 관리하였다. 상공 자세에 도달한 이후에는 ACT가 파지·이송·배치를 수행하도록 구성하였다. 반면 이후의 YOLO·IK 단계에서는 ACT나 텔레오퍼레이션에 의존하지 않고, 검출·좌표 변환·IK를 연결하여 파지·들어 올리기·이송·배치·복귀까지 전체 과정을 자동화하는 경로를 시도하였다.

코드 3-17 색상 순서와 Observe 복귀를 포함한 FSM 실행 흐름

```
while placed_count < 5:
    move_to_observe_pose()
    detections = detect_top_camera()
    target = choose_color_target(detections, color_order)
    if target is None:
        continue

    move_to_pregrasp(target)
    run_act_or_teleop_grasp()
    move_to_drop_pose(placed_count)
    open gripper()
    placed_count += 1
    move_to_observe_pose()
```

규칙 기반 실행에서는 한 번 관측한 블록을 동작 대기열에 저장하여 반복적인 Observe 복귀와 재검출에 따른 이동을 줄이고자 하였다. 초기 4블록 정리에는 162초가 소요되었으나, 동작 대기열과 접근 경로를 개선한 후에는 5블록을 74초에 정리하는 단회 수행을 확인하였다. 적층 경로에서는 블록 높이에 따른 접근 및 하강 동작을 설계하고 시뮬레이션 수준에서 검토하였다. 이 결과는 검출·좌표 제어·규칙 실행을 연결한 경로의 동작 가

---

능성을 보여주었지만, 회전 방향을 반영한 안정적인 파지까지 보장한 결과는 아니었다.

수집 범위는 실행 모드에 따라 구분하였다. OpenCV-FSM-ACT 하이브리드 실험에서는 검출과 상공 접근 및 배치 후 Observe 복귀를 데이터셋에 포함하지 않고, 상공 자세에 도달한 뒤 ACT가 담당할 근거리 파지·들어 올리기·목표 영역까지의 이송·배치 시연을 기록하려고 하였다. 이후 자동 접근을 데이터 수집 보조로 계승한 수집기에서는 검출 결과로 블록 근처까지 이동한 뒤 작업자가 리더 암으로 미세 정렬, 파지, 들어 올리기, 이송 및 배치를 시연하였다. 즉 자동 접근은 접근 궤적을 일정하게 만드는 보조 단계이고, 파지 이후의 실제 조작은 시연 데이터에 포함되었다.

블록 하나를 배치한 뒤에는 바로 다음 블록으로 이동하지 않고 Observe 자세로 복귀한 다음 다시 상단 영상을 관측하였다. 이 절차는 다음 블록을 항상 유사한 시야와 로봇 자세에서 처리하게 하며, 배치 직후 이동 중 상단 카메라가 가려지는 문제와 자세 오차 누적을 줄였다. 초기 수동 수집에서 사용한 시계방향 선택은 이후 빨강, 노랑, 나무색, 초록, 파랑의 색상 순서로 정리하여 수집 조건을 일정하게 유지하였다.

#### 3.1.4.8. 전체 자동화의 한계와 수집용 자동 접근 전환

OpenCV 기반 검출기를 최종 제어 경로에서 제외한 주요 이유는 실험실과 실제 시연장의 환경 차이였다. 조명 밝기, 그림자, 배경 무늬 및 카메라 노출이 달라지면서 고정된 색상 임계값을 동일하게 적용하기 어려웠고, 촬영 조건에 따라 검출 결과가 달라질 수 있었다. 반면 YOLO 기반 검출은 블록의 색상과 위치를 안정적으로 검출하였으며, OpenCV-FSM-ACT 방식에서 나타났던 빨강·노랑·나무색·초록·파랑 순서에 따른 대상 선택 및 위치 접근 오류를 대부분 해결하였다. 검출된 블록의 상공까지 접근하는 동작도 비교적 정확하게 수행되었다. 그러나 상공 도달 이후에는 블록의 회전 방향에 따라 그리퍼 접근 방향, 손목 회전값, 하강 자세 및 파지 위치를 다르게 계산해야 했으며, 실제 파지 과정에서 실패가 반복되었다. 이러한 회전별 파지 자세 구현은 OpenCV 기반 경로와 YOLO-1K 기반 경로 모두에서 공통적으로 높은 난도를 요구하였다.

따라서 YOLO-1K를 이용한 전체 자동 제어 경로는 블록 검출과 상공 접근에는 효과적이었지만, 회전 방향을 반영한 안정적인 파지·이송·배치까지 일관되게 수행하는 최종 구조로 채택하기 어려웠다. 이에 검출기와 좌표-1K 제어기를 최종 추론 과정에서 사용하는 대신, 데이터 수집 단계에서 접근 궤적을 일정하게 만드는 보조 도구로 활용하였다. 초기 8개 대표 구역 접근 방식은 블록 상공 또는 근처까지 이동하는 자동 접근 로직으로 계승하고, 이후의 미세 정렬과 파지, 들어 올리기, 목표 영역으로의 이송 및 배치는 작업자가 텔레오퍼레이션으로 수행하여 기록하였다. 이후에는 대표 구역에 의존한 접근을 더 촘촘

---

한 위치 시나리오로 확장하여 다양한 위치와 회전 조건의 시연 데이터를 확보하였다.

좌표 변환과 IK를 이용한 전체 자동 제어 경로는 블록의 위치 접근에는 활용할 수 있었지만, 회전 방향에 따른 파지 자세와 실제 접촉 위치를 모든 조건에서 안정적으로 보장하기 어려웠다. 이에 최종 시연 데이터 수집기에서는 검출된 블록의 픽셀 좌표를 작업판 좌표로 변환한 뒤, 실측 대응점 기반 RBF 보간으로 블록 상공 접근 자세를 계산하였다. RBF 방식은 IK를 매번 새로 계산하는 대신 실제 로봇의 비선형 오차와 기계적 편차가 반영된 관절값을 이용하여 자동 접근 궤적을 일정하게 유지하기 위한 것이었다.

최종 정책 실행 단계에서는 OpenCV·YOLO 검출, 좌표 변환, IK 계산 및 FSM 기반 대상 선택을 사용하지 않고, SmoIVLA가 상단·손목 영상, 로봇 상태 및 자연어 지시를 바탕으로 블록 선택부터 파지·이송·배치까지의 연속 행동을 직접 생성하도록 구성하였다. 반면 시연 데이터 수집 단계에서는 녹화를 시작한 뒤 YOLO 검출과 좌표 변환·RBF 기반 자동 접근을 수행하고, 블록 상공에 도달하면 제어권을 작업자에게 전달하여 파지·들어 올리기·이송·배치 과정을 텔레오퍼레이션으로 기록하였다. 배치가 완료되면 작업자가 Enter 키를 입력하고, 로봇은 Observe 자세로 자동 복귀한 뒤 YOLO를 이용하여 다음 블록을 검출하고 다시 상공까지 접근하였다. 이 과정을 녹화를 중단하지 않은 상태에서 5개 블록에 대해 반복하여, 자동 접근·텔레오퍼레이션·배치·Observe 복귀·다음 블록 접근을 모두 하나의 에피소드로 저장하였다. 블록 처리 순서는 빨강·노랑·나무색·초록·파랑으로 고정하여 수집 조건을 일정하게 유지하였다.

### 3.2. 최종 시스템 구조

앞 절에서 설명한 개발 과정과 제어 방식의 전환을 바탕으로, 최종 시스템은 YOLO·IK 기반 규칙 제어가 아닌 SmoIVLA E2E 정책을 중심으로 구성하였다. 최종 시스템은 시연 데이터 수집, 정책 학습, GPU 서버와 로봇 클라이언트 사이의 실물 실행 단계로 구분된다. 전체 구성은 그림 3-2와 같다.

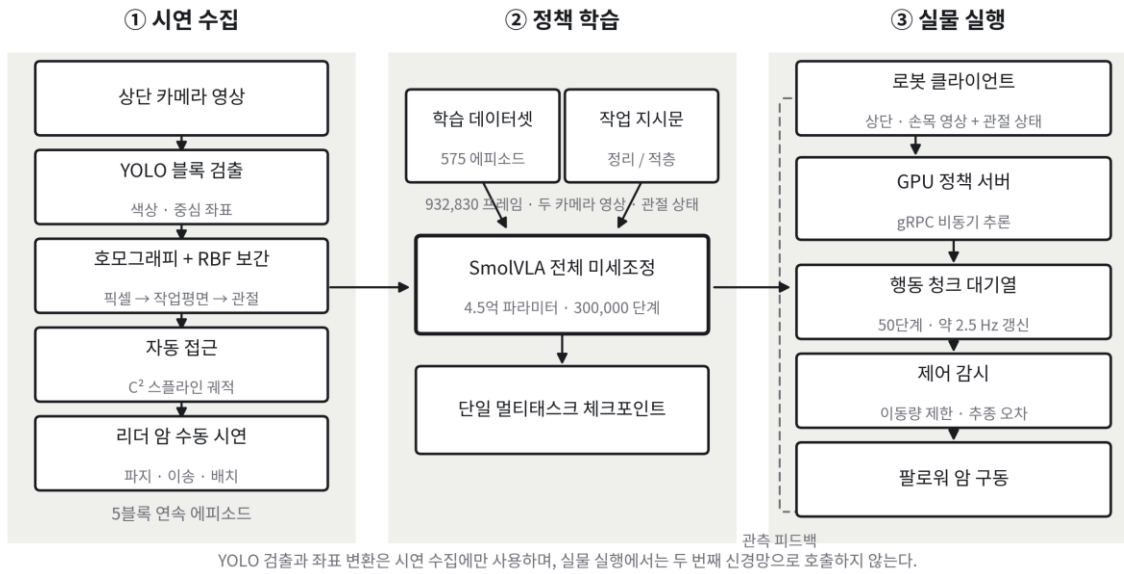


그림 3-3 시스템 구조도 (수집 → 학습 → 실행)

### 3.2.1. 최종 시스템 개요

최종 시스템은 시연 데이터 수집, 정책 학습 및 실물 실행의 세 단계로 구성된다.

시연 데이터 수집 단계에서는 녹화를 시작한 상태에서 상단 카메라의 YOLO 검출기를 이용하여 블록의 색상과 위치를 검출한다. 검출된 픽셀 좌표는 작업판의 실측 좌표로 변환하며, 실측 관절값으로 구성한 RBF 보간 모델을 이용하여 블록 상공의 접근 자세를 계산한다.

로봇이 블록 상공까지 자동으로 이동하면 제어권을 작업자에게 전달한다. 작업자는 리더 암을 이용하여 블록의 미세 정렬, 파지, 들어 올리기, 목표 영역으로의 이송 및 배치를 수행한다. 배치가 완료되면 작업자가 Enter 키를 입력하고, 로봇은 Observe 자세로 자동 복귀한다. 이후 YOLO 검출기를 이용하여 다음 블록을 식별하고, 동일한 방식으로 다음 블록의 상공까지 자동 접근한다. 이 과정은 녹화를 중단하지 않은 상태에서 하나의 연속 에피소드로 저장된다.

정책 학습 단계에서는 상단 카메라 영상, 손목 카메라 영상, 로봇 관절 상태, 자연어 작업 지시문 및 시연 행동을 입력으로 사용하여 SmoIVLA 정책을 미세 조정한다. 정리 작업과 적층 작업의 시연 데이터는 하나의 데이터셋으로 통합하며, 자연어 작업 지시문을 통해 작업별 목표와 배치 방식을 구분한다.

---

실물 실행 단계에서는 학습된 SmolVLA 정책이 두 카메라 영상, 로봇 상태 및 자연어 작업 지시문을 입력받아 블록의 선택, 파지, 이송 및 배치에 필요한 연속 행동을 생성한다. GPU 서버에서 계산된 행동 청크는 로봇 클라이언트로 전송되며, 클라이언트는 관절 이동량 제한과 추종 오차 감시를 적용한 후 팔로워 암에 명령을 전달한다. 정리 작업과 적층 작업은 동일한 정책 모델과 실행 구성을 사용하고, 작업 지시문만 변경하여 수행한다.

### 3.2.2. 시연 데이터 수집 구조

시연 데이터 수집은 다음 순서로 수행된다.

녹화 시작

- YOLO 기반 블록 검출
- 픽셀 좌표의 작업판 좌표 변환
- RBF 기반 블록 상공 접근 자세 계산
- 블록 상공까지 자동 이동
- 작업자 제어로 전환
- 파지·들어 올리기·이송·배치
- Enter 입력
- Observe 자세 자동 복귀
- 다음 블록 검출 및 자동 접근
- 5개 블록 처리 후 에피소드 종료

중요한 점은 YOLO 검출과 자동 접근이 녹화 이전에 별도로 수행되는 것이 아니라, 녹화가 시작된 상태에서 하나의 에피소드에 포함된다는 것이다. 따라서 데이터셋에는 블록을 검출하고 상공까지 접근하는 자동 동작, 작업자가 수행하는 파지와 이송, 배치 완료 후 Observe 자세로 복귀하는 동작 및 다음 블록으로 전환하는 과정이 모두 기록된다.

이 방식은 작업자가 매번 블록 상공까지 직접 이동해야 하는 부담을 줄이면서도, 실제 파지와 배치에 필요한 정밀한 조작은 작업자의 경험을 그대로 기록할 수 있다는 장점이 있다. 또한 매 블록을 처리한 뒤 Observe 자세로 복귀하므로, 다음 블록을 탐색하는 과정과 작업 상태가 일정한 기준 자세에서 다시 시작된다.

### 3.2.3. SmolVLA 정책 학습 구조

SmolVLA 학습에는 다음 네 종류의 입력이 사용된다.

- 상단 카메라 영상

- 
- 손목 카메라 영상
  - 로봇의 현재 관절 상태
  - 자연어 작업 지시문

시연 행동은 각 관측 시점에 대응하는 팔로워 암의 관절 목표값으로 저장된다. 모델은 현재 관측과 작업 지시문을 바탕으로 여러 시점의 미래 관절 행동을 하나의 행동 청크로 생성한다. 최종 설정에서는 `chunk_size=50`과 `n_action_steps=50`을 사용하여 한 번의 추론에서 50개의 연속 행동을 예측하도록 구성하였다.

정리 작업에서는 각 블록을 서로 다른 지정 위치에 배치하는 행동을 학습하고, 적층 작업에서는 중앙 목표 영역의 상공에서 높이를 조절하여 이전 블록 위에 내려놓는 행동을 학습한다. 두 작업은 파지 대상과 처리 순서를 공유하지만 최종 배치 방식이 다르므로, 자연어 지시문을 통해 작업 조건을 정책에 전달하였다.

#### 3.2.4. 최종 정책 실행 구조

최종 실물 실행에서는 시연 데이터 수집 단계에서 사용한 YOLO, 좌표 변환, RBF 보간 및 FSM을 사용하지 않는다. 대신 SmoIVLA가 상단·손목 영상, 관절 상태 및 자연어 지시문을 바탕으로 전체 작업을 직접 수행한다.

정책은 별도의 규칙 기반 대상 선택기 없이 다음 행동을 연속적으로 생성한다.

- 다음에 처리할 블록 선택
- 블록으로의 접근
- 그리퍼 정렬
- 블록 파지
- 블록 들어 올리기
- 목표 영역으로 이동
- 정리 또는 적층 방식에 따른 배치
- 다음 블록으로의 전환

정리와 적층은 동일한 모델 체크포인트와 동일한 실행 경로를 사용하며, TASK\_MODE에 따라 입력되는 자연어 지시문만 변경한다. 따라서 최종 구조는 작업별로 별도의 모델이

나 FSM을 준비하는 방식이 아니라, 하나의 멀티태스크 정책이 작업 지시를 조건으로 서로 다른 행동을 생성하는 방식이다.

### 3.2.5. 수집 단계와 실행 단계의 모듈 차이

데이터 수집 단계와 최종 실행 단계는 동일한 로봇을 사용하지만, 각 모듈의 역할은 서로 다르다. 데이터 수집 단계에서는 작업자가 안정적인 시연을 제공할 수 있도록 검출기와 좌표 보정 모듈을 보조 도구로 사용한다. 반면 최종 실행 단계에서는 이러한 모듈을 제거하고 SmolVLA가 전체 작업을 수행하도록 구성한다.

| 구분       | 시연 데이터 수집            | 최종 정책 실행     |
|----------|----------------------|--------------|
| 블록 검출    | YOLO 사용              | 사용하지 않음      |
| 좌표 변환    | 픽셀 좌표를 작업판 좌표로 변환    | 사용하지 않음      |
| 자동 접근    | RBF 기반 관절 보간 사용      | 사용하지 않음      |
| FSM      | Observe 복귀와 수집 순서 관리 | 사용하지 않음      |
| 파지·이송·배치 | 작업자가 리더 암으로 수행       | SmolVLA가 생성  |
| 자연어 지시문  | 데이터셋의 작업 조건으로 저장     | 모델 입력으로 사용   |
| 산출물      | 5개 블록 전체 시연 에피소드     | 팔로워 암 관절 목표값 |

따라서 YOLO와 RBF는 최종 정책을 구성하는 두 번째 신경망이나 제어기가 아니라, 학습 데이터의 접근 구간을 안정화하기 위한 데이터 수집용 모듈이다. 최종 추론에서는 SmolVLA가 관측 영상과 자연어 지시문을 바탕으로 대상 선택부터 배치까지의 행동을 직접 생성한다.

## 3.3. 하드웨어와 소프트웨어 환경

### 3.3.1. 로봇 및 카메라 구성

로봇 시스템은 SO-101 리더 암과 팔로워 암으로 구성하였다. 리더 암은 시연 단계에서 작업자가 직접 조작하는 입력 장치로 사용하고, 팔로워 암은 수집된 관절 행동을 실행하는 장치로 사용하였다. 로봇은 팔 관절과 그리퍼를 포함한 6개의 제어값을 사용한다.

---

시각 입력은 상단 카메라와 손목 카메라의 두 시점으로 구성하였다. 상단 카메라는 작업 판 전체와 블록의 전역적인 위치를 관찰하고, 손목 카메라는 그리퍼와 블록 사이의 상대적인 위치를 관찰한다. 두 카메라는 모두 640×480 해상도와 30 fps로 설정하였으며, USB 대역폭을 고려하여 MJPG 형식으로 영상을 수집하였다.

상단 카메라는 블록의 전체 배치와 목표 영역을 관찰하는 데 사용하고, 손목 카메라는 접근 및 파지 과정에서 그리퍼 주변의 세부 정보를 제공한다. 최종 학습과 추론에서는 두 카메라의 입력 키와 영상 전처리 과정을 동일하게 유지하였다.

### 3.3.2. Jetson 로봇 클라이언트와 GPU 서버

로봇 클라이언트는 Jetson Orin Nano에 구성하였다. Jetson은 카메라 프레임 수집, 관절 상태 읽기, 로봇 장치 제어 및 GPU 서버와의 통신을 담당한다. SmolVLA의 대규모 정책 연산은 Jetson에서 직접 수행하지 않고, 별도의 RTX 3090 GPU 서버에서 수행하였다.

GPU 서버는 두 카메라 영상과 관절 상태를 입력으로 받아 SmolVLA의 행동 청크를 계산한다. 계산된 행동 청크는 네트워크를 통해 Jetson 클라이언트로 전송되고, 클라이언트는 전달받은 행동을 대기열에 저장한 뒤 로봇 제어 주기에 맞추어 실행한다.

이와 같은 분리 구조를 사용함으로써 로봇 클라이언트에서는 카메라와 모터 통신에 집중하고, 대규모 모델 추론은 GPU 서버에서 수행할 수 있었다. 또한 Jetson의 CUDA·PyTorch 의존성 문제와 서버의 학습 환경을 분리하여, 로봇 장치 환경의 변경이 모델 학습 환경에 직접 영향을 주지 않도록 하였다.

### 3.3.3. 소프트웨어 및 학습 환경

소프트웨어는 LeRobot 프레임워크와 PyTorch를 기반으로 구성하였다. 데이터셋 기록과 재생, 정책 학습 및 로봇 실행은 LeRobot의 데이터 형식과 정책 인터페이스를 사용하였다. 학습 과정에서는 Weights & Biases를 이용하여 학습 손실, 학습률 및 체크포인트별 로그를 기록하였다.

주요 실행 환경은 다음과 같다.

| 구분        | 구성                         |
|-----------|----------------------------|
| 로봇 클라이언트  | Jetson Orin Nano           |
| 운영체제      | Ubuntu 기반 환경               |
| Python    | Python 3.12                |
| 로봇 프레임워크  | LeRobot                    |
| 딥러닝 프레임워크 | PyTorch                    |
| 학습 GPU    | NVIDIA RTX 3090            |
| GPU 메모리   | 24 GB                      |
| 영상 입력     | Top-Wrist, 640×480, 30 fps |
| 서버 통신     | gRPC 기반 비동기 통신             |
| 학습 기록     | Weights & Biases           |

최종 학습은 lerobot/smolvla\_base 체크포인트에서 시작하였다. 실물 실행에서는 학습 시 사용한 카메라 순서, 영상 키, 해상도 및 정규화 방식을 동일하게 유지하여 학습 입력과 실행 입력 사이의 차이를 줄였다.

#### 3.3.4. 카메라·관절·통신 설정

카메라는 장치 연결 순서에 따라 식별자가 변경되지 않도록 고정된 장치 경로를 사용하였다. 상단 카메라와 손목 카메라가 서로 바뀌면 동일한 프레임이라도 모델이 전혀 다른 의미로 해석할 수 있으므로, 데이터 수집과 추론 과정에서 카메라 키의 대응을 확인하였다.

관절 상태는 리더 암과 팔로워 암에서 동일한 관절 순서와 범위로 관리하였다. 캘리브레이션 과정에서는 각 관절의 최소·최대 범위, 초기 자세 및 그리퍼 방향을 확인하였다. 데이터 수집 시 기록된 관절값과 추론 시 팔로워 암에 전달되는 목표값의 표현이 일치하도록 구성하였다.

서버 통신은 gRPC 기반으로 구성하였다. 클라이언트는 일정 주기로 영상과 관절 상태를 묶어 서버에 전달하고, 서버는 자연어 지시문과 함께 정책 입력을 구성하여 행동 청크를 반환한다. 통신 과정에서 서버 응답이 늦어지더라도 기존 행동 대기열을 이용해 즉시 제어가 중단되지 않도록 하였다.

---

### 3.4. 모델 특성을 고려한 데이터셋 설계

본 연구에서는 시연 데이터를 수집하기에 앞서 SmolVLA의 입력과 행동 출력 특성을 고려하여 데이터셋의 배치 조건과 수집 절차를 설계하였다. SmolVLA는 카메라 영상, 자연어 작업 지시문 및 로봇 상태를 조건으로 연속적인 행동을 생성한다. 따라서 데이터셋을 구성할 때에는 블록의 위치와 색상뿐 아니라 주변 블록과의 관계, 파지 가능한 공간 및 해당 관측에 대응하는 행동 궤적을 함께 고려해야 한다.

본 연구의 설계 원칙은 작업 수행에 필요한 환경 변화는 다양하게 제공하고, 동일한 조건에서 작업자의 조작 편차로 발생하는 불필요한 행동 변동은 줄이는 것이다. 이를 위해 블록의 공간 위치, 색상, 밀집도, 배치 방향 및 미세 위치를 분산하는 한편, 자동 접근과 수동 정밀 조작을 결합하여 시연 절차의 일관성을 확보하고자 하였다.

본 절에서는 배치 시나리오와 시연 조건을 설계한 근거를 설명한다. YOLO 검출, 좌표 변환, RBF 보간 및 텔레오퍼레이션 전환의 구체적인 구현은 다음 절에서 설명한다.

#### 3.4.1. 모델 특성과 데이터 설계 원칙

SmolVLA는 영상과 자연어 지시에서 얻은 특징 및 로봇 상태를 이용하여 미래의 행동 청크를 생성한다. 시각 입력은 모델 내부의 특징 표현으로 변환되며, 행동 생성부는 이러한 관측 조건을 바탕으로 여러 시점에 걸친 관절 행동을 예측한다. 따라서 입력 환경의 분포와 시연 행동의 분포를 함께 관리하는 것이 중요하다.

예를 들어 특정 색상의 블록이 대부분 동일한 화면 위치에 배치되면, 색상과 위치 사이에 불필요한 상관관계가 형성될 수 있다. 이 경우 정책이 현재 영상에서 대상의 위치를 충분히 구분하기보다, 학습 데이터에서 반복적으로 나타난 색상과 접근 방향의 조합에 의존할 가능성이 있다. 이를 줄이기 위해 동일한 색상의 블록이 여러 위치와 주변 배치 조건에서 관측되도록 설계하였다.

행동 데이터에서도 유사한 문제가 발생할 수 있다. 동일한 위치의 블록에 접근하면서 작업자마다 이동 방향, 속도, 손목 회전 시점 및 정지 시간이 크게 달라지면, 관측 조건의 차이로 설명되지 않는 행동 변동이 증가한다. 여러 행동 경로가 존재하는 것 자체가 오류는 아니지만, 작업 수행에 필요하지 않은 조작 편차까지 반복적으로 포함되면 제한된 시연 데이터에서 학습해야 할 행동 분포가 복잡해질 수 있다.

이에 본 연구에서는 블록 위치, 색상, 회전 및 주변 공간과 같이 작업에 필요한 차이는 다양화하고, 접근 절차와 제어권 전환 방식 등은 가능한 한 일관되게 유지하였다. 다만

---

자동 접근으로 모든 행동을 고정하지는 않았다. 실제 접촉이 발생하는 미세 정렬, 파지 및 배치는 작업자가 수행하도록 하여, 블록의 회전과 주변 환경에 따라 필요한 조작 차이를 기록하였다.

이러한 설계는 모델 내부의 특정 표현이나 주의집중 방식이 개선되었음을 직접 검증한 결과가 아니라, 관측과 행동 사이의 대응을 명확히 하고 데이터 편종을 줄이기 위한 수집 원칙이다.

### 3.4.2. 공간·색상 분포와 조작 순서 설계

블록의 초기 위치가 특정 영역에 집중되지 않도록 작업 공간을 74개의 배치 가능 구역으로 구분하였다. 각 구역은 배치 시나리오를 생성하고 위치별 수집 빈도를 관리하는 기준으로 사용하였다. 가능한 범위에서 각 구역이 유사한 빈도로 등장하도록 하여, 작업판의 일부 위치만 반복적으로 촬영되는 것을 줄이고자 하였다.

74개 구역은 데이터 배치와 분포 관리에 사용하는 구분이며, 각 구역에 하나의 고정 관절 자세를 대응시키는 제어 구조를 의미하지 않는다. 실제 수집기의 자동 접근에서는 YOLO가 검출한 블록의 픽셀 좌표를 작업판 좌표로 변환하고, 실측 대응점 기반 RBF 보간으로 해당 위치의 접근 관절값을 계산한다. 따라서 같은 구역에 놓인 블록이라도 실제 검출 위치가 다르면 접근 목표값이 달라질 수 있다.

색상 조건은 빨강, 노랑, 나무색, 초록, 파랑의 다섯 종류로 구성하였다. 각 색상이 특정 위치에 고정되지 않도록 배치 시나리오에서 색상과 위치의 조합을 변경하였다. 밀집 묶음에 포함되는 색상 조합도 변경하여, 특정 색상끼리 항상 인접하거나 특정 색상이 항상 밀집 묶음의 바깥쪽에 위치하는 편향을 줄이고자 하였다.

반면 블록의 처리 순서는 빨강·노랑·나무색·초록·파랑으로 고정하였다. 이는 자연어 지시문과 시연 행동의 순서를 일치시키고, 수집 과정에서 작업자의 순서 혼동을 줄이기 위한 것이다. 처리 순서를 고정하더라도 각 색상의 초기 위치를 변경하면, 동일한 순서로 작업하면서 다양한 접근 방향과 거리 조건을 기록할 수 있다.

이와 같이 본 연구에서는 행동 순서와 초기 환경 조합을 분리하였다. 작업 순서는 일정하게 유지하되, 각 순서에서 처리해야 하는 블록의 위치와 주변 환경은 달라지도록 구성하였다. 여기서 균등화는 배치 시나리오의 생성 목표이며, 오류 에피소드 제외 이후의 최종 데이터 분포가 동일한 비율을 유지한다는 의미는 아니다.

---

### 3.4.3. 밀집도·방향·미세 위치 조건 설계

실제 블록 조작에서는 목표 블록의 위치뿐 아니라 주변 블록과의 간격이 파지 가능성에 영향을 준다. 블록이 서로 가까이 놓이면 그리퍼가 진입할 수 있는 공간이 줄어들고, 손목 방향에 따라 인접 블록과 접촉할 수 있다. 이에 분산 배치와 밀집 배치를 구분하여 수집 조건을 구성하였다.

밀집 배치는 블록이 완전히 맞닿는 0 mm 조건과 블록 사이에 5~10 mm의 간격이 있는 조건으로 나누었다. 두 조건은 밀집 시나리오 안에서 각각 약 절반을 차지하도록 계획하였다. 이를 통해 밀집 환경을 완전 밀착 상태 하나로만 표현하지 않고, 작은 간격이 존재하지만 여전히 파지 공간이 제한되는 상황도 포함하고자 하였다.

밀집 묶음의 방향은 작업판 평면을 기준으로 수평, 수직 및 대각선의 세 종류로 구성하였다. 여기서 수직 방향은 블록을 위로 쌓는 적층을 의미하는 것이 아니라, 작업판 평면에서 묶음이 세로 방향으로 놓이는 조건을 의미한다. 세 방향은 가능한 범위에서 1:1:1의 비율로 등장하도록 설계하였다.

동일한 밀집 묶음이라도 배치 방향이 달라지면 그리퍼가 접근할 수 있는 방향과 주변 블록의 위치 관계가 달라진다. 따라서 방향 조건을 다양화하여 특정한 묶음 형태와 접근 동작만 반복되는 것을 줄이고자 하였다.

구역 내부의 세부 위치에는 미세 위치 조건인 Micro-location을 적용하였다. 이는 동일 구역 안에서도 블록이나 묶음을 상·하·좌·우·중앙의 서로 다른 위치에 놓는 방식이다. 구역 중심에만 반복적으로 배치하는 것을 피하고, 구역 내부의 위치 변화도 시연에 포함하기 위한 것이다.

밀집 묶음에는 블록마다 서로 다른 미세 이동을 적용하지 않고 묶음 전체에 동일한 이동을 적용하였다. 개별 블록을 독립적으로 이동시키면 의도한 간격과 배치 방향이 변할 수 있기 때문이다. 묶음의 내부 관계를 유지한 상태에서 전체 위치를 변경함으로써, 동일한 밀집 구조가 서로 다른 절대 위치에서 관측되도록 구성하였다.

### 3.4.4. 파지 방향과 시연 궤적의 일관성

파지 시연은 블록의 회전 방향과 주변 공간의 여유를 함께 고려하여 구성하였다. 분산 배치에서는 주변 블록과의 간섭이 비교적 적으므로 다양한 회전·파지 방향 조건을 포함하고, 밀집 배치에서는 인접 블록과의 접촉 가능성을 고려하여 접근 가능한 방향을 제한하는 것을 원칙으로 하였다.

---

여기서 작업판 위 블록의 회전각, 그리퍼의 파지 방향 및 로봇의 손목 관절값은 구분해야 한다. 블록의 회전각은 작업판 좌표계에서 정의되는 환경 조건인 반면, 손목 관절값은 다른 관절의 자세와 함께 그리퍼 방향을 결정하는 제어값이다. 따라서 배치 시나리오의 방향 조건을 실제 손목 관절 명령으로 동일하게 해석하지 않았다.

밀집 조건에서의 방향 제한은 단순히 행동 다양성을 줄이기 위한 것이 아니라, 주변 블록을 건드리지 않고 그리퍼가 진입할 수 있는 시연을 확보하기 위한 것이다. 반면 블록의 위치나 방향이 달라져 필요한 미세 정렬까지 동일하게 강제하지는 않았다. 작업자는 자동 접근 이후 실제 블록 상태를 확인하고 파지 방향과 하강 자세를 조정하였다.

궤적의 일관성은 자동 접근으로 확보하고자 하였다. 작업자가 모든 접근을 직접 수행하면 같은 위치의 블록이라도 중간 경로, 이동 속도 및 손목 조정 시점이 달라질 수 있다. 이에 YOLO 검출과 좌표 변환, RBF 보간을 이용하여 블록 상공까지의 접근을 자동화하고, 접촉이 필요한 이후 구간을 텔레오퍼레이션으로 기록하였다.

블록 배치 후에는 Enter 입력을 받아 Observe 자세로 복귀하고, 다음 블록의 검출과 접근을 수행하도록 절차를 통일하였다. 녹화는 이 과정에서 중단하지 않았으므로, 자동 접근뿐 아니라 파지·이송·배치와 Observe 복귀, 다음 블록 접근까지 하나의 에피소드에 포함하였다.

이 방식은 행동 청크를 학습할 때 개별 파지 장면뿐 아니라 작업 단계 사이의 전환도 제공하기 위한 것이다. 다만 자동화된 구간에서 기록되는 복귀 동작과 대기 시간 역시 학습 대상이 되므로, 불필요한 정지나 우회 이동이 과도하게 포함되지 않도록 데이터 검토가 필요하다.

### 3.4.5. 제약 조건 기반 배치 시나리오 생성

배치 시나리오는 모든 변수를 독립적으로 무작위 추출하는 대신, 물리적으로 가능한 후보 중에서 부족한 조건을 보완하는 방식으로 구성하였다. 주요 관리 변수는 공간 구역, 밀집 묶음 크기, 배치 방향, 색상, 회전·파지 방향 조건 및 미세 위치이다.

공간 구역은 작업판 전체의 위치 분포를 관리하기 위한 변수이며, 밀집 묶음 크기는 함께 인접한 블록 수를 나타낸다. 배치 방향은 묶음의 수평·수직·대각선 조건을 구분하고, 색상은 블록의 위치 및 묶음 내 조합과 연결하여 관리한다. 미세 위치는 구역 내부의 세부 배치 변화를 나타낸다.

---

시나리오 생성에서는 현재까지 선택한 조건의 출현 빈도를 계산하고, 상대적으로 부족한 조건을 보완할 수 있는 후보를 우선 선택하는 Greedy 방식을 사용하였다. 각 단계에서 새로운 후보가 공간 구역, 색상 및 밀집 조건의 균형에 얼마나 기여하는지 고려하여 시나리오를 추가하는 방식이다.

다만 개별 변수의 빈도를 맞추는 것만으로 모든 변수 조합이 균등해지는 것은 아니다. 예를 들어 색상별 출현 횟수가 같더라도 특정 색상이 밀집 조건에 더 자주 포함될 수 있다. 따라서 개별 변수의 분포뿐 아니라 색상과 위치, 묶음 크기와 방향 등 주요 조합의 편중도 함께 고려하도록 설계하였다.

후보를 선택할 때에는 통계적 균형보다 물리적 실행 가능성을 우선하였다. 작업 영역을 벗어나는 배치, 의도하지 않은 블록 중첩, 로봇의 접근이 어려운 위치 및 그리퍼의 진입 공간이 부족한 조건은 시나리오 선정 과정에서 제한하였다. 밀집 조건 자체는 수집 대상에 포함하되, 작업자가 실제로 시연할 수 있는 배치인지 확인하는 과정이 필요하다.

따라서 본 연구의 시나리오 생성은 작업 가능한 영역과 배치 제약 정의, 후보 생성, 현재 분포 확인, 부족한 조건 평가 및 후보 선택의 순서로 구성하였다. 이는 모든 조합의 완전한 균등화나 최적해를 보장하는 방법이 아니라, 물리적 제약 안에서 주요 조건의 편중을 줄이기 위한 실용적인 구성 방식이다.

#### **3.4.6. 실제 배치 절차와 분포 점검**

생성된 시나리오는 작업자가 실제 작업판에 블록을 배치하는 절차로 연결하였다. 밀집 조건에서는 작은 위치 차이로도 블록 사이의 간격과 파지 공간이 달라지므로, 시나리오의 수치와 실제 배치 사이의 차이를 줄이는 것이 중요하다.

이를 위해 밀집 묶음을 먼저 배치하는 Bundle-First 방식을 적용하였다. 작업자는 밀집 묶음에 포함되는 블록을 먼저 놓고, 묶음의 방향과 블록 사이의 간격을 확인한 뒤 나머지 분산 블록을 배치하였다. 이후 색상별 위치, 처리 순서 및 작업 영역 이탈 여부를 확인하고 녹화를 시작하였다.

분포 관리는 에피소드 수준과 프레임 수준을 구분하여 수행하도록 설계하였다. 에피소드 수준에서는 각 구역과 배치 조건이 몇 회의 시연에 포함되는지 확인한다. 프레임 수준에서는 해당 조건을 포함한 시연이나 행동 구간이 전체 학습 데이터에서 차지하는 시간적 비중을 확인한다.

---

이 구분이 필요한 이유는 동일한 에피소드 수가 동일한 학습 노출량을 의미하지 않기 때문이다. 밀집 배치에서는 미세 정렬에 더 많은 시간이 소요될 수 있고, 적층 작업에서는 높이를 맞추기 위한 하강 과정이 길어질 수 있다. 따라서 촬영 횟수가 같더라도 긴 시연에 포함된 상태와 행동이 더 많은 프레임을 차지할 수 있다.

또한 하나의 에피소드에 분산 블록과 밀집 블록이 함께 포함될 수 있으므로, 에피소드 전체를 하나의 조건으로만 분류하는 데에는 한계가 있다. 세부 조건의 프레임 비중을 정확하게 계산하려면 블록별 처리 구간이나 조건별 구간을 구분해야 한다. 이러한 구간 집계 없는 경우에는 배치 시나리오의 조건별 빈도와 에피소드 길이를 구분하여 해석해야 한다.

수집 이후에는 상단·손목 영상을 검토하여 잘못된 배치, 순서 오류, 파지 실패 및 불필요한 동작이 포함된 에피소드를 정제하였다. 정리 데이터는 원본 444개 에피소드에서 91개를 제외하여 353개로 구성되었으므로, 생성 단계에서 계획한 구역·색상·밀집 조건의 비율이 정제 이후에도 그대로 유지된다고 가정하지 않았다. 계획된 분포와 실제 촬영 분포, 정제 후 학습 분포는 서로 구분하여 관리할 필요가 있다.

종합하면 본 연구의 데이터셋 설계는 환경의 다양성과 시연 절차의 일관성을 함께 확보하는 데 목적이 있다. 블록의 위치·색상·밀집도·방향은 다양화하고, 자동 접근과 Observe 복귀 및 제어권 전환은 일정한 절차로 구성하였다. 이를 통해 작업에 필요한 관측 변화는 충분히 제공하면서, 작업자의 임의적인 접근 방식에서 발생하는 불필요한 행동 변동은 줄이고자 하였다. 다음 절에서는 이러한 설계 원칙을 실제 데이터 기록으로 연결한 최종 시연 데이터 수집 시스템을 설명한다.

### 3.5. 최종 시연 데이터 수집 시스템

#### 3.5.1. YOLO 검출과 RBF 기반 자동 접근

최종 데이터 수집기는 녹화가 시작된 후 상단 카메라 영상에서 블록을 검출한다. 검출 결과에는 블록의 색상과 중심 위치가 포함되며, 작업 영역 밖의 후보는 제외한다. 동일한 색상의 후보가 여러 개 검출되는 경우에는 신뢰도와 작업 영역 내 위치를 함께 고려하여 처리 대상 후보를 결정한다.

검출된 블록의 중심점은 영상의 픽셀 좌표로 표현된다. 이 좌표를 로봇이 접근할 수 있는 작업판 좌표로 변환하기 위해, 실제 작업판에서 측정한 대응점을 이용하여 좌표 변환 모델을 구성하였다. 픽셀 좌표와 작업판 좌표 사이의 변환은 평면 호모그래피를 이용하였다.

---

작업판 좌표가 계산되면 해당 위치의 블록 상공 접근 자세를 결정해야 한다. 초기에는 기구학 모델과 IK를 이용해 관절 목표값을 계산했지만, 실제 SO-101의 프레임 힘, 백래시 및 모터 기어 유격으로 인해 위치에 따른 오차가 크게 나타났다. 이에 최종 데이터 수집기에서는 45개의 실측 티칭 지점에서 얻은 작업판 좌표와 관절값을 이용하여 RBF 기반 관절 보간 모델을 구성하였다.

RBF 보간은 새 작업판 좌표와 가까운 실측 지점들의 관절값을 이용하여 해당 위치의 접근 자세를 계산한다. 이 방식은 이상적인 기구학 모델을 적용하는 대신, 실제 로봇에서 직접 측정한 관절값을 사용한다는 특징이 있다. 따라서 로봇의 비선형적인 기계 오차와 작업판 주변부에서 발생하는 자세 편차를 접근 모델에 반영할 수 있다.

기록된 좌표 모델의 평균 오차는 픽셀에서 작업판 좌표로 변환하는 단계에서 약 2.03 mm, 작업판 좌표에서 관절값을 보간하는 단계에서 약 0.23°로 나타났다. 이 값은 실측 대응점에 대한 모델 적합 오차이며, 실제 파지 성공률을 직접 의미하지 않는다. 실제 파지에는 검출 오차, 관절 추종 오차, 그리퍼 정렬 및 블록과의 접촉 조건이 함께 영향을 준다.

### 3.5.2. 자동 접근 후 텔레오퍼레이션 전환

RBF 보간으로 계산된 관절 목표값을 이용하여 로봇은 블록 상공까지 자동으로 접근한다. 접근이 완료되면 로봇의 제어권을 작업자에게 전달한다. 이때 작업자는 리더 암을 직접 조작하여 블록에 대한 미세 정렬을 수행한다.

자동 접근은 블록의 대략적인 위치와 상공 자세를 맞추는 역할을 담당하며, 블록과 그리퍼가 실제로 접촉하는 마지막 과정은 작업자가 수행한다. 이 구조는 회전 방향과 블록의 미세한 위치 차이를 사람이 직접 보정할 수 있도록 하여, 전체 자동 제어에서 반복적으로 발생했던 회전별 파지 자세 오류를 줄이기 위한 것이다.

자동 접근과 수동 조작 사이의 전환은 녹화를 중단하지 않은 상태에서 이루어진다. 따라서 접근 완료 이후의 텔레오퍼레이션 행동은 앞선 자동 접근 행동과 시간적으로 연결된 상태로 저장된다.

### 3.5.3. 파지·이송·배치 시연

작업자는 리더 암을 이용하여 블록에 그리퍼를 정렬하고, 블록을 파지한 뒤 들어 올린다. 이후 목표 영역까지 이동하여 정리 작업에서는 지정된 슬롯에 블록을 배치하고, 적층 작업에서는 목표 중앙 상공으로 이동한 후 수직 방향으로 내려 블록을 쌓는다.

---

정리 작업에서는 각 블록이 서로 겹치지 않도록 목표 영역 내부의 서로 다른 위치에 배치한다. 적층 작업에서는 이미 놓인 블록의 높이를 고려하여 다음 블록의 하강 위치를 조정한다. 두 작업은 블록을 처리하는 색상 순서는 동일하게 유지하고, 최종 배치 방식만 다르게 구성하였다.

수동 시연 구간에는 파지 전의 미세 정렬, 그리퍼 개폐, 들어 올리기, 목표 위치로의 이동, 하강 및 그리퍼 개방이 모두 포함된다. 따라서 모델은 단순히 블록을 잡는 동작뿐 아니라, 블록을 안정적으로 이동하고 목표 위치에 내려놓는 전체 행동을 학습할 수 있다.

#### **3.5.4. Enter 입력과 Observe 자세 복귀**

블록 배치가 완료되면 작업자는 Enter 키를 입력한다. Enter 입력은 현재 블록의 작업이 끝났음을 알리고 다음 블록 처리를 준비하는 신호로 사용된다.

Enter 입력을 받은 수집기는 로봇을 Observe 자세로 자동 복귀시킨다. Observe 자세는 상단 카메라로 작업판 전체를 확인하고 다음 블록을 검출하기 위한 기준 자세이다. 이 과정에서 로봇은 작업자가 직접 이동시키는 것이 아니라 미리 정의된 관절 목표값과 이동 절차에 따라 자동으로 복귀한다.

Observe 복귀 과정 역시 녹화가 중단되지 않은 상태에서 에피소드에 포함된다. 따라서 데이터셋에는 블록을 배치한 직후 다음 블록을 탐색하기 위해 기준 자세로 돌아가는 행동도 포함된다.

#### **3.5.5. 다음 블록 검출과 반복 처리**

Observe 자세에 도달하면 상단 카메라에서 현재 남아 있는 블록을 다시 검출한다. 이미 배치된 블록은 작업 상태와 위치 정보를 이용하여 처리 대상에서 제외하고, 아직 처리되지 않은 블록 중 다음 순서에 해당하는 블록을 선택한다.

선택된 블록의 중심 픽셀 좌표는 다시 작업판 좌표로 변환되고, RBF 보간 모델을 이용하여 상공 접근 자세를 계산한다. 이후 로봇은 다음 블록 상공까지 자동 이동하고, 작업자는 다시 리더 암으로 파지와 배치를 수행한다.

이 과정을 빨강, 노랑, 나무색, 초록, 파랑 블록에 대해 반복한다. 다섯 번째 블록의 배치가 완료되면 에피소드가 종료되며, 전체 영상과 관절 행동이 하나의 연속 데이터로 저장된다.

---

### 3.5.6. 전체 과정의 단일 에피소드 기록

최종 수집기에서 하나의 에피소드는 다음 과정을 모두 포함한다.

1. 녹화 시작
2. YOLO 기반 블록 검출
3. 픽셀 좌표와 작업판 좌표 사이의 변환
4. RBF 기반 상공 접근 자세 계산
5. 첫 번째 블록 상공까지 자동 접근
6. 작업자의 파지·이송·배치
7. Enter 입력
8. Observe 자세 자동 복귀
9. 다음 블록 검출 및 자동 접근
10. 동일한 과정의 반복
11. 5개 블록 처리 후 에피소드 종료

이러한 원테이크 방식은 초기의 단일 블록 데이터나 파지 구간만 기록하는 방식과 다르다. 하나의 에피소드 안에 목표 선택, 접근, 파지, 이송, 배치, 복귀 및 다음 작업으로의 전환이 모두 포함되므로, 최종 정책은 개별 동작뿐 아니라 여러 블록을 연속적으로 처리하는 작업 흐름까지 학습할 수 있다.

## 3.6. 최종 데이터셋 구성과 품질 관리

### 3.6.1. 정리·적층 데이터 구성

최종 데이터셋은 정리 작업과 적층 작업의 시연 데이터를 통합하여 구성하였다. 정리 작업에서는 다섯 개 블록을 순서대로 집어 지정된 목표 위치에 서로 겹치지 않게 배치하였다. 적층 작업에서는 동일한 순서로 블록을 집어 목표 중앙 상공으로 이동한 후 수직으로 하강하여 탑 형태로 쌓았다.

두 작업은 블록의 처리 순서와 기본적인 파지 동작을 공유하지만, 배치 단계에서 서로 다른 관절 행동을 요구한다. 정리 작업은 수평 방향으로 서로 다른 위치에 배치해야 하

고, 적층 작업은 하나의 수평 위치를 유지하면서 블록 높이에 따라 하강 위치를 조정해야 한다.

### 3.6.2. 에피소드 및 프레임 수

최종 학습 데이터셋은 정리 353개 에피소드와 적층 222개 에피소드로 구성하였다. 전체 에피소드 수는 575개이며, 총 프레임 수는 932,830개이다.

| 작업 | 에피소드 수 | 프레임 수   | 평균 길이   |
|----|--------|---------|---------|
| 정리 | 353    | 496,633 | 약 46.9초 |
| 적층 | 222    | 436,197 | 약 65.5초 |
| 전체 | 575    | 932,830 | 약 54.1초 |

영상 프레임률은 30 fps로 설정하였다. 각 프레임에는 상단 카메라 영상과 손목 카메라 영상이 함께 대응하며, 로봇 관절 상태와 시연 행동도 동일한 시간 기준으로 기록된다. 두 카메라의 영상은 하나의 시점에 대한 서로 다른 관측이므로, 카메라 수를 기준으로 독립적인 에피소드나 프레임 수를 중복 계산하지 않았다.

### 3.6.3. 데이터 정제와 제외 기준

정리 작업은 최초 444개 에피소드에서 시작하였다. 이후 영상 검토를 통해 불필요한 정지, 비정상적인 접근, 파지 실패가 포함된 에피소드 및 작업 흐름이 중단된 에피소드를 제외하여 최종 353개 에피소드로 정제하였다.

에피소드 검토에서는 상단 영상과 손목 영상을 함께 확인하였다. 상단 영상에서는 대상 선택과 목표 위치를 확인하고, 손목 영상에서는 파지 시점의 그리퍼 정렬과 블록 접촉 상태를 확인하였다. 두 영상 중 하나만으로 판단하기 어려운 경우에는 관절 상태와 전체 작업 흐름을 함께 검토하였다.

다음과 같은 경우에는 학습 데이터에서 제외하였다.

- 블록의 처리 순서가 지시문과 다른 경우
- 파지 이전에 블록을 밀거나 떨어뜨린 경우
- 목표 영역에 배치하지 못하고 작업을 중단한 경우

- 불필요한 정지 시간이 과도하게 포함된 경우
- 카메라 프레임 누락 또는 관절 데이터 오류가 발생한 경우
- 자동 접근과 수동 시연 사이의 전환이 비정상적인 경우

이와 같은 정제 과정을 통해 단순히 에피소드 수를 늘리는 것보다 실제 작업 흐름을 일관되게 유지하는 데 중점을 두었다.

#### 3.6.4. 위치·색상·회전 조건 분포

데이터 수집에서는 특정 위치와 특정 색상에 시연이 집중되지 않도록 블록의 초기 위치와 회전 조건을 분산하였다. 정리 작업은 목표 영역의 여러 위치에 블록을 배치하도록 구성하였고, 적층 작업은 중앙 위치에 블록을 반복적으로 배치하도록 구성하였다.

블록 처리 순서는 빨강, 노랑, 나무색, 초록, 파랑으로 고정하였다. 처리 순서를 고정한 이유는 정책이 색상과 작업 지시를 바탕으로 다음 대상을 선택하는 과정을 일관되게 학습하도록 하기 위해서이다.

다만 정리와 적층은 최종 배치 위치의 분포가 다르다. 정리 데이터는 에피소드마다 다섯 개의 지정 위치에 블록을 나누어 배치하지만, 적층 데이터는 모든 블록을 하나의 중앙 위치에 순차적으로 배치한다. 정리 353개 에피소드에서 각 지정 위치가 한 번씩 사용된다고 가정하면 위치별 배치 횟수는 각각 353회이다. 반면 적층은 222개 에피소드에서 다섯 개 블록을 중앙에 배치하므로 중앙 배치 이벤트는 1,110회이다.

이 차이는 멀티태스크 학습에서 중앙 배치 행동이 상대적으로 강하게 나타날 수 있는 원인이 된다. 따라서 단순한 에피소드 수뿐 아니라 배치 이벤트의 위치별 분포도 함께 고려하였다.

### 3.7. 멀티태스크 학습 설계와 반복 실험

#### 3.7.1. 정리·적층 데이터 통합

초기에는 정리 작업만을 대상으로 단일 작업 정책을 학습하였다. 단일 작업 모델에서 지정 위치로 이동하는 행동이 확인된 이후, 적층 데이터를 추가하여 하나의 SmoIVLA 모델이 두 작업을 수행하도록 구성하였다.

정리와 적층 데이터는 동일한 관측 형식과 관절 행동 표현을 사용하므로 하나의 데이터셋으로 통합할 수 있었다. 각 에피소드에는 해당 작업을 설명하는 자연어 지시문을 연결

하였다. 모델은 동일한 카메라 입력과 관절 상태를 받지만, 지시문에 따라 정리 또는 적층 행동을 생성하도록 학습하였다.

### 3.7.2. 학습 설정 변화

멀티태스크 학습을 구성하는 과정에서 데이터 규모, 미세조정 방식, 배치 크기, 학습 단계, 이미지 증강 및 작업 지시문을 단계적으로 변경하였다.

| 실험 | 주요 설정                      | 관찰 및 목적                |
|----|----------------------------|------------------------|
| 1  | 정리 330개, LoRA, 150,000 단계  | 제한된 학습 범위와 파지 정밀도 확인   |
| 2  | 정리 330개, 전체 미세조정, 배치 16    | 단일 작업 기준 모델 구성         |
| 3  | 정리 444개, 전체 미세조정           | 데이터 확대에 따른 변화 확인       |
| 4  | 기존 학습의 단계 연장               | 학습률 감소 이후 추가 학습 영향 확인  |
| 5  | 정리·적층 통합, 각 222개           | 멀티태스크 통합 후 행동 변화 확인    |
| 6  | 정리 308개·적층 222개, 300,000단계 | 데이터 비율과 학습 단계 조정       |
| 7  | 정리 444개·적층 222개            | 정리 데이터 증가에 따른 편향 변화 확인 |
| 8  | 정리 353개·적층 222개, 300,000단계 | 최종 데이터와 지시문으로 학습       |

초기 실험에서는 LoRA와 전체 미세조정을 적용하여 파지와 배치 동작을 관찰하였고, 최종 기본 멀티태스크 모델에는 전체 미세조정을 채택하였다. 학습 방식별 갱신 범위와 선택 근거는 3.10.2절에서 설명하며, 실물 관찰 결과와 비교 조건의 차이는 4.3절에서 분석한다.

또한 기존 체크포인트에서 학습 단계를 단순히 연장하는 경우 학습률 감소 일정이 이미 종료되어 추가 학습 구간의 학습률이 지나치게 낮아질 수 있음을 확인하였다. 이에 최종 학습에서는 전체 학습 단계와 학습률 감소 단계를 모두 300,000단계에 맞추었다.

---

### 3.7.3. 중앙 배치 편향 분석

정리 단일 작업 모델은 지정된 여러 위치에 블록을 배치하는 행동을 보였다. 그러나 정리와 적층 데이터를 통합한 모델에서는 정리 지시문을 입력했음에도 블록을 목표 중앙으로 이동시키는 현상이 관찰되었다.

이 현상은 적층 데이터가 중앙 위치에 반복적으로 배치되는 구조와 관련이 있을 가능성이 있다. 정리 353개 에피소드에서 지정 위치별 배치 횟수는 각각 353회인 반면, 적층은 222개 에피소드에서 다섯 개 블록을 중앙에 배치하므로 중앙 배치 이벤트가 1,110회 발생한다.

따라서 중앙 위치에 대한 배치 행동은 정리 작업의 개별 목표 위치보다 약 3.14배 많이 나타난다. 적층에서는 블록 높이가 매번 달라지므로 모든 관절 행동이 동일한 것은 아니지만, 수평 위치에 대한 분포만 비교하면 중앙 위치가 상대적으로 과대표현된다.

이 분석은 중앙 편향의 가능성을 설명하는 데이터 분포 지표이며, 모델 내부에서 해당 원인이 직접 확인되었다는 의미는 아니다. 실제 편향에는 지시문 유사성, 영상 관측, 관절 행동의 분포 및 체크포인트별 학습 상태가 함께 영향을 줄 수 있다.

### 3.7.4. 체크포인트별 관찰

체크포인트 평가는 학습 손실만으로 판단하지 않고 실물 롤아웃을 함께 확인하였다. 정리 작업에서는 지정 위치를 구분하는지, 적층 작업에서는 중앙 상공으로 이동한 후 수직 하강하는지 확인하였다.

575개 에피소드 데이터셋을 이용한 학습에서는 약 180,000단계 체크포인트에서 정리 작업의 위치 구분이 일부 개선되었으나 중앙 편향과 적층 동작의 불안정성이 남았다. 이후 체크포인트에서는 정리 작업의 중앙 집중이 감소하고 적층 동작이 다시 관찰되는 변화를 확인하였다.

학습 손실은 시연 행동과 모델 예측 행동의 차이를 요약한 값이므로, 손실이 낮다고 해서 실제 작업 성공률이 항상 높아지는 것은 아니다. 데이터셋 구성과 작업 분포가 다른 실험의 손실을 직접 비교하지 않고, 동일한 데이터셋과 실행 조건에서 체크포인트별 행동 변화를 비교하였다.

---

### 3.8. 작업별 자연어 지시문 설계

#### 3.8.1. 정리 작업 지시문

정리 작업에서는 다섯 개 블록을 지정된 순서로 집은 뒤, 서로 다른 목표 위치에 배치하도록 지시하였다.

*Pick up the 5 blocks in sequence (red, yellow, wood, green, blue), then place each block separately into its designated target position.*

이 지시문은 각 블록을 하나의 중앙 위치에 놓는 것이 아니라, 블록별로 구분된 목표 위치에 배치해야 한다는 점을 명시한다. 특히 separately와 designated target position을 사용하여 정리 작업의 공간적 분리를 표현하였다.

#### 3.8.2. 적층 작업 지시문

적층 작업에서는 동일한 순서로 블록을 집은 뒤, 목표 중앙 상공에서 이전 블록 위에 쌓도록 지시하였다.

*Pick up the 5 blocks in sequence (red, yellow, wood, green, blue), then hover over the target area and stack each block on top of the previous block.*

적층 지시문에는 hover over와 on top of the previous block을 포함하였다. 이를 통해 단순히 목표 영역에 내려놓는 것이 아니라, 중앙 상공에서 수직으로 접근하고 이전 블록 위에 배치해야 한다는 작업 특성을 표현하였다.

#### 3.8.3. 지시문 차별화와 작업 조건화

초기 지시문은 두 작업 모두 place each at the target area와 같이 유사한 표현을 사용하였다. 이 경우 정리와 적층의 최종 목적이 충분히 구분되지 않을 수 있으며, 특히 두 작업의 배치 데이터가 함께 학습될 때 중앙 배치 행동이 정리 작업에 전이될 가능성이 있다.

수정된 지시문은 공통적으로 블록의 색상 순서와 파지 과정을 포함하지만, 배치 단계의 표현을 명확히 분리하였다. 정리 작업은 서로 다른 지정 위치를 강조하고, 적층 작업은 목표 중앙 상공과 이전 블록 위 배치를 강조한다.

실행 단계에서는 TASK\_MODE 환경변수에 따라 정리 또는 적층 지시문을 선택한다. 모델 체크포인트, 카메라 입력, 관절 상태 및 행동 실행 구조는 동일하게 유지하고, 자연어 지시문만 변경하여 작업 조건을 전달하였다.

### 3.9. 영상 전처리와 증강

#### 3.9.1. 학습·실행 입력 전처리 일치

학습과 실행에서 입력 영상의 크기, 채널 순서, 자료형 및 정규화 방식이 다르면 모델은 학습 과정에서 경험하지 않은 입력을 받게 된다. 초기 실행 과정에서는 영상의 자료형과 값의 범위가 학습 경로와 달라지는 문제가 확인되었다.

최종 입력 경로에서는 카메라 프레임을 부동소수점 형식으로 변환하고, 픽셀값을 정규화한 뒤 모델 입력 형식으로 전달하였다. 상단 카메라와 손목 카메라의 순서를 고정하고, 학습과 실행에서 동일한 카메라 키를 사용하였다.

#### 3.9.2. 영상 크기 조정과 패딩

원본 영상의 종횡비를 유지하면서 모델 입력 크기에 맞추기 위해 영상 크기를 조정된 뒤 패딩을 적용하였다. 영상의 가로세로 비율을 임의로 늘리거나 줄이면 블록의 형태와 위치가 달라질 수 있으므로, 비율을 유지한 상태에서 빈 영역을 추가하는 방식을 사용하였다.

최종 입력에서는 640×480 영상을 종횡비를 유지한 상태로 축소하고, 모델이 요구하는 정사각형 입력 크기에 맞추어 패딩하였다. 학습과 실행 모두 동일한 크기 변환 및 패딩 규칙을 적용하여 영상 속 블록 위치와 관절 행동 사이의 대응을 유지하였다.

#### 3.9.3. 색상·밝기·선명도 증강

실제 시연 및 평가 환경에서는 조명, 카메라 노출, 블록 표면의 반사 및 그림자가 달라질 수 있다. 이를 보완하기 위해 최종 학습에서는 위치를 변경하지 않는 색상 및 밝기 기반 증강을 사용하였다.

| 변환       | 범위         | 적용 여부 |
|----------|------------|-------|
| 밝기       | 0.8~1.2    | 적용    |
| 대비       | 0.8~1.2    | 적용    |
| 채도       | 0.5~1.5    | 적용    |
| 색조       | -0.05~0.05 | 적용    |
| 선명도      | 0.5~1.5    | 적용    |
| 회전·평행 이동 | 기하 변환      | 미적용   |

---

위 변환 중 최대 세 가지를 무작위로 적용하였다. 색상과 밝기 변화는 블록의 위치를 이동시키지 않으므로, 기존 관절 행동 라벨과 영상 속 블록 위치 사이의 공간적 대응을 유지할 수 있다.

### 3.9.4. 기하 증강 제외와 행동 라벨 정합성

초기 학습에서는 회전과 평행 이동을 포함하는 RandomAffine 증강을 사용하였다. 그러나 영상 속 블록의 위치를 변경하면서 관절 행동 라벨은 변경하지 않으면, 모델은 변형된 영상 위치와 원래 위치에 대한 행동을 동시에 학습하게 된다.

예를 들어 영상 속 블록이 오른쪽으로 이동했지만 관절 행동이 원래 위치를 향하고 있다면, 영상과 행동 사이에 인위적인 라벨 불일치가 발생한다. 로봇 조작에서는 수 픽셀 수준의 위치 변화도 파지 위치에 영향을 줄 수 있으므로, 최종 설정에서는 회전과 평행 이동을 제외하였다.

따라서 최종 증강은 조명과 색상 변화에 한정하고, 물체의 위치와 방향을 직접 변경하는 기하학적 변환은 사용하지 않았다.

## 3.10. 최종 SmolVLA 모델 학습 설정

### 3.10.1. 기반 모델과 VLM 구성

최종 모델은 lerobot/smolvla\_base 체크포인트에서 시작하였다. SmolVLA는 영상, 자연어 지시문 및 로봇 상태를 함께 처리하고 연속적인 관절 행동을 생성할 수 있는 VLA 구조이다.

본 연구에서는 상단 카메라와 손목 카메라의 영상을 입력으로 사용하고, 6개 관절 상태를 상태 입력으로 사용하였다. 정리와 적층 데이터는 하나의 멀티태스크 데이터셋으로 통합하고, 각 에피소드의 작업 지시문을 자연어 조건으로 연결하였다.

### 3.10.2. 미세조정 방식의 비교와 전체 미세조정의 선택 근거

본 연구에서는 SmolVLA의 과제 적응 방식으로 전체 미세조정(Full Fine-Tuning), LoRA 및 행동 생성부 중심 학습(Expert-only)을 검토하였다. 최종 기본 멀티태스크 모델에는 전체 미세조정을 적용하였다. 선택 근거는 관측 표현과 행동 생성을 함께 조정할 수 있다는 학습 범위의 특성과, 개발 과정에서 지정 위치 배치 및 연속 작업 수행을 관찰하였다는 점이다.

### 학습 방식별 갱신 범위

SmolVLA는 영상과 자연어 지시에서 얻은 특징 및 로봇 상태를 조건으로 미래 행동 청크를 생성한다. 따라서 과제 적응에는 관측에서 조작에 필요한 정보를 표현하는 과정과, 해당 정보를 실제 관절 행동으로 연결하는 과정이 함께 관련된다. 세 가지 학습 방식은 이러한 구성 요소 중 어떤 가중치를 갱신하는지에 차이가 있다.

| 학습 방식       | 주요 갱신 범위                                | 과제 적응의 특징                       |
|-------------|-----------------------------------------|---------------------------------|
| 전체 미세조정     | 설정에 따라 시각·언어 백본과 행동 생성부 및 상태 투영층을 함께 갱신 | 관측 표현과 행동 생성의 연결을 공동으로 조정       |
| LoRA        | 선택한 모듈의 기존 가중치를 고정하고 저랭크 적응 행렬을 학습      | 적용 모듈과 rank에 따라 갱신 범위와 표현력이 달라짐 |
| Expert-only | 시각·언어 백본을 고정하고 행동 생성부를 중심으로 갱신          | 사전학습된 관측 표현을 유지하면서 행동 생성을 조정    |

Expert-only는 학습 대상 모듈을 제한하는 방식이며, LoRA는 선택한 가중치의 변화량을 저랭크 형태로 제한하는 방식이다. 따라서 두 방법은 서로 배타적인 분류가 아니며, 행동 생성부에만 LoRA를 적용하는 구성도 가능하다. 실제 학습 범위를 판단할 때에는 방식의 명칭과 함께 동결 여부, 학습 대상 모듈 및 학습 가능한 파라미터 수를 확인해야 한다.

### 본 과제에서 필요한 적응

본 과제에서는 블록의 존재와 색상을 구분하는 것에 더해, 블록과 그리퍼의 상대 위치 및 방향을 파악하고 이를 정밀한 관절 궤적으로 연결해야 한다. 밀집 배치에서는 주변 블록과의 간격 및 그리퍼가 진입할 수 있는 방향도 중요하다. 또한 정리와 적층은 파지 과정을 공유하지만, 파지 이후에는 서로 다른 목표 위치와 배치 궤적을 요구한다.

따라서 사전학습된 모델이 블록을 시각적으로 식별할 수 있다는 사실만으로 본 과제의 조작에 충분하다고 판단하기 어렵다. 영상에서 작은 차이로 나타나는 그리퍼와 블록의 상대 위치가 파지 성공 여부를 바꿀 수 있으며, 동일한 관측에서도 자연어 지시문에 따라 지정 위치 배치와 수직 적층을 구분해야 한다. 본 연구에서는 이러한 정보를 행동 생성에 유용한 형태로 표현하고, SO-101의 관절 행동과 연결하는 적응이 필요하다고 판단하였다.

### Expert-only 적용 시 고려할 제약

---

Expert-only는 고정된 시각·언어 표현에 조작에 필요한 정보가 충분히 포함되어 있을 때 효율적인 선택이 될 수 있다. 그러나 해당 표현에서 정밀 파지 위치, 주변 블록과의 간격 또는 작업별 배치 의도가 충분히 구분되지 않는다면, 행동 생성부는 차이가 작은 특징으로부터 서로 다른 행동을 학습해야 한다.

예를 들어 그리퍼가 블록 중심에 정렬된 상태와 가장자리에 걸친 상태는 영상상 차이가 작지만, 이후에 필요한 행동은 달라질 수 있다. 이러한 차이가 백본 특징에 약하게 반영되어 있다면, 행동 생성부만 갱신하는 방식으로는 필요한 정밀도를 확보하기 어려울 가능성이 있다. 정리와 적층의 언어 조건을 서로 다른 배치 행동으로 연결하는 과정에서도 같은 관점을 적용할 수 있다.

다만 고정된 특징에도 필요한 정보가 충분히 포함되어 있다면 행동 생성부만으로 과제에 적응할 수 있다. 본 연구에서는 동일한 조건에서 백본 동결 여부만 변경한 대조 실험이 확보되지 않았으므로, 이러한 설명을 Expert-only의 일반적인 성능 한계나 본 과제의 확정된 실패 원인으로 해석하지 않았다.

### LoRA 적용 시 고려할 제약

LoRA는 선택한 모듈에 저랭크 변화량을 학습하므로, 적응 범위는 적용 위치와 rank에 영향을 받는다. 행동 생성부에만 적용하면 시각·언어 백본의 표현은 유지되며, 백본 일부에만 적용하면 행동 생성부의 갱신 범위는 별도 설정에 따라 제한될 수 있다. 따라서 학습 가능한 파라미터 수뿐 아니라 과제에 필요한 변화가 어느 모듈에서 발생해야 하는지도 고려해야 한다.

개발 기록의 rank 16 설정에서는 학습 가능한 파라미터가 742,656개로 전체의 약 0.165%였으며, rank 64 설정도 검토하였다. 이 수치는 해당 구성의 갱신 규모를 나타내지만, 파라미터 비율이 작다는 사실만으로 성능 부족을 설명할 수는 없다. 필요한 가중치 변화가 제한적인 과제에서는 적은 수의 적응 파라미터로도 충분한 성능을 확보할 수 있기 때문이다.

본 과제에서는 시각적 위치 정보, 자연어 작업 조건 및 관절 행동의 연결을 함께 조정해야 했을 가능성이 있다. 따라서 사용한 LoRA 설정의 적용 모듈과 rank가 이러한 적응 요구에 충분히 부합했는지를 성능 차이의 원인 후보로 고려하였다. 이를 검증하려면 동일한 데이터와 실행 조건에서 적용 모듈, rank 및 최적화 설정을 구분하여 비교해야 한다.

### 전체 미세조정과 시연 데이터 설계의 연결

---

전체 미세조정은 행동 손실을 줄이는 과정에서 시각·언어 표현과 행동 생성부를 함께 갱신할 수 있다. 본 연구에서는 이를 통해 정밀 파지에 필요한 관측 차이, 정리·적층의 배치 의도 및 SO-101의 관절 궤적 사이의 대응을 공동으로 조정하고자 하였다.

이러한 학습을 지원하기 위해 3.4절에서 설명한 데이터 설계 원칙을 적용하였다. 블록의 위치·회전·밀집 조건은 다양화하고, 블록 상공까지의 자동 접근과 배치 후 Observe 자세 복귀는 일정한 절차로 구성하였다. 접촉이 필요한 파지·이송·배치는 작업자가 직접 시연하여 환경에 따른 정밀 조작 차이를 기록하였다.

이 수집 방식은 작업과 관계없는 접근 편차를 줄이면서, 실제 환경 차이에 대응하는 행동을 제공하기 위한 것이다. 다만 이러한 이점은 전체 미세조정에만 한정되지 않으며, LoRA와 Expert-only를 적용할 때에도 동일한 데이터 품질 관리가 필요하다.

최종 기본 멀티태스크 학습에서는 `use_peft=false`, `freeze_vision_encoder=false`, `train_expert_only=false`, `train_state_proj=true`를 적용하여 시각 인코더와 VLM 백본, 행동 생성부 및 상태 투영층을 학습 대상으로 구성하였다. 이는 전체 미세조정이 모든 조건에서 우수하다는 결론에 따른 선택이 아니라, 본 과제의 적응 요구와 개발 과정에서 확보한 실물 동작을 바탕으로 한 선택이다. 학습 방식별 관찰 결과와 비교의 해석 범위는 4.3절에서 설명한다.

### 3.10.3.Action Chunk 설정

최종 모델은 한 번의 추론에서 50개의 연속 행동을 생성하도록 설정하였다. `chunk_size=50`은 정책이 현재 관측을 바탕으로 일정 길이의 행동 시퀀스를 생성하도록 하며, `n_action_steps=50`은 생성된 청크의 실행 길이를 의미한다.

청크가 너무 짧으면 서버 추론과 통신 요청이 빈번해지고, 청크가 너무 길면 새로운 관측을 반영하는 시점이 늦어질 수 있다. 50단계 설정은 연속 작업의 부드러움과 새로운 관측 반영 주기 사이의 균형을 고려하여 선택하였다.

### 3.10.4. 학습 단계·손실·체크포인트

최종 학습 설정은 다음과 같다.

| 항목       | 설정                     |
|----------|------------------------|
| 기반 모델    | lerobot/smolvla_base   |
| 학습 방식    | 전체 미세조정                |
| 학습 데이터   | 575개 에피소드, 932,830 프레임 |
| 배치 크기    | 16                     |
| 총 학습 단계  | 300,000                |
| 옵티마이저    | AdamW                  |
| 최대 학습률   | $2 \times 10^{-5}$     |
| Warmup   | 3,000단계                |
| 학습률 감쇠   | 코사인 감쇠                 |
| 감쇠 종료 단계 | 300,000단계              |
| 행동 청크 길이 | 50                     |
| 실행 행동 수  | 50                     |
| 영상 증강    | 색상·밝기·선명도 기반           |
| 기하 증강    | 미사용                    |

최종 모델의 학습 식별자는 smolvla\_multitask\_5blocks\_v3\_575ep\_fullft\_b16\_300k이다. 학습 과정에서는 15,000단계 간격으로 체크포인트를 저장하고, 주요 체크포인트를 실물 로봇에 적용하여 작업별 행동 변화를 확인하였다.

학습 종료 시점의 손실은 약 0.0480으로 기록되었다. 그러나 손실은 시연 행동과 예측 행동 사이의 평균 오차를 나타내는 값이므로, 실제 정리 완료율이나 적층 성공률과 동일한 의미로 해석하지 않았다. 최종 체크포인트는 학습 손실뿐 아니라 정리 작업의 위치 분리, 적층 작업의 수직 하강 및 전체 작업의 연속성을 함께 고려하여 평가하였다.

### 3.11.비동기 실행과 진단 도구

#### 3.11.1.GPU 서버·로봇 클라이언트 구조

최종 실행은 GPU 정책 서버와 Jetson 로봇 클라이언트로 분리하였다. 로봇 클라이언트는 카메라 프레임과 관절 상태를 수집하고, GPU 서버는 SmolVLA 정책을 실행하여 행동 청크를 생성한다.

---

클라이언트는 상단·손목 카메라 영상, 현재 관절 상태 및 작업 지시문을 서버에 전달한다. 서버는 입력을 전처리한 뒤 정책 추론을 수행하고, 팔로워 암의 다음 관절 목표값으로 구성된 행동 청크를 반환한다.

### 3.11.2. 관측 전송과 행동 청크 반환

서버에서 반환된 행동은 한 번에 모두 실행하지 않고 클라이언트의 행동 대기열에 저장한다. 클라이언트는 대기열에 남아 있는 행동을 제어 주기에 맞추어 하나씩 적용한다.

행동 대기열을 사용하는 이유는 정책 추론 시간과 로봇 제어 주기를 분리하기 위해서이다. 서버가 다음 관측에 대한 행동을 계산하는 동안에도 클라이언트는 기존 청크의 남은 행동을 실행할 수 있다. 이 구조를 통해 네트워크 지연이나 서버 추론 시간으로 인해 로봇이 매 순간 정지하는 현상을 줄였다.

### 3.11.3. 행동 대기열과 제어 주기

최종 실행에서는 정책이 50개의 행동을 생성하고, 클라이언트는 그중 설정된 개수의 행동을 실행한 뒤 새로운 관측을 요청한다. 내부 시험에서는 `ACTIONS_PER_CHUNK=30`과 `CHUNK_SIZE_THRESHOLD=0.6`을 사용하였다.

30개의 행동을 사용하는 경우 대기열의 잔여량이 전체 청크의 60% 이하로 내려가면 새로운 관측과 행동 청크를 요청한다. 이 방식은 기존 행동의 연속성을 유지하면서도 새로운 영상 관측을 일정 주기로 반영할 수 있도록 한다.

청크 결합 방식은 `latest_only`로 설정하여, 새로운 행동 청크가 도착하면 중복된 이전 예측보다 최신 관측에 기반한 행동을 우선적으로 사용하였다. 다만 청크 길이, 서버 응답 시간 및 네트워크 지연에 따라 실제 갱신 주기는 달라질 수 있으므로, 설정값을 명목상의 제어 주기와 동일하게 해석하지 않았다.

### 3.11.4. 관절 이동량 제한과 추종 감시

정책이 생성한 관절 목표값은 그대로 로봇에 전달하지 않고 관절별 이동량 제한을 적용하였다. 한 번의 제어 주기에서 지나치게 큰 관절 변화가 발생하면 로봇이 급격히 움직이거나 블록과 충돌할 수 있으므로, 현재 관절 상태와 목표 관절값 사이의 차이를 확인하여 명령을 제한하였다.

또한 목표 관절값과 실제 팔로워 암의 현재 관절값 사이의 추종 오차를 감시하였다. 추종 오차가 일정 시간 이상 지속되거나 여러 관절에서 동시에 커지는 경우에는 로봇을 정지시키도록 구성하였다.

---

그리퍼는 블록을 잡은 상태에서 물리적으로 단힘이 제한될 수 있다. 따라서 블록을 파지한 상태에서 나타나는 그리퍼의 정상적인 위치 차이와 팔 관절의 비정상적인 추종 실패를 구분해야 한다. 이를 위해 그리퍼와 팔 관절의 감시 조건을 동일하게 적용하지 않고, 파지 상황에서 발생할 수 있는 정상 오차를 고려하여 정지 조건을 조정하였다.

### 3.11.5. 입력 영상·행동 진단

실행 중 문제가 발생하면 모델 학습 부족과 실행 경로 오류를 구분하기 위해 다음 항목을 점검하였다.

- 클라이언트에서 수집된 원본 영상
- 서버가 수신한 영상
- 모델 입력 직전의 전처리 영상
- 상단·손목 카메라의 순서
- 영상의 자료형과 픽셀값 범위
- 패딩 이후 블록의 위치
- 서버가 반환한 행동 체크
- 실제 팔로워 암의 관절 추종 상태

특히 학습 시 사용한 영상과 실행 시 모델에 전달되는 영상의 크기, 정규화 범위 및 카메라 순서가 일치하는지 확인하였다. 또한 예측 행동의 범위와 관절 이동량을 기록하여, 모델이 비정상적인 명령을 생성했는지 또는 클라이언트가 행동을 잘못 적용했는지 구분하였다.

데이터 검토 도구는 에피소드별 상단·손목 영상을 재생하고 제외할 에피소드의 인덱스를 관리하는 데 사용하였다. 런타임 진단 도구는 클라이언트 영상, 서버 수신 영상 및 모델 입력 영상을 비교하는 데 사용하였다.

이와 같은 비동기 실행 및 진단 구조를 통해 GPU 서버에서 정책 추론을 수행하면서도 Jetson 기반 로봇 클라이언트에서 관절 제어와 안전 감시를 유지하였다. 최종적으로 시스템은 데이터 수집 단계의 자동 접근 모듈과 최종 실행 단계의 SmoIVLA 정책을 분리하면서, 시연·학습·추론·실물 제어가 연결된 통합 구조로 구성되었다.

---

## 4. 연구 결과 분석 및 평가

### 4.1. 평가 범위와 결과 해석

본 장에서는 최종 시스템의 구현 결과와 실물 로봇 평가 결과를 분석한다. 평가는 크게 기본 멀티태스크 모델에 대한 내부 평가와 HIL 데이터를 추가한 후속 평가로 구분하였다. 기본 멀티태스크 모델은 정리 353개와 적층 222개로 구성된 총 575개 에피소드 데이터셋을 이용하여 학습하였으며, 후속 평가에서는 기존 데이터에 HIL 교정 시연을 추가하여 총 704개 에피소드로 확장한 데이터셋을 사용하였다.

기본 멀티태스크 모델은 SmolVLA 기반으로 300,000단계 전체 미세조정을 수행하였다. 이 모델은 최종 시스템의 기본 동작과 데이터 분포에 따른 문제를 확인하기 위한 기준 모델로 사용하였다. 이후 HIL 데이터는 기본 모델이 실물 실행 중 발생시킨 오류 상태와 작업자의 복구 동작을 추가로 학습하기 위해 사용하였다.

평가 지표는 한 회차에서 5개 블록을 모두 처리했는지 여부, 정리 작업에서 목표 영역 내에 블록을 배치했는지 여부, 적층 작업에서 중앙 상공 접근과 수직 하강이 수행되었는지 여부 및 전체 작업 소요 시간으로 구성하였다. 블록 단위 파지 성공률과 5개 블록 전체 완료율은 서로 다른 지표이므로 구분하여 기록하였다.

기본 575개 데이터셋 모델의 내부 평가에서는 정리 작업의 5블록 전체 완료율이 80%로 나타났으며, 평균 작업 시간은 약 70초였다. 이후 HIL 교정 데이터를 추가한 후속 내부 평가에서는 작업 성공률이 87%까지 개선되었다. 두 결과는 동일한 모델과 동일한 데이터셋에서 얻은 값이 아니라, 기본 멀티태스크 모델과 HIL 추가 학습 모델의 서로 다른 평가 단계에서 얻은 결과이다.

공식 과제의 정리 작업 제한 시간은 180초이며, 적층 작업의 제한 시간은 300초이다. 또한 최종 배치 이후 일정 시간 동안 블록이 안정적으로 유지되어야 한다. 본 연구의 내부 평가는 개발 과정에서의 모델과 제어 구조를 비교하기 위한 목적이므로, 공식 평가와 동일한 조건의 반복 통계와 구분하여 해석하였다.

### 4.2. 최종 모델의 작업 수행 결과

#### 4.2.1. 정리 작업 결과

기본 575개 데이터셋으로 학습한 모델은 정리 지시문을 입력했을 때 블록을 순서대로 선택하고, 각 블록을 목표 영역으로 이동시키는 행동을 생성하였다. 특히 파지 이전의 접근과 파지 후 들어 올리기 동작은 비교적 안정적으로 수행되었으며, 모델이 5개 블록을 연속적으로 처리하는 기본 구조를 학습했음을 확인하였다. 정리 작업의 초기 상태, 블록 이송 중 상태 및 최종 배치 상태는 그림 4-1과 같다.

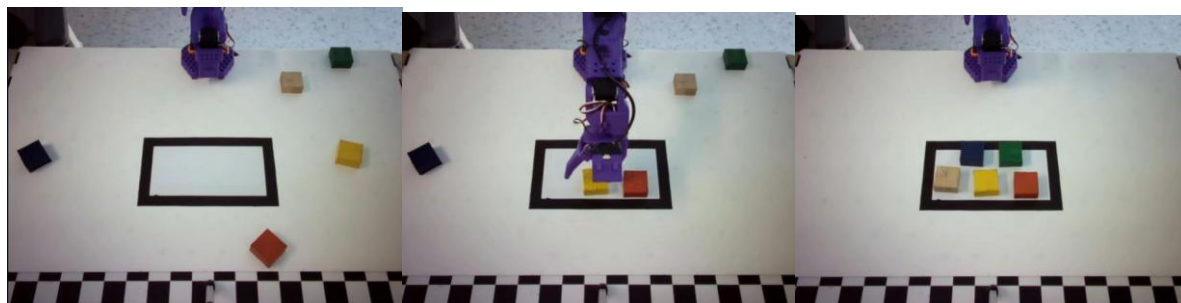


그림 4-1 정리 수행 스냅샷 (시작 / 진행 중 / 완료)

기본 모델의 내부 평가에서는 5개 블록 전체 완료율이 80%로 측정되었고, 평균 소요 시간은 약 70초였다. 이는 정리 작업 제한 시간인 180초보다 짧은 시간으로, 접근·파지·이송·배치가 연속적으로 수행되었음을 의미한다.

다만 모든 회차에서 블록이 서로 겹치지 않게 배치된 것은 아니었다. 일부 회차에서는 목표 영역 안에 블록을 배치했지만 블록 사이의 간격이 충분하지 않거나, wood 블록과 yellow 블록이 인접 위치에 겹쳐 놓이는 현상이 나타났다. 따라서 5개 블록을 목표 영역에 배치하는 전체 작업 성공과, 각 블록을 지정 위치에 정확히 분리하여 배치하는 정밀도는 별도의 평가 항목으로 구분할 필요가 있다.

#### 4.2.2. 적층 작업 결과

적층 작업에서는 정리 작업과 달리 목표 중앙 상공으로 이동한 뒤 수직 방향으로 하강하여 블록을 이전 블록 위에 놓는 동작이 요구된다. 동일한 모델에 적층 지시문을 입력했을 때, 모델은 정리 작업과 다른 중앙 접근 및 수직 하강 행동을 생성하였다.

적층 과정에서는 블록을 집은 후 목표 중앙 상공으로 이동하고, 기존에 놓인 블록의 높이에 맞춰 하강하는 행동이 관찰되었다. 특히 파지 후 중앙 상공으로 이동하는 경로와 수직 하강 동작은 적층 데이터의 시연 패턴과 유사하게 나타났다.

적층 작업의 초기 상태, 상공 접근 및 적층 진행 상태, 최종 적층 상태는 그림 4-2와 같다.

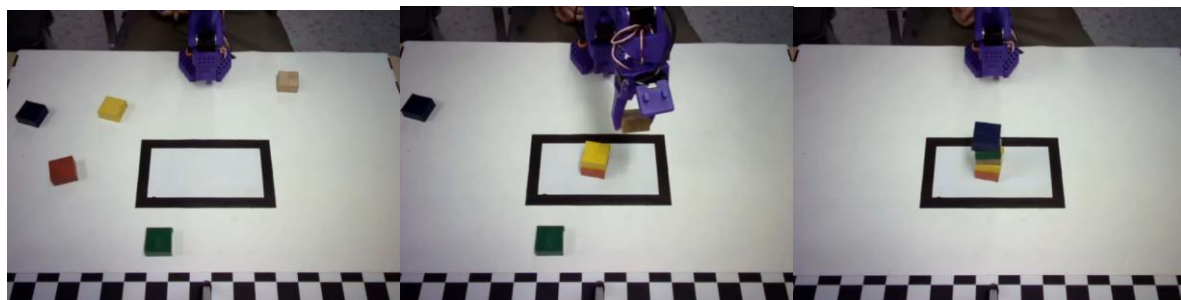


그림 4-2 적층 수행 스냅샷 (시작 / 진행 중 / 완료)

그러나 적층은 블록이 하나씩 누적되면서 작은 위치 오차가 다음 단계로 전달된다. 첫 번째 블록이 중앙에서 조금 벗어나면 두 번째 블록 이후의 기준 위치도 함께 변하고, 누적 높이가 증가할수록 손목 자세와 하강 위치의 오차가 커질 수 있다. 따라서 적층 작업에서는 단순히 블록을 목표 영역에 놓았는지뿐 아니라, 각 층의 수평 오차와 최종 타워의 안정성을 함께 측정해야 한다.

본 연구에서는 적층 작업의 중앙 접근, 상공 정렬 및 수직 하강 동작을 확인하였으나, 기본 모델에 대해 정리 작업과 동일한 방식의 5블록 안정 완수율을 충분한 반복 횟수로 산출하지는 못하였다. 따라서 적층 결과는 정량 성공률보다 작업별 행동 차이와 최종 시스템의 동작 가능성을 중심으로 분석하였다.

#### 4.2.3. HIL 추가 학습 이후의 변화

기본 모델은 파지와 이송 동작을 수행할 수 있었지만, 실물 실행 중 작은 위치 오차가 발생했을 때 이를 스스로 복구하는 능력에는 한계가 있었다. HIL 데이터는 정책이 실패하거나 불안정한 상태에 진입했을 때 작업자가 직접 개입하여 복구한 행동을 기록한 것이다.

기존 575개 에피소드에 정리 HIL 33개와 적층 HIL 10개를 추가로 수집하였다. 이후 정리 HIL과 적층 HIL을 각각 3회 반복 병합하여, 학습용 데이터셋을 총 704개 에피소드로 구성하였다. 따라서 독립적으로 수집한 추가 HIL 에피소드는 43개이며, 반복 병합 후 학습 데이터에 포함된 HIL 항목은 129개이다. HIL 추가 학습 이후에는 파지 위치가 약간 어긋난 상황에서 그리퍼와 팔의 자세를 조정하거나, 배치 과정에서 블록 위치를 보정하는 행동이 일부 개선되었다.

후속 내부 평가에서 작업 성공률은 87%로 기록되었다. 이는 기본 575개 데이터셋 모델의 80%와 비교하여 개선된 결과이다. 다만 두 모델의 학습 단계, 데이터 구성 및 평가 시점이 완전히 동일하지 않으므로, 이 차이를 HIL의 단일 효과로 단정하지 않고 후속 학습과 실행 설정을 포함한 전체 개선 결과로 해석하였다.

| 구분              | 기본 멀티태스크 모델    | HIL 추가 학습 모델     |
|-----------------|----------------|------------------|
| 기본 데이터          | 575개 에피소드      | 기존 575개          |
| 독립 HIL 데이터      | 없음             | 정리 33개 + 적층 10개  |
| HIL 반복 병합       | 없음             | 각 데이터셋 3회 반복     |
| 학습 데이터 내 HIL 항목 | 없음             | 129개             |
| 통합 데이터          | 575개           | 704개             |
| 학습 목적           | 기본 정리·적층 행동 학습 | 실패 상태 및 복구 행동 보강 |

### 4.3. 미세조정 방식·학습 손실·체크포인트별 결과 분석

#### 4.3.1. 미세조정 방식별 실물 관찰

개발 과정에서는 LoRA와 전체 미세조정을 적용한 모델을 실물 로봇에서 실행하여 파지 정밀도와 배치 동작을 관찰하였다. 정리 330개 에피소드를 사용한 주요 실험 설정은 다음과 같다.

| 항목        | LoRA               | 전체 미세조정            |
|-----------|--------------------|--------------------|
| 정리 에피소드 수 | 330개               | 330개               |
| 배치 크기     | 32                 | 16                 |
| 학습 단계     | 150,000            | 150,000            |
| 학습률       | $1 \times 10^{-4}$ | $2 \times 10^{-5}$ |
| 주요 관찰     | 파지 정밀도 부족          | 지정 위치 배치 동작 관찰     |

LoRA 모델에서는 블록에 접근한 이후의 정밀 정렬과 파지가 충분히 안정적이지 않았으며, 전체 미세조정 모델에서는 지정 위치로 블록을 이동하여 배치하는 동작을 관찰하였

---

다. 이러한 결과는 최종 기본 멀티태스크 모델의 학습 방식을 전체 미세조정으로 선택하는 데 참고하였다.

다만 두 실험은 동일한 에피소드 수를 사용하였지만 배치 크기와 학습률이 달랐다. 동일한 학습 단계에서도 배치 크기가 다르면 학습 중 처리하는 표본 수가 달라지므로, 두 모델이 동일한 학습량으로 비교되었다고 볼 수 없다. 또한 개발 과정에서 전처리와 관절 명령 처리 등의 수정이 이루어졌으므로, 관찰된 차이를 미세조정 방식 하나의 효과로 귀속하지 않았다.

Expert-only는 학습 범위를 검토한 방식에 포함되지만, 본 절에서는 LoRA 및 전체 미세조정과 동일한 조건으로 정리된 정량 비교 결과를 제시하지 않는다. 따라서 세 방식의 일반적인 성능 순위를 도출하기보다, 개발 기록에서 확인된 동작과 최종 선택의 근거를 중심으로 결과를 해석하였다.

#### 4.3.2. 성능 차이의 가능한 원인과 해석 범위

전체 미세조정에서는 시각·언어 표현과 행동 생성부를 함께 갱신할 수 있으므로, 본 환경의 관측 차이와 관절 행동을 공동으로 조정했을 가능성이 있다. 반면 LoRA는 적용 모듈과 rank에 따라 갱신 범위가 달라지고, Expert-only는 고정된 백본 표현을 이용하여 행동을 학습한다. 이러한 차이는 정밀 파지와 작업 조건화에서 나타난 성능 차이를 설명할 수 있는 원인 후보이다.

그러나 본 연구에서는 백본 특징의 분리 정도나 모듈별 기여도를 직접 측정하지 않았다. 또한 적용 모듈과 rank만 변경한 LoRA 비교, 백본 동결 여부만 변경한 Expert-only 비교가 동일 조건으로 확보되지 않았다. 따라서 갱신 범위의 차이는 구조적 설명이자 후속 검증 가설로 제시하였다.

데이터 수집 방식 역시 실험 해석에 함께 고려해야 한다. 자동 접근과 텔레오퍼레이션을 결합한 시연은 접근 궤적의 불필요한 변동을 줄이기 위한 것이지만, 모든 과거 모델에 최신 배치 시나리오와 동일한 수집 절차가 적용된 것은 아니다. 따라서 후기에 정리된 밀집 배치 전략이나 전처리 개선을 초기 모델의 조건으로 소급하여 해석하지 않았다.

전체 미세조정의 선택은 과제 수행에 필요한 표현과 행동을 함께 조정할 수 있다는 특성 및 실물 실행에서 확보한 동작을 근거로 하였다. 이는 동일 조건의 대조 실험으로 전체 미세조정의 우월성을 입증한 결과와는 구분된다.

### 4.3.3. 학습 손실과 실물 성능의 관계

SmolVLA의 학습 손실은 관측 조건에서 시연 행동을 생성하도록 학습하는 Flow Matching 목적함수에 따른 오차이다. 이 값은 실제 로봇이 블록을 파지하거나 전체 작업을 완료한 비율을 직접 나타내지 않는다.

정리 단일 작업의 330개 에피소드 전체 미세조정 실행에서는 약 0.037913의 손실이 기록되었고, 지정 위치 배치 동작이 관찰되었다. 별도의 멀티태스크 실행에서는 약 0.030831의 손실이 기록되었으나 정리 작업의 중앙 배치 편향이 남아 있었다. 최종 575개 에피소드 학습 실행의 종료 시점 손실은 약 0.0480이었다. 해당 학습 실행의 단계별 Flow Matching 손실 변화는 그림 4-3과 같다.

그림 4-3 기본 멀티태스크 모델의 학습 손실 변화



이 수치들은 데이터 구성과 작업 비율 및 학습 설정이 서로 다른 실행에서 얻은 값이다. 따라서 손실의 크기만으로 모델의 실물 성능을 순위화하지 않았다. 특히 전체 손실에는 접근과 이송 등 여러 구간이 함께 반영되므로, 특정 블록의 파지 실패나 지정 위치 간 겹침과 같은 국소적인 문제가 하나의 평균값에서 충분히 드러나지 않을 수 있다.

이에 학습 로그는 최적화 진행 상태를 확인하는 자료로 사용하고, 실제 작업 성능은 별도의 실물 실행으로 확인하였다. 입력 영상의 정규화와 크기 변환, 카메라 순서 및 관찰

---

명령 처리를 점검한 뒤 파지와 배치 동작을 관찰하여, 학습 문제와 실행 경로의 오류를 구분하고자 하였다.

#### 4.3.4. 체크포인트별 행동 관찰

575개 에피소드 학습 실행에서는 총 300,000단계 동안 저장된 체크포인트를 실물 로봇에 적용하여 정리와 적층 행동의 변화를 관찰하였다. 체크포인트 선택에서는 학습 손실 뿐 아니라 대상 선택, 파지 후 들어 올리기, 지정 위치 배치 및 적층 시 수직 하강 동작을 함께 확인하였다.

180,000단계 체크포인트에서는 정리 작업의 목표 위치 구분이 일부 개선되는 양상이 관찰되었다. 그러나 이후 학습 단계가 증가한 체크포인트가 모든 동작에서 일관되게 더 좋은 결과를 보인 것은 아니었다. 정리 작업의 중앙 편향과 적층 작업의 배치 행동은 체크포인트에 따라 달라졌으며, 파지와 이송이 유지되더라도 세부 배치 위치는 불안정할 수 있었다.

두 작업의 차이는 파지 이후의 이동 과정에서 비교적 뚜렷하게 나타났다. 정리 지시에서는 목표 위치로 이송하는 행동이, 적층 지시에서는 중앙 상공으로 이동한 후 수직으로 하강하는 행동이 관찰되었다. 다만 작업별 경로가 구분되는 것과 정리 위치의 분리 또는 적층 정렬이 정확하게 수행되는 것은 별개의 평가 항목이다.

이러한 관찰은 체크포인트별 행동 차이를 확인한 정성 평가이며, 동일한 초기 배치와 시행 수를 적용한 통계적 성공률 비교는 아니다. 향후에는 동일한 배치 시나리오에서 각 체크포인트를 반복 실행하고, 블록별 파지 성공률, 전체 완료율, 배치 오차 및 수행 시간을 함께 비교할 필요가 있다.

따라서 본 연구에서는 가장 늦게 저장된 체크포인트나 가장 낮은 손실을 기록한 체크포인트를 자동으로 최종 선택하지 않고, 두 작업의 실물 동작과 실행 안정성을 함께 고려하였다.

.

#### 4.4. 실행 설정과 자원 사용

최종 실행은 RTX 3090 GPU 서버에서 정책 추론을 수행하고 Jetson Orin Nano에서 로봇 제어를 수행하는 비동기 구조로 구성하였다. Jetson 클라이언트는 카메라 영상과 관절 상태를 서버로 전송하고, GPU 서버는 SmoIVLA의 행동 청크를 계산하여 반환한다.

최종 모델은 한 번의 추론에서 50개의 행동을 생성하도록 설정하였다. 클라이언트는 행동 대기열에 저장된 행동을 제어 주기에 맞추어 실행하고, 대기열의 잔량이 임계값 이하로 떨어지면 새로운 관측을 서버에 요청하였다.

| 항목           | 설정                         |
|--------------|----------------------------|
| 정책 행동 청크     | 50단계                       |
| 실행 행동 수      | 30단계                       |
| 청크 잔량 임계 비율  | 0.6                        |
| 제어 주파수       | 30 Hz                      |
| 청크 결합 방식     | latest_only                |
| 상대 목표 이동량 상한 | 1.75                       |
| 정책 서버        | RTX 3090                   |
| 로봇 클라이언트     | Jetson Orin Nano           |
| 카메라          | Top·Wrist, 640×480, 30 fps |

행동 청크를 50단계로 설정하면 한 번의 서버 응답으로 일정 시간 동안 연속 동작을 실행할 수 있다. 반면 청크가 너무 길면 새로운 영상 관측을 반영하는 시간이 늦어질 수 있으므로, 실제 실행에서는 30개의 행동을 소비한 뒤 대기열 상태를 확인하여 새로운 청크를 요청하였다.

정책 추론과 로봇 제어를 분리한 결과, Jetson에서 대규모 VLA 모델을 직접 실행하지 않고도 로봇의 카메라와 모터를 안정적으로 제어할 수 있었다. 서버 추론이 진행되는 동안 클라이언트는 이전에 받은 행동을 계속 실행하므로, 서버 처리 시간이나 네트워크 지연으로 인해 매 순간 로봇이 정지하는 현상을 줄일 수 있었다.

실행 과정에서는 목표 관절값과 실제 관절 상태 사이의 차이를 감시하였다. 팔 관절에서 큰 추종 오차가 일정 시간 이상 지속되면 로봇을 정지시키고, 그리퍼에서는 블록을 잡았을 때 발생하는 물리적인 단힘 제한을 정상 오차로 고려하였다.

---

## 4.5. 실패 유형과 개선 방향

### 4.5.1. 정밀 파지 실패

첫 번째 주요 실패 유형은 블록에 접근했지만 그리퍼가 블록 중심에서 벗어나거나 회전 방향과 맞지 않아 안정적으로 파지하지 못하는 경우이다. 블록의 중심 위치가 정확하게 검출되더라도, 그리퍼의 접근 방향과 손목 회전이 블록의 실제 방향과 일치하지 않으면 블록을 밀거나 한쪽 면만 잡게 된다.

이 문제는 YOLO 검출 정확도만으로 해결하기 어렵다. 검출된 중심점, 블록 회전 방향, 그리퍼의 폭, 접근 높이 및 로봇 관절 오차가 함께 작용하기 때문이다. 최종 시스템에서는 이러한 회전별 파지 자세를 모두 규칙으로 계산하는 대신, 실제 작업자가 수행한 파지 동작을 시연 데이터에 포함하여 정책이 영상과 관절 행동의 대응을 학습하도록 하였다.

### 4.5.2. 중앙 배치 편향

두 번째 주요 실패 유형은 정리 작업에서도 블록을 목표 중앙으로 이동시키는 현상이다. 적층 작업은 다섯 개 블록을 하나의 중앙 위치에 반복적으로 배치하므로, 중앙 접근과 중앙 하강 행동이 데이터에 많이 포함된다.

정리와 적층의 에피소드 수만 비교하면 적층 데이터가 전체의 일부에 해당하지만, 블록 단위 배치 이벤트로 비교하면 중앙 배치가 훨씬 많이 나타난다. 이러한 행동 분포와 초기 지시문의 유사성이 결합되면서 정리 작업의 지정 위치 구분이 약해졌을 가능성이 있다.

이를 개선하기 위해 정리 지시문에는 `each block separately`와 `designated target position`을 포함하고, 적층 지시문에는 `hover over`와 `on top of the previous block`을 포함하였다. 또한 정리 전용 데이터의 위치 분포를 확인하고, 체크포인트별로 중앙 편향이 감소하는지 실물 실행을 통해 비교하였다.

### 4.5.3. 배치 과정의 겹침과 위치 오차

정리 작업에서 전체 블록이 목표 영역에 들어갔더라도 블록 사이의 간격이 충분하지 않거나 일부 블록이 겹치는 경우가 있었다. 이는 목표 영역에 들어갔는지를 판단하는 단순 성공 기준과, 다섯 블록을 서로 분리하여 배치하는 정밀 기준 사이에 차이가 있음을 보여준다.

---

이 문제를 줄이기 위해서는 각 목표 슬롯의 중심뿐 아니라 블록의 실제 크기와 그리퍼가 블록을 놓는 위치까지 고려해야 한다. 또한 정리 데이터에서 각 목표 위치에 대한 시연 횟수를 균등하게 유지하고, 블록이 이미 놓인 상태에서 다음 블록을 배치하는 중간 상태를 충분히 포함해야 한다.

#### 4.5.4. 적층 높이 누적 오차

적층 작업에서는 첫 번째 블록의 위치 오차가 이후 블록의 기준 위치에 영향을 준다. 블록이 중앙에서 조금 벗어나거나 수직으로 놓이지 않으면 다음 블록을 내려놓을 때 접촉면이 달라지고, 최종적으로 타워가 기울어질 수 있다.

적층 성능을 개선하기 위해서는 층별 높이와 블록의 기울기, 목표 중앙과의 수평 오차를 별도로 기록할 필요가 있다. 현재 데이터는 작업자의 수동 시연을 통해 적층 행동을 포함하고 있으나, 각 층의 정량적인 접촉 오차를 별도로 라벨링하지 않았다는 한계가 있다.

#### 4.5.5. HIL 데이터의 불완전한 구간

HIL 데이터는 실패 상태를 포함한다는 점에서 기존 성공 시연과 다르지만, 모든 HIL 에피소드가 동일한 길이와 구간을 포함하는 것은 아니다. 일부 에피소드는 파지 직후 종료되거나 복구 동작의 일부만 포함할 수 있다.

이 경우 모델은 실패 상태를 관찰하는 방법은 학습할 수 있지만, 해당 상태에서 블록을 들어 올리고 목표 영역까지 이동하는 후속 행동은 충분히 학습하지 못할 수 있다. 또한 실패·복구 구간이 과도하게 포함되면 모델이 정상적인 파지 후 행동보다 정지나 그리퍼 재조정 행동을 더 자주 생성할 가능성도 있다.

따라서 후속 데이터 수집에서는 실패 발생, 작업자의 개입, 파지 복구, 들어 올리기, 이송 및 배치 완료까지의 전체 구간을 하나의 교정 에피소드로 기록해야 한다. 또한 성공 시연과 복구 시연의 비율을 조정하여 기존의 정상적인 파지 및 이송 능력이 약화되지 않도록 해야 한다.

#### 4.6. 결과물 구성과 구현 범위

본 연구에서 구현한 결과물은 하드웨어, 데이터 수집, 정책 학습, 비동기 실행 및 진단 도구로 구분된다.

| 구분       | 구현 결과                          |
|----------|--------------------------------|
| 로봇 하드웨어  | SO-101 리더·팔로워 암                |
| 로봇 클라이언트 | Jetson Orin Nano 기반 장치 제어      |
| 영상 입력    | 상단·손목 카메라 2개                   |
| 데이터 수집   | YOLO 검출 및 RBF 자동 접근 기반 수집기     |
| 수동 시연    | 리더 암 기반 파지·이송·배치               |
| 데이터셋     | 정리 353개, 적층 222개, 기본 575개 에피소드 |
| 정책 모델    | SmolVLA 멀티태스크 정책               |
| 학습 방식    | 전체 미세조정                        |
| 추론 서버    | RTX 3090 GPU 기반 비동기 정책 서버      |
| 통신       | gRPC 기반 관측·행동 전송               |
| 제어 안전    | 관절 이동량 제한 및 추종 오차 감시           |
| 후속 학습    | HIL 교정 데이터 추가 및 704개 에피소드 구성   |

본 시스템의 중요한 특징은 시연 수집 단계와 최종 실행 단계를 분리했다는 점이다. 데이터 수집 단계에서는 YOLO와 RBF를 이용하여 작업자의 초기 접근 부담을 줄이고, 작업자가 정밀 파지와 배치를 수행하도록 하였다. 반면 최종 실행에서는 YOLO나 FSM을 별도로 호출하지 않고 SmolVLA가 영상과 자연어 지시를 이용하여 전체 작업을 수행하도록 구성하였다.

이를 통해 초기 규칙 기반 제어에서 발견된 환경 의존성과 회전별 파지 자세 계산의 어려움을 최종 정책에 직접 포함시키지 않으면서도, 해당 개발 과정에서 얻은 좌표 보정과 자동 접근 기술을 데이터 수집에 활용할 수 있었다.

#### 4.7. 과제 평가 항목에 따른 수행 결과

과제 평가 기준에 따른 본 연구의 수행 결과는 다음과 같다.

| 평가 항목      | 수행 결과                             |
|------------|-----------------------------------|
| 5개 블록 정리   | SmolVLA 기반 연속 정리 작업 구현            |
| 5개 블록 적층   | 중앙 상공 접근 및 수직 하강 기반 적층 동작 구현      |
| 단일 모델 사용   | 정리·적층에 동일한 SmolVLA 체크포인트 사용       |
| 자연어 작업 조건화 | 작업별 지시문으로 정리·적층 구분                |
| 시각 입력      | 상단·손목 카메라 영상 사용                   |
| 로봇 상태 입력   | 6개 관절 상태 사용                       |
| 자동 데이터 수집  | YOLO-좌표 변환·RBF 기반 상공 접근           |
| 수동 정밀 조작   | 리더 암을 이용한 파지·이송·배치                |
| 실물 실행      | Jetson 클라이언트와 RTX 3090 서버의 비동기 추론 |
| 안전 제어      | 관절 이동량 제한과 추종 오차 감시               |
| 기본 내부 평가   | 575개 모델 정리 전체 완료율 80%             |
| 후속 내부 평가   | HIL 추가 학습 후 작업 성공률 87%            |

기본 575개 데이터셋 모델은 하나의 정책으로 정리와 적층을 수행할 수 있는 가능성을 확인하였다. 정리 작업에서는 5개 블록을 순서대로 처리하고 목표 영역으로 이동하는 행동이 나타났으며, 적층 작업에서는 중앙 상공 접근과 수직 하강이라는 별도의 행동 패턴이 관찰되었다.

또한 최종 시스템은 단순한 정책 학습에 그치지 않고, 실제 로봇에서 동작하기 위한 데이터 수집기, GPU 서버, 로봇 클라이언트, 비동기 통신 및 안전 감시 기능을 함께 구현하였다. HIL 데이터를 추가한 후속 단계에서는 실물 실행 중 발생한 오류 상태와 복구 동작을 학습에 반영하여 작업 성공률을 87%까지 개선하였다.

다만 정리 작업의 지정 위치 분리도, 적층 작업의 층별 높이 오차 및 실패 상태에서의 안정적인 복구는 추가 개선이 필요한 부분이다. 따라서 본 연구의 결과는 하나의

---

SmolVLA 정책이 시각·언어 조건에 따라 서로 다른 로봇 조작 작업을 수행할 수 있음을 확인한 동시에, 실제 로봇의 정밀 조작을 위해서는 데이터 분포 균형, 복구 시연의 완전성 및 평가 지표의 세분화가 필요함을 보여준다.

## **4.8. Correction-only HIL 기반 복구 데이터 수집 및 학습**

### **4.8.1. 풀 에피소드 HIL의 한계와 수집 방식 변경**

기본 시연 데이터셋은 정상적인 파지·이송·배치 행동을 중심으로 구성되어 있다. 따라서 실제 정책 실행 중 발생하는 헛파지, 부분 파지, 블록 밀림 및 잘못된 접근과 같은 상태는 학습 데이터에 충분히 포함되지 않는다. 정책이 이러한 상태에 진입하면 기존 시연과 다른 관측을 입력받게 되며, 이후에도 정상 작업에 해당하는 행동을 계속 생성하여 오류를 확대할 수 있다. 이는 모방학습에서 학습 시연의 상태 분포와 정책 실행 중의 상태 분포가 달라지는 문제와 관련된다[4].

초기 후속 학습에서는 기존 성공 시연 575개에 풀 에피소드 HIL 데이터 129개를 추가하여 총 704개 에피소드로 학습하였다. 풀 에피소드 HIL은 자율 실행, 실패, 사람의 개입, 복구 및 이후 작업 수행을 하나의 에피소드로 기록하는 방식이다. 이러한 데이터는 정책이 실제로 방문한 실패 상태와 그 전후 맥락을 보존할 수 있지만, 별도의 구간 선택이나 가중 없이 일반적인 모방학습에 사용하면 실패를 유발한 자율 행동도 모방 대상으로 포함될 수 있다.

초기 풀 에피소드 HIL 학습에서는 복구 행동이 일부 관찰되었으나, 파지 이후 들어 올리기와 같은 기본 동작이 불안정해지는 현상도 나타났다. 이에 실패 상태는 유지하면서 해당 상태에서 수행해야 할 복구·교정 행동을 선택적으로 제공하도록 수집 방식을 변경하였다.

본 연구에서 사용하는 Correction-only HIL은 개입 이전의 자율 실행을 모방 시연에서 제외하고, 개입 이후의 복구·교정 구간을 별도 에피소드로 기록하는 방식이다. 여기서 Correction-only는 복구 동작을 제외한다는 의미가 아니라, 학습 대상으로 채택한 사람 감독하의 복구와 교정 구간을 구분하여 저장한다는 의미이다. 이는 HG-DAGger의 사람 주도 개입 원리와 RaC의 복구·교정 개념을 본 과제의 수집 환경에 적용한 구성이다[11], [12].

---

#### 4.8.2. 사람 개입과 제어권 전환

Correction-only HIL의 실행 과정은 자율 실행, 개입 일시정지, 복구 모드 선택, 교정 기록, 결과 검토 및 자율 실행 재개의 순서로 구성하였다.

AUTONOMOUS 상태에서는 SmoIVLA가 로봇의 행동을 생성하고, 작업자는 대상 블록의 위치, 그리퍼의 정렬 상태 및 작업 진행을 관찰한다. 실패가 예상되거나 이미 발생한 경우 작업자가 SPACE 키를 입력하여 자율 정책의 행동 전송을 중단하고 INTERVENTION\_PAUSED 상태로 전환한다.

개입 이후에는 팔로워 암의 현재 자세를 유지한 상태에서 리더 암을 팔로워 암의 관절 위치에 맞춘다. 이 동기화 과정은 제어권 전환을 준비하는 절차이므로 모방 시연에 포함하지 않는다. 동기화가 완료되면 작업자는 현재 블록의 위치와 관측 가능성을 고려하여 Local Correction 또는 Observe Correction을 선택한다.

두 복구 방식은 실패 상태에서 필요한 재접근 범위에 따라 구분된다. Local Correction은 현재 자세 주변에서 대상 블록을 다시 정렬할 수 있는 경우에 사용하고, Observe Correction은 작업 공간을 다시 관측하고 목표 블록을 재탐색해야 하는 경우에 사용한다.

#### 4.8.3. Local Correction과 Observe Correction

Local Correction은 블록의 위치가 크게 변하지 않았고, 손목 카메라에서 대상 블록을 확인할 수 있으며, 현재 자세 부근에서 재접근이 가능한 경우에 적용한다. 작업자는 L 키를 입력하여 Local Correction을 선택한다. 로봇은 필요한 상승 또는 후퇴를 통해 블록과의 간섭을 줄이고, 작업자는 리더 암을 이용하여 그리퍼를 다시 정렬한 뒤 파지와 들어 올리기 등의 교정 행동을 수행한다.

이때 실패 상태에서 자동으로 상승·후퇴하는 구간을 학습 대상으로 포함하는 경우에는 해당 이동이 시작되기 전부터 기록해야 한다. 반대로 Local Hover 도달 이후의 행동만 기록하면, 해당 데이터가 직접 제공하는 모방 대상은 재정렬과 재파지 이후의 행동으로 제한된다. 따라서 자동 복구 이동과 수동 교정의 기록 범위는 구분하여 관리해야 한다.

Observe Correction은 블록이 크게 밀렸거나 손목 카메라만으로 위치를 파악하기 어려운 경우에 적용한다. O 키를 입력하면 복구·교정 에피소드의 기록을 시작하고, 로봇은 Observe 자세로 이동하여 상단 카메라의 시야를 확보한다. 이후 YOLO로 목표 블록을 다시 검출하고, 자동 접근 모듈을 통해 블록 상공으로 이동한다. 작업자는 해당 위치에서 제어권을 받아 정밀 정렬, 파지 및 필요한 후속 작업을 수행한다.

---

Observe Correction에서는 실패 상태에서 Observe 자세로 이동하는 과정과 목표 블록으로 다시 접근하는 과정이 복구 행동에 해당한다. 반면 교정 기록이 끝난 뒤 다음 자율 실행을 준비하기 위한 이동은 별도의 준비 구간으로 구분한다. 두 복구 방식의 품질은 이동 경로의 종류가 아니라, 해당 실패 상태에서 적절하게 작업을 복구했는지를 기준으로 판단한다.

#### 4.8.4. 교정 종료와 에피소드 저장

작업자는 교정 구간을 완료하면 C 키를 입력하여 기록을 종료하고 로봇을 일시정지한다. 기록 종료 이후의 결과 확인 시간과 대기 프레임은 학습 데이터에 포함하지 않는다. 작업자는 교정 결과를 검토하여 정상적인 복구와 교정이 확인된 에피소드를 저장하고, 파지에 실패했거나 불필요한 움직임이 포함된 에피소드는 제외한다.

저장 또는 폐기 이후 R 키를 입력하면 로봇은 다음 자율 실행을 준비한다. 이 과정에서 기존 행동 대기열과 필요한 실행 상태를 초기화한 뒤 정책 추론을 재개한다. Q 키는 수집 프로그램을 종료하는 데 사용한다.

하나의 물리적 롤아웃에서 여러 번 개입한 경우에는 각 개입을 별도의 Correction-only HIL 에피소드로 저장하였다. 따라서 기본 데이터셋의 에피소드가 5개 블록 전체 작업을 나타내는 것과 달리, Correction-only HIL의 에피소드는 하나의 복구·교정 구간을 나타낸다. 두 데이터셋의 에피소드 수를 비교할 때에는 이러한 기록 단위의 차이를 고려해야 한다.

#### 4.8.5. 메타데이터와 작업 지시문

교정 에피소드의 발생 맥락을 추적하기 위해 원본 롤아웃과 개입 구간을 연결하는 메타 데이터를 구성하였다.

| 항목                 | 내용                       |
|--------------------|--------------------------|
| episode_index      | 교정 데이터셋 내 에피소드 번호        |
| parent_rollout_id  | 개입이 발생한 원본 자율 롤아웃 번호     |
| intervention_index | 원본 롤아웃 내 개입 순서           |
| recovery_type      | Local 또는 Observe 복구 방식   |
| task_phase         | 파지·들어 올리기·이송·배치 등의 작업 단계 |
| target_block       | 교정 대상 블록의 색상             |
| total_frames       | 저장된 교정 구간의 프레임 수         |
| timestamp          | 개입 및 기록 시점               |

Correction-only HIL에서도 정상 시연과 동일한 작업 지시문을 유지하였다. 정리 작업에는 각 블록을 지정 위치에 분리하여 배치하는 지시문을 사용하고, 적층 작업에는 이전 블록 위에 순차적으로 쌓는 지시문을 사용하였다.

별도의 복구 지시문을 추가하지 않은 이유는 자율 실행 중 모델이 카메라 영상과 로봇 상태를 바탕으로 정상 행동과 복구 행동을 구분하도록 하기 위해서이다. 학습 시에만 제공되는 복구 명령에 의존하지 않도록, 정상 시연과 교정 시연이 동일한 작업 조건을 공유하게 하였다.

#### 4.8.6. 정지 프레임 정제와 데이터셋 구성

교정이 완료된 뒤 작업자가 기록 종료 키를 누르기까지 지연이 발생하면, 에피소드 끝에 동일한 자세를 유지하는 정지 프레임이 저장될 수 있다. 특히 파지 직후의 정지 프레임이 반복되면, 모델이 들어 올리거나 이송보다 현재 자세를 유지하는 행동을 학습할 가능성이 있다.

에피소드 경계를 넘어가는 패딩 행동과 실제로 기록된 정지 행동은 구분해야 한다. 패딩 행동은 유효 마스크를 통해 직접적인 손실 기여를 제외할 수 있지만, 에피소드 내부의 정지 행동은 별도로 정제하지 않으면 실제 시연으로 사용된다. 이에 교정 완료 이후의 불필요한 정지 구간을 제거하였다.

---

초기 Correction-only HIL 원본 데이터는 220개 에피소드와 40,358개 프레임으로 구성되었다. 정제 후 통합 데이터셋은 eslab1234/multitask\_5blocks\_v3\_684ep\_trimmed\_merged로 관리하였다. 원본 교정 데이터 수와 통합 데이터 수는 서로 다른 집계이므로, 정제 과정에서 전체 에피소드 제외와 프레임 절삭을 구분하여 데이터 이력을 관리해야 한다.

#### 4.8.7. 미세조정과 관찰 결과

Correction-only HIL의 학습에는 기존 성공 시연과 품질 검토를 통과한 복구·교정 시연을 함께 사용하였다. 이 단계에서는 복구 행동을 새로운 모방 대상으로 제공하는 것이 목적이므로, 일반적인 Flow Matching 기반 모방학습을 적용하였다.

학습 범위로는 Vision Encoder를 고정하고 VLM과 Action Expert를 갱신하는 방식과, Vision Encoder를 포함한 전체 미세조정 방식을 검토하였다. 시각 인코더 고정은 해당 모듈의 가중치 변화를 제한하는 선택이지만, 다른 모듈은 계속 갱신되므로 기존 행동의 유지를 보장하지는 않는다. 전체 미세조정은 실패 상태에서 나타나는 블록 위치 변화와 그리퍼의 상대 자세를 관측 표현에도 반영할 수 있다는 점을 고려하였다.

낮은 학습률의 전체 미세조정 이후에는 파지 위치 재정렬, 그리퍼와 팔의 자세 조정 및 일부 배치 보정 행동이 관찰되었다. 다만 복구 행동의 출현과 전체 작업 성능의 향상은 구분하여 평가해야 한다. 기본 모델, 풀 에피소드 HIL 모델 및 Correction-only HIL 모델의 결과를 분리하고, 정상 조건의 완료율과 실패 조건의 복구 성공률을 함께 확인하는 것이 필요하다.

### 4.9. Reward-Weighted Flow Matching 기반 행동 품질 학습

#### 4.9.1. 도입 목적과 적용 범위

Correction-only HIL은 실패 상태에서 수행해야 할 복구·교정 행동을 제공하지만, 정상적인 자율 행동과 실패를 유발한 행동 사이의 상대적인 품질을 직접 비교하지는 않는다. 또한 일반적인 모방학습은 채택된 행동을 동일한 목표로 사용하므로, 성공한 행동들 사이의 안정성과 효율 차이를 명시적으로 반영하기 어렵다.

본 연구에서는 이러한 차이를 학습에 반영하기 위해 Reward-Weighted Flow Matching(RWFM)을 도입하였다. RWFM은 행동에 부여한 보상 또는 품질 점수에 따라 Flow Matching 손실의 기여도를 조절하는 방법이다[25]. 본 연구의 구성에서는 수집된 자

을 실행 데이터에 구간별 품질 정보를 부여하고, 이를 기존 시연 및 Correction-only HIL 데이터와 함께 추가 학습에 사용한다.

이 단계의 목적은 새로운 복구 행동을 전문가 시연 없이 처음부터 발견하는 것이 아니라, 확보된 정상 행동과 복구 행동의 학습 비중을 조절하는 것이다. Q함수나 시간차 학습으로 미래 누적 보상을 추정하는 방식과는 구분되며, 수집된 행동에 대한 오프라인 보상 가중 학습으로 구성하였다.

#### 4.9.2. 자율 롤아웃 수집과 데이터 구성

RWFM 단계에서는 HIL 추가 학습 정책의 전체 자율 롤아웃을 기록하였다. Correction-only HIL이 개입 이후의 복구·교정 구간을 선택적으로 저장하는 것과 달리, 이 단계에서는 정상 행동과 낮은 품질의 행동을 함께 평가하기 위해 자율 실행의 전체 맥락을 보존하였다.

수집 대상에는 정상적인 접근·파지·이송·배치뿐 아니라, 정책이 스스로 수행한 재정렬과 복구, 실패를 유발한 접근, 충돌 및 정체가 포함된다. 초기 수집은 어려운 배치 조건을 중심으로 약 100개 롤아웃에서 시작하였으며, 추가 수집은 50개 에피소드 단위로 구성하였다. 수집량뿐 아니라 실패 유형과 정상·복구 구간의 분포를 함께 확인하는 것을 기준으로 하였다.

학습 데이터는 다음 세 종류로 구성하였다.

| 데이터 종류              | 학습에서의 역할             |
|---------------------|----------------------|
| 기존 성공 시연            | 정상적인 파지·이송·배치 능력 유지  |
| Correction-only HIL | 실패 상태에서의 복구·교정 행동 제공 |

기본 시연, 교정 시연 및 자율 롤아웃은 에피소드 길이와 행동 분포가 다르므로, 혼합 비율은 에피소드 수와 프레임 수준의 학습 노출량을 구분하여 관리해야 한다.

#### 4.9.3. 행동 구간별 품질 라벨

전체 롤아웃에 하나의 보상을 부여하면 성공과 실패가 발생한 구간을 구분하기 어렵다. 예를 들어 총 60초의 실행 중 마지막 2초에 충돌이 발생한 경우, 전체 에피소드에 동일한 낮은 가중치를 적용하면 앞선 58초의 정상 행동까지 함께 낮게 반영된다.

이에 행동 구간을 정상 수행, 자기 교정, 성공적인 복구, 실패 유발 행동, 충돌·정체 및 작업 종료 이후의 행동으로 구분하였다. 구간 판정에는 카메라 영상, 로봇 관절 상태, 그리퍼 상태 및 작업 결과를 함께 사용하였다.

| 행동 구간                        | 품질 수준 | 학습 방향                     |
|------------------------------|-------|---------------------------|
| 기존 성공 전문가 시연                 | 기준값   | 정상 작업 수행 능력 유지            |
| 정상 자율 행동                     | 기준값   | 현재 정책의 기본 행동 유지           |
| self-correction 성공           | 높음    | 스스로 수행한 안정적인 보정 행동 강화     |
| 성공적인 recovery                | 높음    | 실패 상태에서 정상 상태로 복귀하는 행동 강화 |
| 실패 유발 행동                     | 낮음    | 실패 행동의 모방 기여도 감소          |
| 충돌 또는 정체                     | 매우 낮음 | 불안정한 행동의 영향 감소            |
| terminal failure 이후 의미 없는 행동 | 제외    | 손실 계산에서 제거                |

Local 방식과 Observe 방식은 복구 경로가 다르다는 이유만으로 품질의 우열을 부여하지 않았다. 해당 상태에서의 복구 성공 여부와 행동의 적절성을 기준으로 평가하도록 구성하였다.

#### 4.9.4. 보상 가중 Flow Matching 목적함수

Flow Matching의 학습에서는 관측 조건이 주어졌을 때, 생성 과정의 중간 행동 상태에서 목표 행동 분포로 이어지는 벡터장을 예측한다. 실제 로봇 시점을  $t$ , 생성 시간을  $\tau$ 로 구분하면 일반적인 손실은 다음과 같이 표현할 수 있다.

$$\mathcal{L}_{\text{FM}} = \mathbb{E}[\|v_{\theta}(x_{\tau}, \tau \mid c_t) - u_{\tau}\|_2^2]$$

여기서  $x_{\tau}$ 는 생성 과정의 중간 행동 상태,  $c_t$ 는 카메라 영상·작업 지시·로봇 상태에 따른 관측 조건,  $u_{\tau}$ 는 목표 벡터장이다.

행동 구간  $i$ 의 품질 점수를  $R_i$ 라 하면, 지수 가중치는 다음과 같이 정의할 수 있다.

$$w_i = \exp\left(\frac{R_i - R_{\max}}{\beta}\right), \beta > 0$$

$R_{\max}$ 는 정해진 비교 범위의 최대 품질 점수이며,  $\beta$ 는 점수 차이가 가중치에 미치는 영향을 조절하는 온도 계수이다. 구간별 Flow Matching 손실을  $\mathcal{L}_i$ 라 할 때, 가중치를 정규화한 손실은 다음과 같다.

$$\mathcal{L}_{\text{RWM}} = \frac{\sum_i w_i \mathcal{L}_i}{\sum_i w_i}$$

낮은 보상을 받은 행동은 상대적인 모방 기여도가 줄어든다. 다만 지수 가중치는 양수이므로, 낮은 보상이 해당 행동을 직접적으로 반대 방향으로 학습시키거나 완전히 제외하는 것을 의미하지는 않는다.

#### 4.9.5. 행동 청크 내부의 시점별 가중

SmolVLA는 여러 시점의 행동을 하나의 청크로 생성한다. 본 연구의 정책 청크 길이는 50이므로, 하나의 학습 표본에는 기준 시점에서 시작하는 50개 행동이 포함된다. 동일한 청크 안에서도 정상 접근과 실패 유발 행동이 함께 나타날 수 있다.

따라서 청크 전체에 하나의 품질 점수를 적용하는 구성과 함께, 행동 시점별 품질 정보를 사용하는 구성을 설계하였다. 기준 프레임이  $t$ 라면 행동 목표와 대응하는 품질 정보는 다음과 같이 정렬된다.

$$\begin{aligned} A_t &= (a_t, a_{t+1}, \dots, a_{t+49}) \\ R_t &= (r_t, r_{t+1}, \dots, r_{t+49}) \end{aligned}$$

각 행동 시점의 품질 점수는 가중치로 변환하여 해당 시점의 벡터장 예측 오차에 적용한다. 이는 RWM의 보상 가중 원리를 SmolVLA의 청크 내부 손실에 적용한 설계이며, 궤적 전체에 하나의 가중치를 적용하는 방식과 구분된다.

#### 4.9.6. 데이터 로더와 유효 구간 처리

데이터 로더는 기준 프레임에서 행동 청크를 구성할 때 동일한 시간 범위의 품질 정보도 함께 불러오도록 구성하였다. 행동과 보상 정보의 시간 정렬이 어긋나면 정상 행동에 낮은 가중치를 적용하거나 실패 행동에 높은 가중치를 적용할 수 있으므로, 두 정보는 동일한 프레임 인덱스를 기준으로 관리해야 한다.

SmolVLA의 손실은 평균 연산 이전에 행동 시점과 차원별로 계산되므로, 해당 단계에서 시점별 가중치를 적용할 수 있다[24]. 에피소드 경계를 넘어가는 패딩 행동에는 유효 마스크를 적용하고, 작업 종료 이후 학습에서 제외하기로 한 구간도 별도로 마스킹한다.

마스킹은 해당 행동 시점의 직접적인 손실 기여를 제외하는 처리이다. 입력 청크나 모델 내부 연산에서도 해당 정보가 완전히 제거된다는 의미는 아니므로, 손실 제외와 입력 데이터 제거를 구분하여 해석해야 한다. 최종 목적함수의 평균 범위와 정규화 방식은 실제 구현에서 사용한 연산과 일치하도록 관리해야 한다.

#### 4.9.7. 실패 유발 행동과 종료 이후 행동의 구분

실패를 유발한 행동과 실패 이후에 이어진 무의미한 행동은 구분하여 처리하였다. 예를 들어 헛파지 이후 빈 그리퍼로 목표 영역까지 이동하는 행동은 최초의 파지 실패와 구분되는 후속 행동이다.

본 구성에서는 실패를 유발한 접근·파지 행동에는 낮은 품질 가중치를 적용하고, 충돌·정체 구간의 모방 기여도도 낮추도록 설계하였다. 작업 종료 상태 이후의 무의미한 행동은 직접적인 손실 계산에서 제외하였다.

이때 복구 가능한 중간 실패를 작업 종료 상태로 일괄 처리해서는 안 된다. 재접근과 교정으로 작업을 계속할 수 있는 상태는 복구 학습의 대상이므로, 작업 종료 여부와 복구 가능성을 별도로 판정해야 한다.

#### 4.9.8. 학습 범위와 단계별 구성

전체 학습은 기본 작업 학습, 복구·교정 학습 및 행동 품질 학습의 세 단계로 구분하였다.

| 단계      | 학습 데이터                                     | 학습 목적                      |
|---------|--------------------------------------------|----------------------------|
| Stage 0 | 성공 전문가 시연                                  | 정상 상태에서의 기본 작업 수행          |
| Stage 1 | 성공 시연 + Correction-only HIL                | 실패 상태에서의 복구·교정 능력          |
| Stage 2 | 성공 시연 + Correction-only HIL + 보상 라벨 자율 롤아웃 | 행동 품질에 따른 Action Expert 정제 |

Stage 2에서는 앞선 학습으로 확보한 관측 표현을 유지하면서 행동 생성부를 조정하는 것을 기본 방향으로 설정하였다. 이에 Vision Encoder와 VLM을 고정하고 Action Expert를 중심으로 학습하도록 구성하였다.

---

초기 과제 적응에서 전체 미세조정을 선택한 것과 Stage 2에서 행동 생성부 중심 학습을 선택한 것은 서로 다른 목적에 따른 결정이다. 초기에는 관측 표현과 관절 행동의 대응을 함께 조정해야 했지만, 후속 단계에서는 이미 적응된 표현 위에서 행동의 학습 비중을 조정하는 데 초점을 두었다.

다만 새로운 자율 롤아웃에서 기존 학습에 포함되지 않은 실패 상태가 나타난다면 행동 생성부만의 갱신으로 충분하지 않을 수 있다. 이 경우 추가 교정 시연과 관측 표현의 재적응이 필요하다.

#### 4.9.9. 비교 평가와 결과 해석

RWFM의 효과를 구분하려면 기본 모델, Correction-only HIL 모델 및 RWFM 추가 학습 모델을 각각 평가해야 한다. 또한 에피소드 전체 가중과 행동 시점별 가중을 비교하여, 세부 품질 정보의 사용이 성능에 미치는 영향을 확인할 필요가 있다.

정리 작업의 평가 항목은 블록별 파지 성공률, 5개 블록 전체 완료율, 지정 위치 오차, 블록 간 겹침 여부 및 수행 시간으로 구성한다. 적층 작업에서는 층별 수평 오차, 높이 오차, 타워의 기울기, 안정 유지 여부 및 수행 시간을 확인한다. 실패 조건에서는 복구 성공률과 작업 재개 여부를 추가로 측정한다.

비교에는 동일한 초기 배치와 평가 횟수, 카메라 및 제어 설정을 적용해야 한다. 정상 조건의 수행 능력과 실패 상태의 대응 능력을 분리하여, 복구 행동을 강화하는 과정에서 기존 파지·이송·배치 능력이 약화되는지도 함께 확인해야 한다.

본 보고서에서는 RWFM의 구성과 학습 의도를 설명하되, 비교 집계가 제시되지 않은 항목을 정량적인 성능 향상으로 해석하지 않는다. 기본 모델과 HIL 모델에서 기록된 결과 역시 RWFM의 독립적인 효과를 나타내는 수치와 구분한다.

## 5. 결론 및 향후 연구 방향

### 5.1. 결론

본 연구에서는 LeRobot SO-101 로봇 팔과 상단·손목 카메라를 이용하여, 무작위로 배치된 5개 블록을 순서대로 정리하거나 수직으로 적층하는 지능형 로봇 조작 시스템을 구현하였다. 시스템은 시연 데이터 수집, 정책 학습, GPU 서버 기반 비동기 추론 및 실물 로봇 제어의 전체 과정을 하나의 파이프라인으로 연결하였다.

---

초기에는 ACT, OpenCV 및 FSM을 결합하여 블록 검출과 로봇 제어를 분리하는 방식을 시도하였다. 그러나 장시간의 전체 작업에서 대상 선택과 파지 오차가 누적되었고, 조명·그림자·배경 변화에 따라 고정된 색상 임계값의 성능이 달라지는 문제가 확인되었다. 또한 YOLO와 좌표 변환, IK를 이용한 전체 자동 제어 경로에서는 블록의 위치와 중심을 검출하는 과정은 비교적 안정적으로 수행되었지만, 블록의 회전 방향에 맞춰 그리퍼 접근 자세와 손목 회전을 계산하는 과정에서 반복적인 파지 실패가 발생하였다.

이러한 문제를 해결하기 위해 최종 데이터 수집기에서는 YOLO 검출과 실측 좌표 변환을 이용하여 블록 상공까지 자동으로 접근하고, 상공 이후의 미세 정렬·파지·이송·배치는 작업자가 리더 암으로 수행하도록 구성하였다. 초기 IK 기반 접근 자세 계산은 실제 SO-101의 프레임 휨, 백래시 및 모터 기어 유격을 충분히 반영하지 못했으므로, 최종 수집기에서는 실측 작업판 좌표와 관절값 대응점을 이용한 RBF 보간 방식을 적용하였다.

최종 수집 과정은 녹화를 시작한 상태에서 YOLO 검출, 좌표 변환, RBF 기반 자동 접근, 작업자 텔레오퍼레이션, 배치 및 Observe 자세 복귀를 모두 수행하는 방식으로 구성하였다. 각 블록의 배치가 완료되면 작업자가 Enter 키를 입력하고, 로봇은 Observe 자세로 복귀한 뒤 다음 블록을 다시 검출하였다. 이 과정을 5개 블록에 대해 반복하여 자동 접근과 수동 조작을 포함하는 전체 작업을 하나의 연속 에피소드로 저장하였다.

수집된 기본 멀티태스크 데이터셋은 정리 353개 에피소드와 적층 222개 에피소드로 구성되며, 총 575개 에피소드와 932,830개 프레임을 포함한다. 정리 작업에서는 다섯 개의 지정 위치에 블록을 서로 겹치지 않게 배치하고, 적층 작업에서는 하나의 중앙 위치에서 블록을 수직으로 쌓는 행동을 기록하였다. 두 작업은 동일한 모델을 사용하되 자연어 지시문을 다르게 입력하여 작업 조건을 구분하였다.

최종 정책은 lerobot/smolvlr\_base를 기반으로 전체 미세조정을 수행하였다. 배치 크기 16, 총 300,000단계, 최대 학습률  $2 \times 10^{-5}$ , chunk\_size=50 및 n\_action\_steps=50을 사용하였다. 학습과 실행 과정에서 상단·손목 영상, 관절 상태, 자연어 지시문 및 시연 행동의 입력 구조를 일치시켰으며, GPU 서버에서 정책을 추론하고 Jetson Orin Nano에서 로봇 제어를 수행하는 비동기 구조를 구현하였다.

기본 575개 에피소드 모델의 내부 평가에서는 정리 작업의 5블록 전체 완료율 80%와 평균 작업 시간 약 70초를 기록하였다. 적층 작업에서는 중앙 상공 접근, 수직 하강 및 이전 블록 위에 배치하는 동작을 확인하였다. 이후 HIL 교정 데이터 129개를 추가하여 총 704개 에피소드로 데이터셋을 확장하고 후속 학습을 수행한 결과, 내부 평가에서 작업 성공률이 87%까지 개선되었다.

---

본 연구의 결과는 하나의 SmoIVLA 정책이 동일한 로봇과 동일한 실행 구조에서 자연어 지시문에 따라 정리와 적층이라는 서로 다른 조작 작업을 수행할 수 있음을 보여준다. 또한 실물 로봇에서는 단순히 모델의 크기나 학습 단계만 늘리는 것보다, 데이터 수집 방식, 영상 전처리, 관절 표현, 작업 지시문 및 실행 안전 계층을 함께 설계하는 것이 중요하다라는 점을 확인하였다.

다만 정리 작업의 지정 위치 분리, 적층 과정에서의 높이 누적 오차, 회전된 블록에 대한 정밀 파지 및 실패 상태에서의 안정적인 복구는 여전히 개선이 필요한 부분이다. 특히 학습 손실이 감소하더라도 특정 배치 위치에 대한 행동 편향이 나타날 수 있으므로, 향후에는 단순한 평균 손실뿐 아니라 위치별 배치 정확도와 실패 복구 성능을 함께 평가해야 한다.

## 5.2. Correction-only HIL의 의의와 한계

Correction-only HIL의 의의는 정책이 실제 실행 중 방문한 실패 상태에 대해, 해당 상태에서 수행해야 할 복구·교정 행동을 제공했다는 점에 있다. 정상 시연 중심의 학습에서는 충분히 경험하기 어려운 헛파지, 부분 파지, 블록 밀림 및 접근 오차를 추가 학습 대상으로 구성하였다. 이때 실패 상태 자체를 제거하지 않고, 실패를 유발한 자율 행동과 이후의 적절한 교정 행동을 구분하여 모방 대상으로 선택하였다.

이 방식은 정상 작업 시연과 복구 시연의 역할을 구분한다. 기존 성공 시연은 파지·이송·배치의 기본 수행 능력을 제공하고, 교정 시연은 실행 중 오차가 발생했을 때 재접근과 재정렬로 작업을 이어가는 행동을 제공한다. 또한 동일한 작업 지시문을 유지하여, 별도의 복구 명령 대신 현재 영상과 로봇 상태에 따라 행동을 전환하도록 학습을 구성하였다.

그러나 교정 구간만 수집하는 방식은 실패 이전의 자율 행동이 작업 결과에 미친 영향을 직접 평가하지 않는다. 잘못된 접근 이후의 복구 행동을 학습하더라도, 최초 접근 행동의 품질을 낮게 평가하는 별도의 학습 신호가 자동으로 제공되는 것은 아니다. 이러한 점에서 Correction-only HIL은 실패 상태의 대응 행동을 보완하는 방법이며, 자율 실행 행동 전체의 품질을 평가하는 방법과는 역할이 다르다.

교정 구간의 완전성과 학습 비중도 한계로 남는다. 파지 직후 종료된 시연은 들어 올리기와 이송으로 이어지는 연결을 충분히 제공하지 못할 수 있으며, 불필요한 정지 프레임이나 반복적인 재정렬이 많이 포함되면 정상 작업의 연속성이 약화될 수 있다.

---

또한 기본 시연과 교정 시연은 에피소드의 길이와 기록 단위가 다르므로, 에피소드 수뿐 아니라 프레임 수준의 학습 노출량을 함께 관리해야 한다.

따라서 Correction-only HIL의 효과는 복구 행동이 나타났는지만으로 판단하기 어렵다. 정상 조건에서의 파지와 전체 완료 성능을 유지하면서, 실패 상태에서의 재시도와 작업 재개 능력이 개선되는지를 함께 확인해야 한다. 본 연구는 이러한 검증을 위한 복구·교정 데이터 수집 및 추가 학습 구조를 구성하였으며, 정량적인 개선 정도는 모델별 평가 조건과 결과를 구분하여 해석해야 한다.

### 5.3. Reward-Weighted Flow Matching의 의의와 한계

RWFM의 의의는 수집된 행동을 동일한 모방 대상으로 사용하는 데서 나아가, 행동의 결과와 품질에 따라 학습 기여도를 조절하도록 구성했다는 점에 있다. 기존 성공 시연은 정상적인 작업 수행의 기준을 제공하고, Correction-only HIL은 실패 상태의 복구·교정 행동을 제공한다. RWFM은 이러한 행동과 자율 실행에서 얻은 행동의 상대적인 품질을 추가로 반영하는 역할을 담당한다.

특히 행동 체크 내부의 시점별 품질 정보를 사용하면 하나의 실행 구간에 섞인 정상 행동, 실패 유발 행동 및 복구 행동을 구분하여 반영할 수 있다. 이는 긴 롤아웃의 마지막 실패만으로 앞선 정상 구간까지 동일하게 평가하는 문제를 줄이기 위한 설계이다. 다만 시점별 가중치 에피소드 단위 가중치보다 우수한지는 동일 조건의 비교 실험으로 확인해야 한다.

RWFM의 효과는 품질 라벨의 정확성과 데이터 구성에 영향을 받는다. 실패가 관찰된 시점과 실패를 유발한 행동 시점이 다를 수 있으며, 성공한 복구 행동이라도 이동 시간이나 주변 블록과의 접촉 정도는 다를 수 있다. 따라서 단순한 성공·실패 구분뿐 아니라 구간 판정 기준과 시간 정렬의 일관성이 중요하다.

낮은 보상의 행동에 작은 양의 가중치를 적용하는 것은 해당 행동을 직접적으로 금지하거나 반대 방향으로 학습시키는 것과 다르다. 또한 복구 구간에 높은 가중치를 부여하더라도 정상 작업 능력이 자동으로 유지되지는 않는다. 복구 시연의 양과 길이, 가중치 및 기존 성공 시연과의 혼합 비율에 따라 재정렬·후퇴·정지 행동의 비중이 커질 수 있으므로, 정상 조건과 실패 조건의 수행 결과를 함께 평가해야 한다.

본 연구의 오프라인 RWFM 구성은 기록된 행동의 품질을 학습 비중에 반영하지만, 다음 상태와 미래 누적 보상을 이용해 행동의 장기적인 가치를 추정하지는 않는다. 따라서 즉시 작업을 진행하지 않는 복구 행동이 이후 성공에 얼마나 기여하는지, 또는 현재의

---

접근 행동이 여러 단계 뒤의 실패와 어떻게 연결되는지를 명시적으로 평가하는 데에는 한계가 있다. 이러한 한계는 후속 연구에서 Q 함수 기반 오프라인 강화학습과 새로운 상호작용 데이터를 이용한 정책 개선을 검토하는 근거가 된다.

#### 5.4. Q함수 기반 오프라인 강화학습과 시뮬레이션을 활용한 확장

본 연구의 후속 방향은 HIL과 보상 가중 Flow Matching(RWFM)을 통해 확보한 정책을 Q 함수 기반 오프라인 강화학습으로 확장하고, 이후 시뮬레이션 상호작용을 이용한 온라인 강화학습으로 복구 행동을 보강하는 것이다. 기존 시연을 활용한 초기 정책 학습과 새로운 행동 결과를 통한 정책 개선을 단계적으로 연결하고자 한다.

RWFM은 성공·부분 성공·실패에 부여한 보상 또는 품질 점수를 이용하여 시연 행동의 학습 비중을 조절한다. 따라서 실패 정보를 활용하지 않는 방식은 아니다. 다만 지수 가중치를 사용하면 낮은 보상의 실패 행동에도 양의 가중치가 남을 수 있으며, 마스킹은 해당 행동의 직접적인 모방 손실을 제외하는 역할을 한다. 현재의 RWFM은 다음 상태와 미래 보상을 이용한 가치 추정을 수행하지 않으므로, 어떤 행동이 장기적으로 실패를 유발하거나 복구 후 성공에 기여하는지를 명시적으로 평가하는 데에는 한계가 있다.

이를 보완하기 위해 상태와 행동에 따른 기대 누적 보상을 추정하는 Q-critic을 도입할 수 있다. 정상 시연과 HIL 데이터를 관측, 실제 실행 행동, 보상, 다음 관측 및 종료 여부로 구성된 전이 데이터로 정리하고, 이를 이용하여 행동의 장기적인 가치를 학습하는 방식이다. 예를 들어 즉시 작업을 진행하지 않는 상승·재정렬 행동도 이후 재파지와 작업 완료에 기여한다면 높은 가치를 가질 수 있다. 반대로 목표에 가까워지는 행동이라도 충돌이나 불안정한 배치를 유발한다면 낮게 평가하도록 보상을 설계할 수 있다.

이때 정책의 모방 학습에서 제외된 실패 행동도, 기록된 전이가 유효하다면 critic 학습에는 활용할 수 있다. 행동을 정답으로 모방하는 것과 그 행동의 결과를 평가하는 것은 서로 다른 역할이기 때문이다. 다만 HIL에서는 사람이 개입한 이후의 성공이 개입 이전 행동의 품질을 과대평가하게 만들지 않도록 제어권 전환을 기록하고, 보상과 정책 평가의 대상을 명확히 해야 한다. 복구 가능한 중간 실패를 모두 에피소드 종료로 처리하는 경우에도 실패 상태에서 성공으로 이어지는 연결을 학습하기 어려워지므로 종료 기준을 구분할 필요가 있다.

Flow Matching 정책과 가치 기반 학습을 결합하는 선행 사례로 Flow Q-Learning(FQL)이 있다. FQL은 반복적인 flow 생성 과정을 직접 가치 최대화로 최적화하는 어려움을 다루

---

기 위해 별도의 one-step 정책을 강화학습으로 학습한다. 이는 본 연구의 확장에 참고할 수 있는 방법이지만, SmolVLA에 critic을 추가하는 것만으로 동일한 알고리즘이 구현되는 것은 아니다. 관측 표현, 행동 청크의 정의 및 정책 갱신 방식을 별도로 설계해야 한다. [1](#)

특히 SmolVLA에서는 예측한 행동 청크와 실제 실행한 구간이 다를 수 있다. 따라서 critic의 평가 단위를 단일 행동 또는 실제 실행한 행동 구간으로 정하고, 해당 구간의 보상과 종료 시점의 관측을 일치시켜야 한다. 청크 단위로 평가하는 경우에는 구간 길이에 맞는 누적 보상과 할인율을 사용해야 한다. 또한 오프라인 데이터에 거의 없는 행동의 Q 값이 과대평가될 수 있으므로, Conservative Q-Learning(CQL)과 같은 보수적 가치 추정 또는 데이터 분포에서 과도하게 벗어나지 않도록 하는 정책 제약을 검토할 수 있다. [2](#)

오프라인 학습 이후에는 시뮬레이션에서 정책을 실행하여 새로운 전이를 수집하고, 그 결과를 바탕으로 온라인 강화학습을 수행하는 방향으로 확장할 수 있다. 여기서 온라인 학습은 실제 로봇에서의 실행만을 의미하지 않으며, 현재 정책이 시뮬레이터와 상호작용하여 새 데이터를 만들고 갱신하는 과정도 포함한다. 기존 시연과 새 상호작용 데이터를 함께 활용하는 접근은 RLPD와 같은 연구에서 제시되었다. 본 연구에서는 이를 참고하여 기존 작업 능력을 유지하면서 새로운 실패 조건에 적응하는 방법을 검토할 수 있다. [3](#)

시뮬레이션 환경에는 SO-101의 관절 제어, 그리퍼 형상, 블록의 접촉·마찰 특성 및 상단·손목 카메라를 반영한다. 초기에는 정상 파지와 배치를 중심으로 학습하고, 이후 블록 위치 편차, 밀집 간격 변화, 파지 미끄러짐 및 적층 정렬 오류를 점진적으로 추가하여 복구 행동을 학습시키는 방안을 고려한다. 학습한 정책의 실물 적용을 위해서는 영상 조건뿐 아니라 마찰, 질량, 제어 지연 등에도 변화를 주는 도메인 랜덤화가 필요하다. 시뮬레이션 강화학습과 물리 특성 랜덤화를 실제 조작으로 연결한 선행 연구는 이러한 방향의 가능성을 보여주지만, SO-101 환경에서의 전이 성능은 별도 실물 평가로 확인해야 한다. [4](#)

확장 효과는 기본 575개 데이터셋 모델, 일반 HIL 추가 학습 모델, RWFM 모델, Q-critic 기반 오프라인 강화학습 모델 및 시뮬레이션 온라인 추가 학습 모델을 단계적으로 비교하여 평가한다. 동일한 초기 배치와 평가 횟수, 카메라 및 제어 설정을 유지하고, 정상 조건과 실패 복구 조건을 분리하여 측정한다. 정리에서는 블록별 파지 성공률, 전체 완료율, 지정 위치 오차, 겹침 여부와 수행 시간을 평가한다. 적층에서는 층별 수평 오차, 누적 높이 오차, 타워 기울기, 안정 유지 여부와 수행 시간을 평가하며, 공통적으로 복구 성공률과 사람 개입 횟수를 기록한다.

이러한 확장의 목표는 낮은 품질의 행동을 적게 모방하는 단계에서 나아가, 행동이 이후 상태와 작업 성공에 미치는 영향을 평가하고 새로운 상호작용으로 정책을 개선하는 것이

---

다. 이를 통해 정상적인 정리·적층 능력의 유지 여부와 실패 복구 능력의 변화를 함께 검증하고자 한다.

## 6. 참고 문헌

- [1] M. Shukor et al., "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics," arXiv:2506.01844, 2025. <https://arxiv.org/abs/2506.01844>
- [2] Hugging Face, "LeRobot Documentation," [Online]. Available: <https://huggingface.co/docs/lerobot/> (accessed Sep. 11, 2026).
- [3] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," Proceedings of Robotics: Science and Systems XIX, 2023. doi:10.15607/RSS.2023.XIX.016.
- [4] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, PMLR, vol. 15, pp. 627–635, 2011. <https://proceedings.mlr.press/v15/ross11a.html>
- [5] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," International Conference on Learning Representations, 2023. arXiv:2210.02747. <https://arxiv.org/abs/2210.02747>
- [6] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," International Conference on Learning Representations, 2022. arXiv:2106.09685. <https://arxiv.org/abs/2106.09685>
- [7] Project Team Seonneomjima, "Intelligent Robotic Arm Control Using LeRobot SO-101," Graduation Project Interim Report, Department of Computer Science and Engineering, Pusan National University, 2026 (in Korean).
- [8] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. M. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," Proceedings of Robotics: Science and Systems XIX, 2023. doi:10.15607/RSS.2023.XIX.026.

- 
- [9] Ultralytics, "Ultralytics YOLO," [Online]. Available: <https://github.com/ultralytics/ultralytics> (accessed Sep. 11, 2026).
- [10] Hugging Face, "Human-In-the-Loop Data Collection," LeRobot Documentation, [Online]. Available: [https://huggingface.co/docs/lerobot/hil\\_data\\_collection](https://huggingface.co/docs/lerobot/hil_data_collection) (accessed Sep. 14, 2026).
- [11] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "HG-DAGger: Interactive Imitation Learning with Human Experts," arXiv:1810.02890, 2018. <https://arxiv.org/abs/1810.02890>
- [12] Z. Hu, R. Wu, N. Enock, J. Li, R. Kadakia, Z. Erickson, and A. Kumar, "RaC: Robot Learning for Long-Horizon Tasks by Scaling Recovery and Correction," arXiv:2509.07953, 2025. <https://arxiv.org/abs/2509.07953>
- [13] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, "Robot Learning on the Job: Human-in-the-Loop Autonomy and Learning During Deployment," arXiv:2211.08416, 2022. <https://arxiv.org/abs/2211.08416>
- [14] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, PMLR, vol. 15, pp. 627–635, 2011. <https://proceedings.mlr.press/v15/ross11a.html>
- [15] S. Park, Q. Li, and S. Levine, "Flow Q-Learning," Proceedings of the 42nd International Conference on Machine Learning, PMLR, vol. 267, pp. 48104–48127, 2025. <https://proceedings.mlr.press/v267/park25f.html>
- [16] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-Learning for Offline Reinforcement Learning," Advances in Neural Information Processing Systems, 2020. arXiv:2006.04779. <https://arxiv.org/abs/2006.04779>
- [17] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient Online Reinforcement Learning with Offline Data," Proceedings of the 40th International Conference on Machine Learning, PMLR, vol. 202, pp. 1577–1594, 2023. <https://proceedings.mlr.press/v202/ball23a.html>

- 
- [18] OpenAI et al., "Learning Dexterous In-Hand Manipulation," arXiv:1808.00177, 2018. <https://arxiv.org/abs/1808.00177>
- [19] Hugging Face, "SmolVLA: Efficient Vision-Language-Action Model trained on LeRobot Community Data," 2025. [Online]. Available: <https://huggingface.co/blog/smolvla> (accessed Sep. 14, 2026).
- [20] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [21] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, PMLR, vol. 15, pp. 627–635, 2011. <https://proceedings.mlr.press/v15/ross11a.html>
- [22] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "HG-Dagger: Interactive Imitation Learning with Human Experts," IEEE International Conference on Robotics and Automation, pp. 8077–8083, 2019. doi:10.1109/ICRA.2019.8793698. <https://doi.org/10.1109/ICRA.2019.8793698>
- [23] M. Shukor et al., "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics," arXiv:2506.01844, 2025. <https://arxiv.org/abs/2506.01844>
- [24] Hugging Face, "SmolVLA Policy Implementation: modeling\_smolvla.py," LeRobot GitHub Repository, [Online]. Available: [https://github.com/huggingface/lerobot/blob/main/src/lerobot/policies/smolvla/modeling\\_smolvla.py](https://github.com/huggingface/lerobot/blob/main/src/lerobot/policies/smolvla/modeling_smolvla.py) (accessed Sep. 14, 2026).
- [25] S. Pfrommer, Y. Huang, and S. Sojoudi, "Reinforcement Learning for Flow-Matching Policies," arXiv:2507.15073, 2025. <https://arxiv.org/abs/2507.15073>